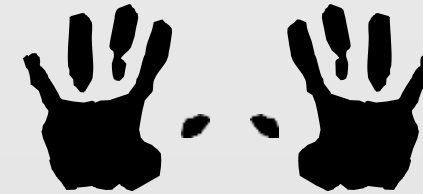


Number Systems

Decimal Number System

- ▮ Base (also called radix) = 10
 - ◆ 10 digits { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
- ▮ Digit Position
 - ◆ Integer & fraction
- ▮ Digit Weight
 - ◆ Weight = $(Base)^{Position}$
- ▮ Magnitude
 - ◆ Sum of “Digit x Weight”
- ▮ Formal Notation



2	1	0		-1	-2
5	1	2	.	7	4
<i>100</i>	<i>10</i>	<i>1</i>		<i>0.1</i>	<i>0.01</i>
			.		
<i>500</i>	<i>10</i>	<i>2</i>		<i>0.7</i>	<i>0.04</i>

$$d_2*B^2 + d_1*B^1 + d_0*B^0 + d_{-1}*B^{-1} + d_{-2}*B^{-2}$$

$$(512.74)_{10}$$

Octal Number System

▮ Base = 8

◆ 8 digits { 0, 1, 2, 3, 4, 5, 6, 7 }

▮ Weights

◆ Weight = $(Base)^{Position}$

▮ Magnitude

◆ Sum of “Digit x Weight”

▮ Formal Notation

64	8	1		1/8	1/64
5	1	2	.	7	4
2	1	0		-1	-2

$$\textcolor{red}{5}_2 * 8^2 + \textcolor{red}{1} * 8^1 + \textcolor{red}{2} * 8^0 + \textcolor{red}{7} * 8^{-1} + \textcolor{red}{4} * 8^{-2}$$

$$= (330.9375)_{10}$$

$$(\textcolor{red}{512.74})_8$$

Binary Number System

▮ Base = 2

◆ 2 digits { 0, 1 }, called *binary digits* or “*bits*”

▮ Weights

◆ Weight = (Base)^{Position}

▮ Magnitude

◆ Sum of “*Bit x Weight*”

▮ Formal Notation

▮ Groups of bits 4 bits = *Nibble*

8 bits = *Byte*

4	2	1		1/2	1/4
1	0	1	●	0	1
2	1	0		-1	-2

$$\frac{1}{2} * 2^2 + 0 * 2^1 + 1 * 2^0 + 0 * 2^{-1} + 1 * 2^{-2}$$

$$=(5.25)_{10}$$

$$(101.01)_2$$

1 0 1 1

1 1 0 0 0 1 0 1

Hexadecimal Number System

▮ Base = 16

◆ 16 digits { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F }

▮ Weights

◆ Weight = $(Base)^{Position}$

▮ Magnitude

◆ Sum of “Digit x Weight”

▮ Formal Notation

256	16	1		1/16	1/256
1	E	5	.	7	A
2	1	0		-1	-2

$$1 * 16^2 + 14 * 16^1 + 5 * 16^0 + 7 * 16^{-1} + 10 * 16^{-2}$$
$$=(485.4765625)_{10}$$
$$(1E5.7A)_{16}$$

The Power of 2

n	2^n
0	$2^0=1$
1	$2^1=2$
2	$2^2=4$
3	$2^3=8$
4	$2^4=16$
5	$2^5=32$
6	$2^6=64$
7	$2^7=128$



n	2^n
8	$2^8=256$
9	$2^9=512$
10	$2^{10}=1024$
11	$2^{11}=2048$
12	$2^{12}=4096$
20	$2^{20}=1\text{M}$
30	$2^{30}=1\text{G}$
40	$2^{40}=1\text{T}$

Kilo

Mega

Giga

Tera



Addition

Decimal Addition

$$\begin{array}{r}
 \text{1} \quad \text{1} \quad \quad \leftarrow \text{Carry} \\
 \quad 5 \quad 5 \\
 + \quad 5 \quad 5 \\
 \hline
 1 \quad 1 \quad 0
 \end{array}$$

$\swarrow$ $= \text{Ten} \geq \text{Base}$
→ Subtract a Base

Binary Addition

□ Column Addition

$$\begin{array}{r}
 \begin{array}{cccccc}
 1 & 1 & 1 & 1 & 1 & 1 \\
 & 1 & 1 & 1 & 1 & 0 & 1 \\
 + & & 1 & 0 & 1 & 1 & 1 \\
 \hline
 1 & 0 & 1 & 0 & 1 & 0 & 0
 \end{array}
 \end{array}
 \begin{array}{l}
 \\
 = 61 \\
 = 23 \\
 \\
 = 84
 \end{array}$$

$\geq (2)_{10}$




-
- $$\begin{array}{r}
 101002 \\
 - 02002 \\
 \hline
 011010
 \end{array}$$
- The result is $011010_2 = 54_{10}$.

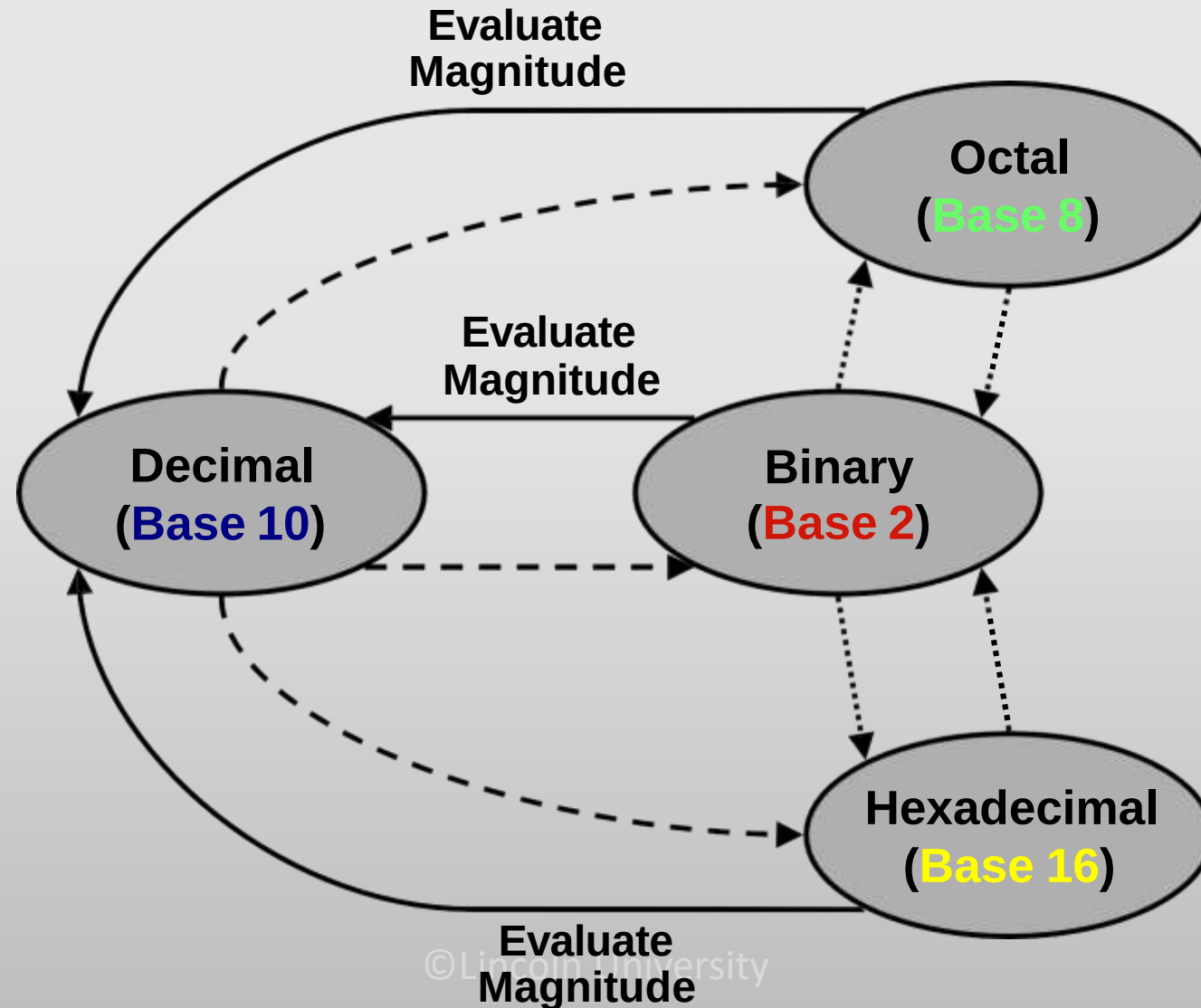
Binary Multiplication

□ Bit by bit

$$\begin{array}{r} 10111 \\ 1010 \\ \hline 00000 \\ 10111 \\ 00000 \\ 10111 \\ \hline 11100110 \end{array}$$



Number Base Conversions



Decimal (*Integer*) to Binary Conversion

- ▮ Divide the number by the 'Base' (=2)
- ▮ Take the remainder (either 0 or 1) as a coefficient
- ▮ Take the quotient and repeat the division

Example: $(13)_{10}$

	Quotient	Remainder	Coefficient
$13 / 2 =$	6	1	$a_0 = 1$
$6 / 2 =$	3	0	$a_1 = 0$
$3 / 2 =$	1	1	$a_2 = 1$
$1 / 2 =$	0	1	$a_3 = 1$

Answer: $(13)_{10} = (a_3 a_2 a_1 a_0)_2 = (1101)_2$

MSB

LSB

Decimal (*Fraction*) to Binary Conversion

- ▮ Multiply the number by the 'Base' (=2)
- ▮ Take the integer (either 0 or 1) as a coefficient
- ▮ Take the resultant fraction and repeat the process

Example: $(0.625)_{10}$

		Integer	Fraction	Coefficient
0.625	$* 2 =$	1	. 25	$a_{-1} = \mathbf{1}$
0.25	$* 2 =$	0	. 5	$a_{-2} = \mathbf{0}$
0.5	$* 2 =$	1	. 0	$a_{-3} = \mathbf{1}$

Answer: $(0.625)_{10} = (0.a_{-1} a_{-2} a_{-3})_2 = (0.101)_2$

↑
MSB

↑
LSB

Decimal to Octal Conversion

Example: $(175)_{10}$

	Quotient	Remainder	Coefficient
$175 / 8 =$	21	7	$a_0 = 7$
$21 / 8 =$	2	5	$a_1 = 5$
$2 / 8 =$	0	2	$a_2 = 2$

Answer: $(175)_{10} = (a_2 a_1 a_0)_8 = (257)_8$

Example: $(0.3125)_{10}$

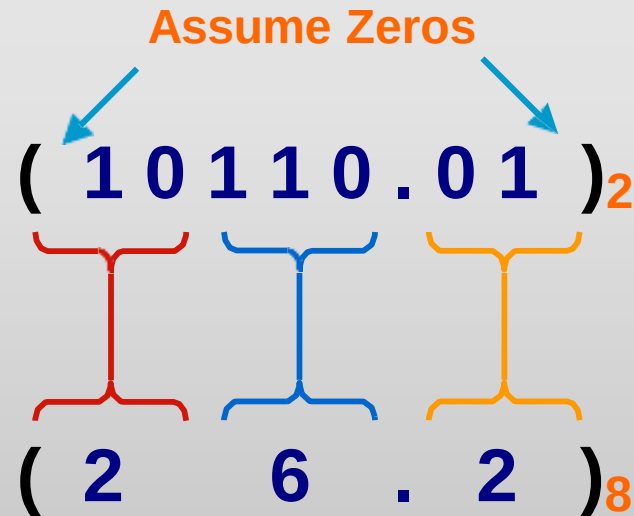
	Integer	Fraction	Coefficient
$0.3125 * 8 =$	2	5	$a_{-1} = 2$
$0.5 * 8 =$	4	0	$a_{-2} = 4$

Answer: $(0.3125)_{10} = (0.a_{-1} a_{-2} a_{-3})_8 = (0.24)_8$

Binary – Octal Conversion

- ▮ $8 = 2^3$
- ▮ Each group of 3 bits represents an octal digit

Example:



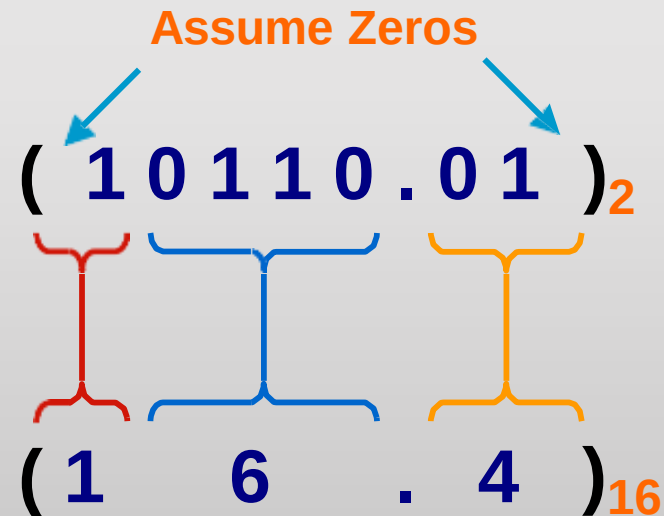
Octal	Binary
0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1

Works **both** ways (*Binary to Octal & Octal to Binary*)

Binary –Hexadecimal Conversion

- 16 = 2^4
- Each group of 4 bits represents a hexadecimal digit

Example:



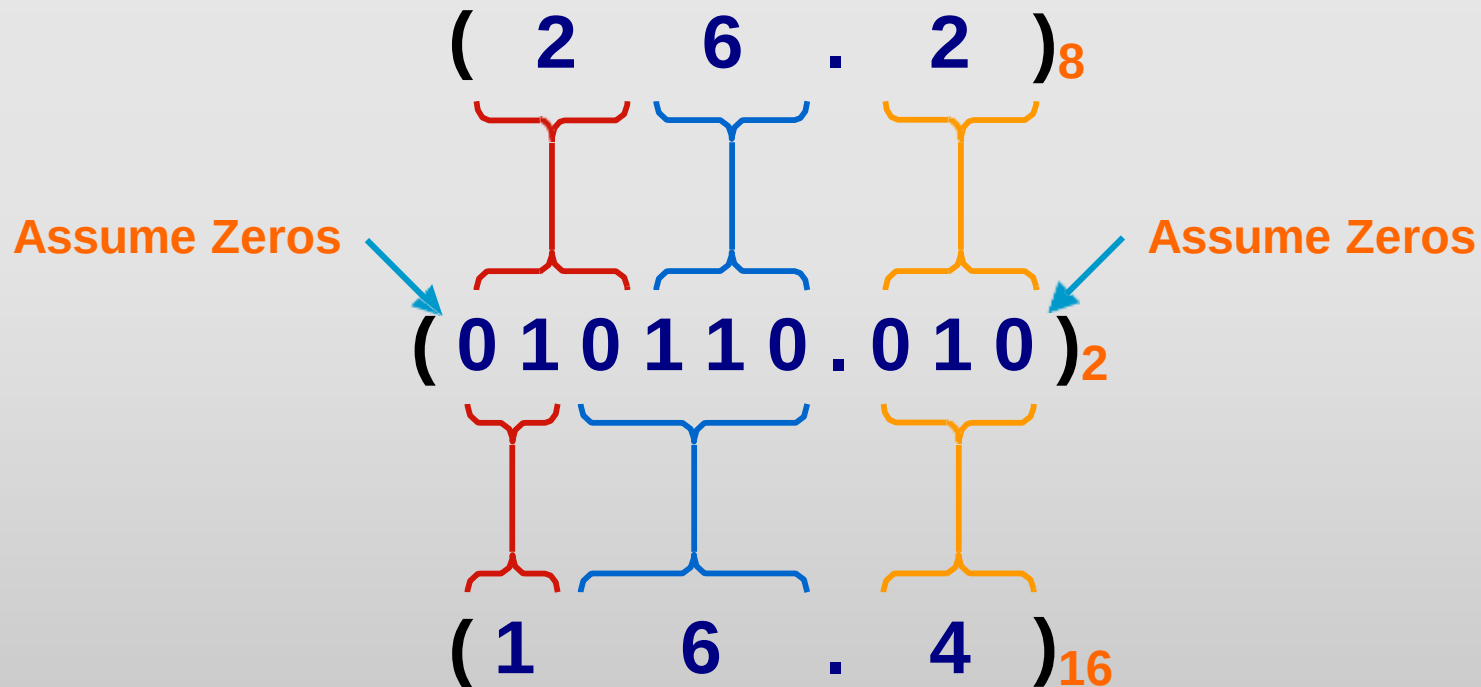
Hex	Binary
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
A	1 0 1 0
B	1 0 1 1
C	1 1 0 0
D	1 1 0 1
E	1 1 1 0
F	1 1 1 1

Works **both** ways (*Binary to Hex & Hex to Binary*)

Octal –Hexadecimal Conversion

- Convert to **Binary** as an intermediate step

Example:



Works **both** ways (*Octal to Hex & Hex to Octal*)

Decimal, Binary, Octal and Hexadecimal

Decimal	Binary	Octal	Hex
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F



Complements

- There are two types of complements for each base- r system: the radix complement and diminished radix complement.

- Diminished Radix Complement - $(r-1)$'s Complement**

- Given a number N in base r having n digits, the $(r-1)$'s complement of N is defined as:

$$(r^n - 1) - N$$

- Example for 6-digit decimal numbers:**

- 9's complement is $(r^n - 1) - N = (10^6 - 1) - N = 999999 - N$
- 9's complement of 546700 is $999999 - 546700 = 453299$

- Example for 7-digit binary numbers:**

- 1's complement is $(r^n - 1) - N = (2^7 - 1) - N = 1111111 - N$
- 1's complement of 1011000 is $1111111 - 1011000 = 0100111$

- Observation:**

- Subtraction from $(r^n - 1)$ will never require a borrow
- Diminished radix complement can be computed digit-by-digit
- For binary: $1 - 0 = 1$ and $1 - 1 = 0$

Complements

1's Complement (*Diminished Radix* Complement)

- ◆ All '0's become '1's
- ◆ All '1's become '0's

Example $(10110000)_2$
 $\Rightarrow (01001111)_2$

If you add a number and its 1's complement ...

$$\begin{array}{r}
 1\ 0\ 1\ 1\ 0\ 0\ 0\ 0 \\
 +\ 0\ 1\ 0\ 0\ 1\ 1\ 1\ 1 \\
 \hline
 1\ 1\ 1\ 1\ 1\ 1\ 1\ 1
 \end{array}$$

Complements

▮ Radix Complement

The r 's complement of an n -digit number N in base r is defined as $r^n - N$ for $N \neq 0$ and as 0 for $N = 0$. Comparing with the $(r - 1)$'s complement, we note that the r 's complement is obtained by adding 1 to the $(r - 1)$'s complement, since $r^n - N = [(r^n - 1) - N] + 1$.

▮ Example: Base-10

The 10's complement of 012398 is 987602
The 10's complement of 246700 is 753300

▮ Example: Base-2

The 2's complement of 1101100 is 0010100
The 2's complement of 0110111 is 1001001

Complements

2's Complement (*Radix Complement*)

◆ Take 1's complement then add 1

OR ◆ Toggle all bits to the left of the first '1' from the right

OR

Example:

Number:

1's Comp.:

$$\begin{array}{r}
 1\ 0\ 1\ 1\ 0\ 0\ 0\ 0 \\
 0\ 1\ 0\ 0\ 1\ 1\ 1\ 1 \\
 + \qquad \qquad \qquad 1 \\
 \hline
 0\ 1\ 0\ 1\ 0\ 0\ 0\ 0
 \end{array}$$

$$\begin{array}{r}
 1\ 0\ 1\ 1\ 0\ 0\ 0\ 0 \\
 0\ 1\ 0\ 0\ 1\ 1\ 1\ 1 \\
 + \qquad \qquad \qquad 1 \\
 \hline
 0\ 1\ 0\ 1\ 0\ 0\ 0\ 0
 \end{array}$$

Complements

▮ Subtraction with Complements

◆ The subtraction of two n -digit unsigned numbers $M - N$ in base r can be done as follows:

1. Add the minuend M to the r 's complement of the subtrahend N . Mathematically, $M + (r^n - N) = M - N + r^n$.
2. If $M \geq N$, the sum will produce an end carry r^n , which can be discarded; what is left is the result $M - N$.
3. If $M < N$, the sum does not produce an end carry and is equal to $r^n - (N - M)$, which is the r 's complement of $(N - M)$. To obtain the answer in a familiar form, take the r 's complement of the sum and place a negative sign in front.

Complements

Example 1.5

- Using 10's complement, subtract $72532 - 3250$.

	$M =$	72532
10's complement of	$N =$	<u>+ 96750</u>
	Sum =	169282
	Discard end carry $10^5 =$	<u>- 100000</u>
	Answer =	69282

Example 1.6

- Using 10's complement, subtract $3250 - 72532$.

	$M =$	03250
10's complement of	$N =$	<u>+ 27468</u>
	Sum =	30718

→ There is no end carry.

→ Therefore, the answer is $-(10\text{'s complement of } 30718) = -69282$.

Complements

Example 1.7

- Given the two binary numbers $X = 1010100$ and $Y = 1000011$, perform the subtraction (a) $X - Y$; and (b) $Y - X$, by using 2's complement.

(a)	$X =$	1010100
	2's complement of $Y =$	<u>+0111101</u>
	Sum =	10010001
	Discard end carry $2^7 =$	<u>-10000000</u>
	Answer. $X - Y =$	0010001

(b)	$Y =$	1000011
	2's complement of $X =$	<u>+ 0101100</u>
	Sum =	1101111



There is no end carry.
 Therefore, the answer is
 $Y - X = - (2\text{'s complement of } 1101111) = - 0010001.$

Complements

- ▮ Subtraction of unsigned numbers can also be done by means of the $(r - 1)$'s complement. Remember that the $(r - 1)$'s complement is one less than the r 's complement.
- ▮ Example 1.8
 - ◆ Repeat Example 1.7, but this time using 1's complement.

(a) $X - Y = 1010100 - 1000011$

$$\begin{array}{r}
 X = 1010100 \\
 \text{1's complement of } Y = \pm 0111100 \\
 \text{Sum} = 10010000 \\
 \text{End-around carry} = \quad + \quad 1 \\
 \hline
 \text{Answer. } X - Y = 0010001
 \end{array}$$

(b) $Y - X = 1000011 - 1010100$

$$\begin{array}{r}
 Y = 1000011 \\
 \text{1's complement of } X = + 0101011 \\
 \text{Sum} = 1101110
 \end{array}$$

There is no end carry,
 Therefore, the answer is $Y - X = - (1\text{'s complement of } 1101110) = - 0010001$.

Data Representation

The ways to represent the data are:

- ▣ Unsigned Representation
- ▣ Signed Representation
 - Signed magnitude representation
 - Signed 1's complement representation
 - Signed 2's complement Representation

1.6 Signed Binary Numbers

- ▮ To represent negative integers, we need a notation for negative values.
- ▮ It is customary to represent the sign with a bit placed in the leftmost position of the number since binary digits.
- ▮ The convention is to make the **sign bit 0 for positive** and **1 for negative**.
- ▮ Example:

Signed-magnitude representation:	10001001
Signed-1's-complement representation:	11110110
Signed-2's-complement representation:	11110111

Weighted and Non weighted Code

❖ Weighted Code:-

- In weighted code, each digit position has a weight or value. The sum of all digits multiplied by a weight gives the total amount being represented.
- We can express any decimal number in tens, hundreds, thousands and so on.
- Eg:- Decimal number 4327 can be written as
$$4327 = 4000 + 300 + 20 + 7$$
- In the power of 10, it becomes
$$4327 = 4(10^3) + 3(10^2) + 2(10^1) + 7(10^0)$$
- BCD or 8421 is a type of weighted code where each digit position is being assigned a specific weight.

❖ Non-weighted code:-

- In non-weighted code, there is no positional weight i.e. each position within the binary number is not assigned a prefixed value. No specific weights are assigned to bit position in non-weighted code.
- The non-weighted codes are:-
 - a) The Gray code
 - b) The Excess-3 code

Binary Codes

BCD or 8421 code:-

- ❖ It is composed of four bits representing the decimal digits 0 through 9.
- ❖ The 8421 indicates the binary weights of the four bits ($2^3, 2^2, 2^1, 2^0$).
- ❖ A number with k decimal digits will
- ❖ require $4k$ bits in BCD.
- ❖ Decimal 396 is represented in BCD with 12 bits as 0011 1001 0110, with each group of 4 bits representing one decimal digit.
- ❖ A decimal number in BCD is the same as its equivalent binary number only when the number is between 0 and 9.
- ❖ The binary combinations 1010 through 1111 are not used and have no meaning in BCD.

Table 1.4

Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

BCD or 8421 Code

- ❖ Convert following to BCD form
a) 473₁₀ b) 312₁₀ c) 257₁₀ d) 112₁₀
- ❖ Convert following BCD to their decimal equivalent
a) 10000110 b) 00110010.10010100
- ❖ Convert the following binary number numbers to their BCD equivalent.
a) 1101.01₂ b) 11.01₂
(Binary to decimal Decimal to BCD)

Comparison of BCD with Binary

1. BCD is less efficient than binary

eg:- $(78)_{10} = (0111\ 1000)_{BCD}$
 $= (1001110)_2$

To encode the same decimal number, BCD needs more no. of bits than binary. Hence BCD is less efficient as compared to Binary

2. BCD arithmetic is more complicated than binary arithmetic.

3. Advantage of a BCD code is that the conversion from decimal to BCD or vice versa is simple.

BCD or 8421 Code

Decimal	BCD
weight	8 4 2 1
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1

- Unused or Don't care conditions

- 1010

- 1011

- 1100

- 1101

- 1110

- 1111

Comparison of BCD with Binary

1. BCD is less efficient than binary

eg:- $(78)_{10} = (0111\ 1000)_{BCD}$
 $= (1001110)_2$

To encode the same decimal number, BCD needs more no. of bits than binary. Hence BCD is less efficient as compared to Binary

2. BCD arithmetic is more complicated than binary arithmetic.

3. Advantage of a BCD code is that the conversion from decimal to BCD or vice versa is simple.

8 4 -2 -1 Code

Decimal	84-2-1 code
weight	8 4 -2 -1
0	0 0 0 0
1	0 1 1 1
2	0 1 1 0
3	0 1 0 1
4	0 1 0 0
5	1 0 1 1
6	1 0 1 0
7	1 0 0 1
8	1 0 0 0
9	1 1 1 1

- Unused or Don't care conditions

- 0001

- 0010

- 0011

- 1100

- 1101

- 1110

2421 Code

Decimal	2421 Code
weight	2 4 2 1
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	1 0 1 1
6	1 1 0 0
7	1 1 0 1
8	1 1 1 0
9	1 1 1 1

- Unused or Don't care conditions

- 0101

- 0110

- 0111

- 1000

- 1001

- 1010

Excess-3 Code

Decimal	Excess-3 code
0	0 0 1 1
1	0 1 0 0
2	0 1 0 1
3	0 1 1 0
4	0 1 1 1
5	1 0 0 0
6	1 0 0 1
7	1 0 1 0
8	1 0 1 1
9	1 1 0 0

- Unused or Don't care conditions

- 0000

- 0001

- 0010

- 1101

- 1110

- 1111

Decimal	BCD	Excess-3	8 4 -2 -1	2 4 2 1
0	0 0 0 0	0 0 1 1	0 0 0 0	0 0 0 0
1	0 0 0 1	0 1 0 0	0 1 1 1	0 0 0 1
2	0 0 1 0	0 1 0 1	0 1 1 0	0 0 1 0
3	0 0 1 1	0 1 1 0	0 1 0 1	0 0 1 1
4	0 1 0 0	0 1 1 1	0 1 0 0	0 1 0 0
5	0 1 0 1	1 0 0 0	1 0 1 1	1 0 1 1
6	0 1 1 0	1 0 0 1	1 0 1 0	1 1 0 0
7	0 1 1 1	1 0 1 0	1 0 0 1	1 1 0 1
8	1 0 0 0	1 0 1 1	1 0 0 0	1 1 1 0
9	1 0 0 1	1 1 0 0	1 1 1 1	1 1 1 1

The Gray code

❖ Introduction:

It is non weighted code in which each number differs from previous number by a single bit.

or

Only one bit is changed as we move from one number to another successive number.

CD \ AB	00	01	11	10
00	0	1	2	3
01	7	6	5	4
11	8	9	10	11
10	15	14	13	12

The Excess-3 code

- ❖ It is an important BCD code , is a 4 bit code and used with BCD numbers
- ❖ To convert any decimal numbers into its excess-3 form ,add 3 to each decimal digit and then convert the sum to a BCD number
- ❖ As weights are not assigned, it is a kind of non weighted codes.
- ❖ Convert the following into Excess-3 number
a)149 b) 2546 c) 152 d) 2694
- ❖ Add the following numbers in excess-3 code
a) 108+789 b) 275+496

BCD addition

- ❖ Add two numbers as same as binary addition
 - Case 1: If the result is less than or equals to 9 and carry is zero then it is valid BCD.
 - Case 2: If result is greater than 9 and carry is zero then add 6 in four bit combination.
 - Case 3: If result is less than or equals to 9 but carry is 1 then add 6 in four bit combination.

4	0100	4	0100	8	1000
<u>+5</u>	<u>+0101</u>	<u>+8</u>	<u>+1000</u>	<u>+9</u>	<u>+1001</u>
9	1001	12	1100	17	10001
			<u>+0110</u>		<u>+0110</u>
			10010		10111

BCD Addition

Example:

◆ Consider the addition of $184 + 576 = 760$ in BCD:

BCD	1	1		
	0001	1000	0100	184
	<u>+ 0101</u>	<u>0111</u>	<u>0110</u>	<u>+576</u>
Binary sum	0111	10000	1010	
Add 6	<u> </u>	<u>0110</u>	<u>0110</u>	<u> </u>
BCD sum	0111	0110	0000	760

Binary Codes

▮ Other Decimal Codes

Error-Detection code

❖ Introduction:

Electric wires or other communication medium can transmit binary information from one location to another. Any external noise introduced into the physical communication medium may change some of the bits from 0 to 1 or vice versa. The purpose of an error-detection code is to detect such bit-reversal errors. One of the most common ways to achieve error detection is by means of a **parity bit**. A parity bit is the extra bit included to make the total number of 1's in the resulting code word either even or odd.

Even Parity : Here the total number of 1's in the message is made even

Odd Parity : Here the total number of 1's in the message is made odd

A message of 4-bits and a parity bit P are shown in the table below:

Odd parity		Even parity	
Message	P	Message	P
0000	1	0000	0
0001	0	0001	1
0010	0	0010	1
0011	1	0011	0
0100	0	0100	1
0101	1	0101	0
0110	1	0110	0
0111	0	0111	1
1000	0	1000	1
1001	1	1001	0
1010	1	1010	0
1011	0	1011	1
1100	1	1100	0
1101	0	1101	1
1110	0	1110	1
1111	1	1111	0

A message of 3-bits and a parity bit P are shown in the table below:

3-bit Message			Odd Parity Bit	Even Parity Bit
X	Y	Z		
0	0	0	1	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	0	1

Alpha-Numeric Codes

1. American Standard Code for Information Interchange (ASCII) Character Code

American Standard Code for Information Interchange.

A popular code used to represent information sent as character-based data.

It uses 7-bits to represent.

ASCII uses a linear ordering of letters.

ASCII has a maximum of 128 characters.

ASC

ASCII TABLE II

Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char	Decimal	Hexadecimal	Binary	Octal	Char
0	0	0	0	[NULL]	48	30	110000	60	0	96	60	1100000	140	~
1	1	1	1	[START OF HEADING]	49	31	110001	61	1	97	61	1100001	141	a
2	2	10	2	[START OF TEXT]	50	32	110010	62	2	98	62	1100010	142	b
3	3	11	3	[END OF TEXT]	51	33	110011	63	3	99	63	1100011	143	c
4	4	100	4	[END OF TRANSMISSION]	52	34	110100	64	4	100	64	1100100	144	d
5	5	101	5	[ENQUIRY]	53	35	110101	65	5	101	65	1100101	145	e
6	6	110	6	[ACKNOWLEDGE]	54	36	110110	66	6	102	66	1100110	146	f
7	7	111	7	[BELL]	55	37	110111	67	7	103	67	1100111	147	g
8	8	1000	10	[BACKSPACE]	56	38	111000	70	8	104	68	1101000	150	h
9	9	1001	11	[HORIZONTAL TAB]	57	39	111001	71	9	105	69	1101001	151	i
10	A	1010	12	[LINE FEED]	58	3A	111010	72	:	106	6A	1101010	152	j
11	B	1011	13	[VERTICAL TAB]	59	3B	111011	73	;	107	6B	1101011	153	k
12	C	1100	14	[FORM FEED]	60	3C	111100	74	<	108	6C	1101100	154	l
13	D	1101	15	[CARRIAGE RETURN]	61	3D	111101	75	=	109	6D	1101101	155	m
14	E	1110	16	[SHIFT OUT]	62	3E	111110	76	>	110	6E	1101110	156	n
15	F	1111	17	[SHIFT IN]	63	3F	111111	77	?	111	6F	1101111	157	o
16	10	10000	20	[DATA LINK ESCAPE]	64	40	1000000	100	@	112	70	1110000	160	p
17	11	10001	21	[DEVICE CONTROL 1]	65	41	1000001	101	A	113	71	1110001	161	q
18	12	10010	22	[DEVICE CONTROL 2]	66	42	1000010	102	B	114	72	1110010	162	r
19	13	10011	23	[DEVICE CONTROL 3]	67	43	1000011	103	C	115	73	1110011	163	s
20	14	10100	24	[DEVICE CONTROL 4]	68	44	1000100	104	D	116	74	1110100	164	t
21	15	10101	25	[NEGATIVE ACKNOWLEDGE]	69	45	1000101	105	E	117	75	1110101	165	u
22	16	10110	26	[SYNCHRONOUS IDLE]	70	46	1000110	106	F	118	76	1110110	166	v
23	17	10111	27	[END OF TRANS. BLOCK]	71	47	1000111	107	G	119	77	1110111	167	w
24	18	11000	30	[CANCEL]	72	48	1001000	110	H	120	78	1111000	170	x
25	19	11001	31	[END OF MEDIUM]	73	49	1001001	111	I	121	79	1111001	171	y
26	1A	11010	32	[SUBSTITUTE]	74	4A	1001010	112	J	122	7A	1111010	172	z
27	1B	11011	33	[ESCAPE]	75	4B	1001011	113	K	123	7B	1111011	173	{
28	1C	11100	34	[FILE SEPARATOR]	76	4C	1001100	114	L	124	7C	1111100	174	
29	1D	11101	35	[GROUP SEPARATOR]	77	4D	1001101	115	M	125	7D	1111101	175	}
30	1E	11110	36	[RECORD SEPARATOR]	78	4E	1001110	116	N	126	7E	1111110	176	~
31	1F	11111	37	[UNIT SEPARATOR]	79	4F	1001111	117	O	127	7F	1111111	177	[DEL]
32	20	100000	40	[SPACE]	80	50	1010000	120	P					
33	21	100001	41	!	81	51	1010001	121	Q					
34	22	100010	42	"	82	52	1010010	122	R					
35	23	100011	43	#	83	53	1010011	123	S					
36	24	100100	44	\$	84	54	1010100	124	T					
37	25	100101	45	%	85	55	1010101	125	U					
38	26	100110	46	&	86	56	1010110	126	V					
39	27	100111	47	'	87	57	1010111	127	W					
40	28	101000	50	(	88	58	1011000	130	X					
41	29	101001	51	)	89	59	1011001	131	Y					
42	2A	101010	52	*	90	5A	1011010	132	Z					
43	2B	101011	53	+	91	5B	1011011	133	[					
44	2C	101100	54	,	92	5C	1011100	134	\					
45	2D	101101	55	-	93	5D	1011101	135	]					
46	2E	101110	56	.	94	5E	1011110	136	^					
47	2F	101111	57	/	95	5F	1011111	137	_					

ASCII Character

Letter	ASCII Code	Binary	Letter	ASCII Code	Binary
a	097	01100001	A	065	01000001
b	098	01100010	B	066	01000010
c	099	01100011	C	067	01000011
d	100	01100100	D	068	01000100
e	101	01100101	E	069	01000101
f	102	01100110	F	070	01000110
g	103	01100111	G	071	01000111
h	104	01101000	H	072	01001000
i	105	01101001	I	073	01001001
j	106	01101010	J	074	01001010
k	107	01101011	K	075	01001011
l	108	01101100	L	076	01001100
m	109	01101101	M	077	01001101
n	110	01101110	N	078	01001110
o	111	01101111	O	079	01001111
p	112	01110000	P	080	01010000
q	113	01110001	Q	081	01010001
r	114	01110010	R	082	01010010
s	115	01110011	S	083	01010011
t	116	01110100	T	084	01010100
u	117	01110101	U	085	01010101
v	118	01110110	V	086	01010110
w	119	01110111	W	087	01010111
x	120	01111000	X	088	01011000
y	121	01111001	Y	089	01011001
z	122	01111010	Z	090	01011010

Alpha-Numeric Codes

2. Extended Binary Coded Decimal Interchange Code(EBCDIC)

It uses 8-bits to represent.

It does not use a linear ordering of letters.

It has a maximum of 256 characters.

Each code is shown in decimal, hexadecimal, and character form.

129	81	a	193	C1	A	240	F0	0
130	82	b	194	C2	B	241	F1	1
131	83	c	195	C3	C	242	F2	2
132	84	d	196	C4	D	243	F3	3
133	85	e	197	C5	E	244	F4	4
134	86	f	198	C6	F	245	F5	5
135	87	g	199	C7	G	246	F6	6
136	88	h	200	C8	H	247	F7	7
137	89	i	201	C9	I	248	F8	8
						249	F9	9
145	91	j	209	D1	J	64	40	blank
146	92	k	210	D2	K	76	4C	<
147	93	l	211	D3	L	77	4D	(
148	94	m	212	D4	M	78	4E	+
149	95	n	213	D5	N	79	45	
150	96	o	214	D6	O	80	50	&
151	97	p	215	D7	P	90	5A	!
152	98	q	216	D8	Q	91	5B	\$
153	99	r	217	D9	R	92	5C	*
						93	5D	)
162	A2	s	226	E2	S	94	5E	;
163	A3	t	227	E3	T	96	60	-
164	A4	u	228	E4	U	97	61	/
165	A5	v	229	E5	V	107	6B	,
166	A6	w	230	E6	W	108	6C	%
167	A7	x	231	E7	X	109	6D	_
168	A8	y	232	E8	Y	110	6E	>
169	A9	z	233	E9	Z	111	6F	?
122	7A	:	125	7D	,			
123	7B	#	126	7E	=			
124	7C	@	127	7F	"			

What are Gates?

- The building blocks that creates digital circuits are logic gates.
- Each gate has its own logic symbol which allows complex functions to be represented by a logic diagram
- The function of each gate can be represented by a truth table or using Boolean notation

LOGIC GATES

- Types of gates
 - NOT
 - AND
 - OR
 - NAND
 - NOR
 - EX-OR
 - EX-NOR
 - BUFFER GATE

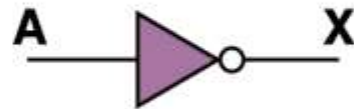
NOT Gate

- A NOT gate accepts one input value and produces one output value
- By definition, if the input value for a NOT gate is 0, the output value is 1, and if the input value is 1, the output is 0
- A NOT gate is sometimes referred to as an *inverter* because it inverts the input value

Boolean Expression

$$X = A'$$

Logic Diagram Symbol

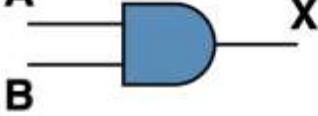


Truth Table

A	X
0	1
1	0

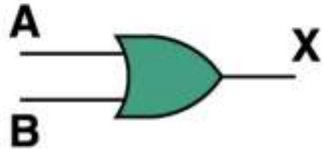
AND Gate

- An AND gate accepts two input signals
- If the two input values for an AND gate are both 1, the output is 1; otherwise, the output is 0

Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = A \cdot B$		<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	0	1	0	1	0	0	1	1	1
A	B	X															
0	0	0															
0	1	0															
1	0	0															
1	1	1															

OR Gate

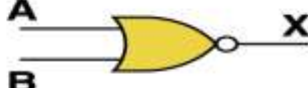
- If the two input values are both 0, the output value is 0; otherwise, the output is 1

Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = A + B$		<table><tr><th>A</th><th>B</th><th>X</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	X	0	0	0	0	1	1	1	0	1	1	1	1
A	B	X															
0	0	0															
0	1	1															
1	0	1															
1	1	1															

NAND and NOR Gates

- The NAND and NOR gates are essentially the opposite of the AND and OR gates, respectively.
- They are also called universal gates.

Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = (A \cdot B)'$		<table> <tr> <th>A</th><th>B</th><th>X</th></tr> <tr> <td>0</td><td>0</td><td>1</td></tr> <tr> <td>0</td><td>1</td><td>1</td></tr> <tr> <td>1</td><td>0</td><td>1</td></tr> <tr> <td>1</td><td>1</td><td>0</td></tr> </table>	A	B	X	0	0	1	0	1	1	1	0	1	1	1	0
A	B	X															
0	0	1															
0	1	1															
1	0	1															
1	1	0															

Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = (A + B)'$		<table> <tr> <th>A</th><th>B</th><th>X</th></tr> <tr> <td>0</td><td>0</td><td>1</td></tr> <tr> <td>0</td><td>1</td><td>0</td></tr> <tr> <td>1</td><td>0</td><td>0</td></tr> <tr> <td>1</td><td>1</td><td>0</td></tr> </table>	A	B	X	0	0	1	0	1	0	1	0	0	1	1	0
A	B	X															
0	0	1															
0	1	0															
1	0	0															
1	1	0															

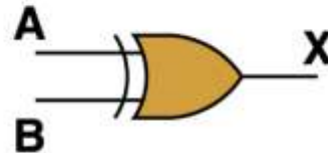
XOR Gate

- XOR, or *exclusive* OR, gate
 - An XOR gate produces 0 if its two inputs are the same, and a 1 otherwise

Boolean Expression

$$X = A \oplus B$$

Logic Diagram Symbol

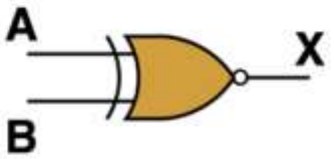


Truth Table

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

XNOR Gate

- XNOR, or *exclusive NOR*, gate
 - An XNOR gate produces 0 if its two inputs are the different, and a 1 otherwise

Boolean Expression	Logic Diagram Symbol	Truth Table															
$X = A \odot B$		<table> <tr> <th>A</th><th>B</th><th>X</th></tr> <tr> <td>0</td><td>0</td><td>1</td></tr> <tr> <td>0</td><td>1</td><td>0</td></tr> <tr> <td>1</td><td>0</td><td>0</td></tr> <tr> <td>1</td><td>1</td><td>1</td></tr> </table>	A	B	X	0	0	1	0	1	0	1	0	0	1	1	1
A	B	X															
0	0	1															
0	1	0															
1	0	0															
1	1	1															

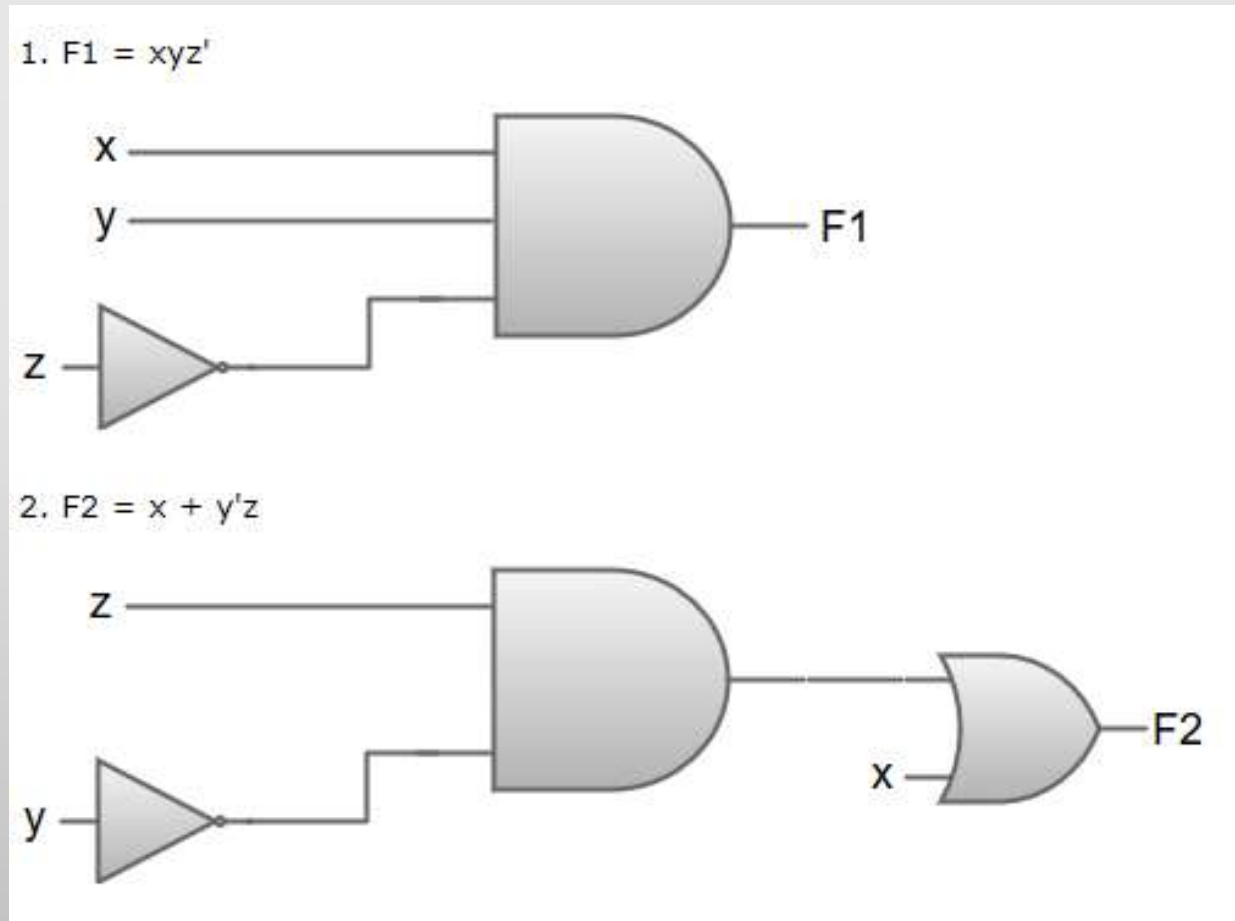
Boolean Algebra

- Mathematical system, formulate logic with symbols.
- Mathematics of digital system.
- Algebraic Manipulation

Boolean Function

- Expression using binary operators.
- Bracket, NOT, AND, OR
- Eg; $F = x.y'.z$

Boolean algebra simplifications using logic gates



Theorems

Boolean Expression
$x * 0 = 0$
$x + 1 = 1$
$x * 1 = x$
$x + 0 = x$
$x * x = x$
$x + x = x$
$x * x' = 0$
$x + x' = 1$
$(x')' = x$

Theorems

Commutative Law

It states that the interchanging of the order of operands in a Boolean equation does not change its result. For example:

OR operator $\rightarrow A + B = B + A$

AND operator $\rightarrow A \cdot B = B \cdot A$

Associative Law

It states that the AND operation are done on two or more than two variables. For example:

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C \quad A + (B + C) = (A + B) + C$$

Distributive Law

It states that the multiplication of two variables and adding the result with a variable will result in the same value as multiplication of addition of the variable with individual variables. For example:

$$A + BC = (A + B) (A + C) \quad \text{and} \quad A \cdot (B + C) = AB + AC$$

Theorems

De Morgan's Theorem

$$(A+B)' = A'.B' \quad (A+B+C)' = A'.B'.C'$$

$$(A.B)' = A'+B' \quad (A.B.C)' = A'+B'+C'$$

Duality Principle:

To find the dual of any expression:

- change the operator
- $0 \rightarrow 1, 1 \rightarrow 0$

Eg: $a+a'=1$ dual : $a.a'=0$

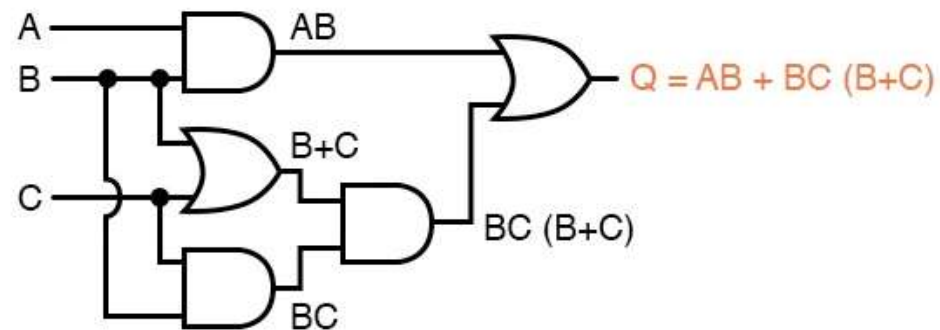
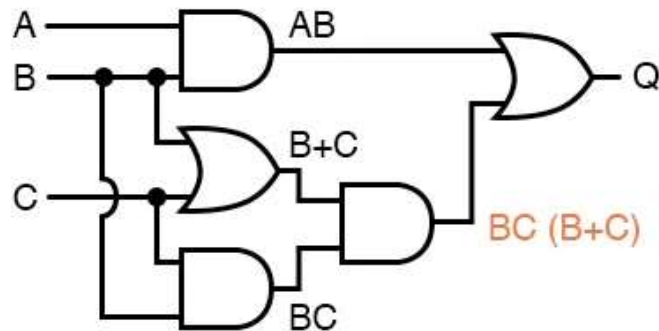
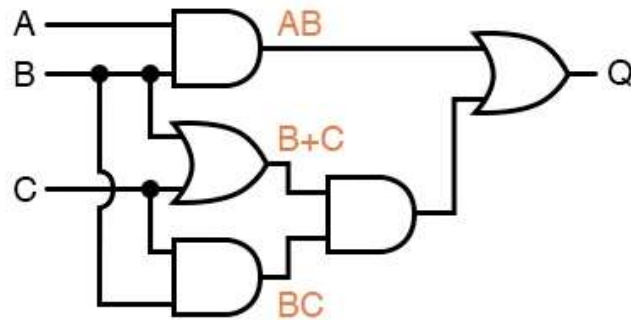
How to find complement?

- find dual
- complement each variable.

Eg: $F = x+y$ 1. dual = $x.y$ 2. $F' = x'.y'$

Gate Implementation

$$Q = A.B + B.C.(B+C)$$



Simplification of Boolean Functions

Simplify the following Boolean functions to a minimum number of literals.

- $x+x'.y = (x+x').(x+y) = 1.(x+y) = (x+y)$

- $x(x'+y) = x.x' + xy = 0+xy = xy$

- $x'y'z + x'yz + xy' = x'z(y+y') + xy' = x'z+xy'$

- $xy + x'z + yz = xy+yz +x'z +x.x' = y(x+z)+x'(x+z) = (x+z)(y+x')$

- $(x+y)(x'+z)(y+z)$
 $= (x+y)(x'+z)(y+z+x.x')$ [x.x' = 0]
 $= (x+y)(x'+z)(y+z+x) (y+z+x')$ [distributive property]
 $= (x+y)(x'+z)(x+y+z) (x'+y+z)$
Let; (x+y)=A and (x'+z)=B
 $= A.B.(A+z).(B+y)$
 $= A.(A+z).B.(B+y) = A.A+A.z.BB+By = A+A.z.B+By = A(1+z).B(1+y)$
 $= A.B$
 $= (x+y).(x'+z)$

Find the complement of $F = x + y'z$.
Also prove $F.F' = 0$ and $F + F' = 1$.

Implement $F = xy + x'y' + yz'$

- using AND, OR and NOT gates
- using NOT and OR only
- using NOT and AND only

$$Q = A'B' + A'C + B'C'$$

$$Q = A'C + B'C'$$

Redundant Theorem:

1. 3 variables
2. Each variable repeated twice
3. One variable must be complemented

- Take complemented variable and remaining is redundant.

$$F = AB + BC' + AC = BC' + AC$$

$$F = (A+B).(A'+C).(B+C) = (A+B).(A'+C)$$

Find the complement of $F = x + y'z$.
 Also prove $F.F' = 0$ and $F + F' = 1$.

$$F' = (x + y'z)' = x' \cdot (y'z)' = x' \cdot (y + z') = x'y + x'z'$$

To prove: $F.F' = 0$

LHS. $(x + y'z) \cdot (x'y + x'z')$

To prove: $F + F' = 1$

LHS.
$$\begin{aligned} & x + y'z + x'y + x'z' \\ &= x + x'y + y'z + x'z' \\ &= x + y + y'z + x'z' \\ &= x + x'z' + y + y'z \\ &= A + 1 \\ &= 1 \end{aligned}$$

$$F + F' = [(F + F')]' = [(F'.F)]' = [0]' = 1$$

Canonical and Standard Forms

- Boolean functions expressed as a sum of minterms or product of maxterms are said to be in **canonical**.
 - Minterms and Maxterms
 - Sum-of-Minterms and Product-of- Maxterms
 - Product and Sum terms
 - Sum-of-Products (SOP) and Product-of-Sums (POS)

No of minterms = no of maxterms = 2^n

Definitions

- *Literal*: A variable or its complement
- *Product term*: literals connected by •
- *Sum term*: literals connected by +
- *Minterm*: a product term in which all the variables appear exactly once, either complemented or uncomplemented
- *Maxterm*: a sum term in which all the variables appear exactly once, either complemented or uncomplemented

Minterm

- Represents exactly one combination in the truth table.
- Denoted by m_j , where j is the decimal equivalent of the minterm's corresponding binary combination (b_j).
- A variable in m_j is complemented if its value in b_j is 0, otherwise is uncomplemented.
- Example: Assume 3 variables (A,B,C), and $j=3$. Then, $b_j = 011$ and its corresponding minterm is denoted by $m_j = A'BC$

Maxterm

- Represents exactly one combination in the truth table.
- Denoted by M_j , where j is the decimal equivalent of the maxterm's corresponding binary combination (b_j).
- A variable in M_j is complemented if its value in b_j is 1, otherwise is uncomplemented.
- Example: Assume 3 variables (A,B,C), and $j=3$. Then, $b_j = 011$ and its corresponding maxterm is denoted by $M_j = A+B'+C'$

Truth Table notation for Minterms and Maxterms

- Minterms and Maxterms are easy to denote using a truth table.
- Example:
Assume 3 variables x, y, z (order is fixed)

Minterms: $x=0$; x'

$x=1$; x

And vice versa for maxterm

x	y	z		Minterm	Maxterm
0	0	0		$x'y'z' = m_0$	$x+y+z = M_0$
0	0	1		$x'y'z = m_1$	$x+y+z' = M_1$
0	1	0		$x'yz' = m_2$	$x+y'+z = M_2$
0	1	1		$x'yz = m_3$	$x+y'+z' = M_3$
1	0	0		$xy'z' = m_4$	$x'+y+z = M_4$
1	0	1		$xy'z = m_5$	$x'+y+z' = M_5$
1	1	0		$xyz' = m_6$	$x'+y'+z = M_6$
1	1	1		$xyz = m_7$	$x'+y'+z' = M_7$

Canonical Forms

- Any Boolean function $F()$ can be expressed as a *unique* **sum** of **minterms** and a unique **product** of **maxterms** (under a fixed variable ordering).
- In other words, every function $F()$ has two canonical forms:
 - Canonical Sum-Of-Products (sum of minterms)
 - Canonical Product-Of-Sums (product of maxterms)

Canonical Forms (cont.)

- Canonical Sum-Of-Products:
The minterms included are those m_j such that $F() = 1$ in row j of the truth table for $F()$.
- Canonical Product-Of-Sums:
The maxterms included are those M_j such that $F() = 0$ in row j of the truth table for $F()$.

Example

- Truth table for $f_1(a,b,c)$ at right
- The canonical sum-of-products form for f_1 is

$$f_1(a,b,c) = m_1 + m_2 + m_4 + m_6$$

$$= a'b'c + a'bc' + ab'c' + abc'$$
- The canonical product-of-sums form for f_1 is

$$f_1(a,b,c) = M_0 \cdot M_3 \cdot M_5 \cdot M_7$$

$$= (a+b+c) \cdot (a+b'+c') \cdot$$

$$(a'+b+c') \cdot (a'+b'+c').$$
- Observe that: $m_j = M_j'$

a	b	c		f_1
0	0	0		0
0	0	1		1
0	1	0		1
0	1	1		0
1	0	0		1
1	0	1		0
1	1	0		1
1	1	1		0

Shorthand: $\sum$ and $\prod$

- $f_1(a,b,c) = \sum m(1,2,4,6)$, where $\sum$ indicates that this is a sum-of-products form, and $m(1,2,4,6)$ indicates that the minterms to be included are m_1 , m_2 , m_4 , and m_6 .
- $f_1(a,b,c) = \prod M(0,3,5,7)$, where $\prod$ indicates that this is a product-of-sums form, and $M(0,3,5,7)$ indicates that the maxterms to be included are M_0 , M_3 , M_5 , and M_7 .
- Since $m_j = M_j'$ for any j ,
$$\sum m(1,2,4,6) = \prod M(0,3,5,7) = f_1(a,b,c)$$

Conversion Between Canonical Forms

- Replace $\sum$ with $\prod$ (or *vice versa*) and replace those j 's that appeared in the original form with those that do not.

- Example:

$$\begin{aligned}f_1(a,b,c) &= a'b'c + a'bc' + ab'c' + abc' \\&= m_1 + m_2 + m_4 + m_6 \\&= \sum(1,2,4,6) \\&= \prod(0,3,5,7) \\&= (a+b+c) \cdot (a+b'+c') \cdot (a'+b+c') \cdot (a'+b'+c')\end{aligned}$$

Conversion to SOP

- Expand *non-canonical* terms by inserting equivalent of 1 in each missing variable x:
 $(x + x') = 1$
- Remove duplicate minterms
- $f_1(a,b,c) = a'b'c + bc' + ac'$
 $= a'b'c + (a+a')bc' + a(b+b')c'$
 $= a'b'c + abc' + a'bc' + abc' + ab'c'$
 $= a'b'c + abc' + a'bc + ab'c'$

Conversion to POS

- Expand noncanonical terms by adding 0 in terms of missing variables (*e.g.*, $xx' = 0$) and using the distributive law
- Remove duplicate maxterms
- $$\begin{aligned} f_1(a,b,c) &= (a+b+c) \bullet (b'+c') \bullet (a'+c') \\ &= (a+b+c) \bullet (aa'+b'+c') \bullet (a'+bb'+c') \\ &= (a+b+c) \bullet (a+b'+c') \bullet (a'+b'+c') \bullet \\ &\quad (a'+b+c') \bullet (a'+b'+c') \\ &= (a+b+c) \bullet (a+b'+c') \bullet (a'+b'+c') \bullet (a'+b+c') \end{aligned}$$

$$F(A,B,C)=A+B'C$$

For SOP conversion;

$$F=A+B'C$$

$$F=A(B+B')(C+C')+B'C(A+A')$$

$$F= (AB+AB')(C+C')+ AB'C+A'B'C$$

$$F= ABC + ABC' + AB'C + AB'C' + AB'C+A'B'C$$

$$F= ABC + ABC' + AB'C + AB'C' +A'B'C$$

$$F = \sum m(\dots)$$

$$F(A,B,C)=A+B'C$$

For POS conversion;

$$F=A+B'C$$

$$F=(A+B').(A+C) \quad [\text{Similar to POS}]$$

$$F = (A+B'+C.C').(A+B.B'+C)$$

$$F = (A+B'+C).(A+B'+C').(A+B+C).(A+B'+C)$$

$$F = (A+B'+C).(A+B'+C').(A+B+C)$$

$$F = \prod M(\dots)$$

Karnaugh Maps

- Karnaugh maps (K-maps) are *graphical* representations of boolean functions.
- One **cell** corresponds to a row in the truth table.
- Also, one cell corresponds to a minterm or a maxterm in the boolean expression
- Multiple-cell areas of the map correspond to standard terms.

Minimization by K-map (Karnaugh map)

- Algebraic minimization of Boolean functions is rather awkward because it lacks specific rules to predict each succeeding step in the manipulative process.
- The map method provides a simple straightforward procedure for minimizing Boolean functions. This method may be regarded as a pictorial form of a truth table.
- The map method, first proposed by Veitch and modified by Karnaugh, is also known as the "Veitch diagram" or the "Karnaugh map."
 - The k-map is a diagram made up of grid of squares.
 - Each square represents one minterm.
 - The minterms are ordered according to Gray code (only one variable changes between adjacent squares).
 - Squares on edges are considered adjacent to squares on opposite edges.

Minimization by K-map (Karnaugh map)

- Karnaugh maps become clumsier to use with more than 4 variables.
- In fact, the map presents a visual diagram of all possible ways a function may be expressed in a standard form. By recognizing various patterns, the user can derive alternative algebraic expressions for the same function, from which he can select the simplest one.
- We shall assume that the simplest algebraic expression is anyone in a sum of products or product of sums that has a minimum number of literals. (This expression is not necessarily unique)

Two-Variable Map

		x_2			
		0	1		
x_1	0	0 m_0	1 m_1		
	1	2 m_2	3 m_3		

OR

		x_1			
		0	1		
x_2	0	0 m_0	2 m_2		
	1	1 m_1	3 m_3		

NOTE: ordering of variables is **IMPORTANT** for $f(x_1, x_2)$, x_1 is the row, x_2 is the column.

Cell 0 represents $x_1'x_2'$; Cell 1 represents $x_1'x_2$; etc. If a minterm is present in the function, then a 1 is placed in the corresponding cell.

Two-Variable Map (cont.)

- Any two adjacent cells in the map differ by ONLY one variable, which appears complemented in one cell and uncomplemented in the other.
- Example:
 $m_0 (=x_1'x_2')$ is adjacent to $m_1 (=x_1'x_2)$ and $m_2 (=x_1x_2')$ but NOT $m_3 (=x_1x_2)$
- There is only one possibility of grouping 4 adjacent min terms.
- The possible combinations of grouping 2 adjacent min terms are $\{(m_0, m_1), (m_2, m_3), (m_0, m_2) \text{ and } (m_1, m_3)\}$.

2-Variable Map -- Example

- $f(x_1, x_2) = x_1'x_2' + x_1'x_2 + x_1x_2'$
 $= m_0 + m_1 + m_2$
 $= x_1' + x_2'$
- 1s placed in K-map for specified minterms m_0, m_1, m_2
- Grouping (ORing) of 1s allows simplification
- What (simpler) function is represented by each dashed rectangle?
 - $x_1' = m_0 + m_1$
 - $x_2' = m_0 + m_2$
- Note m_0 covered twice

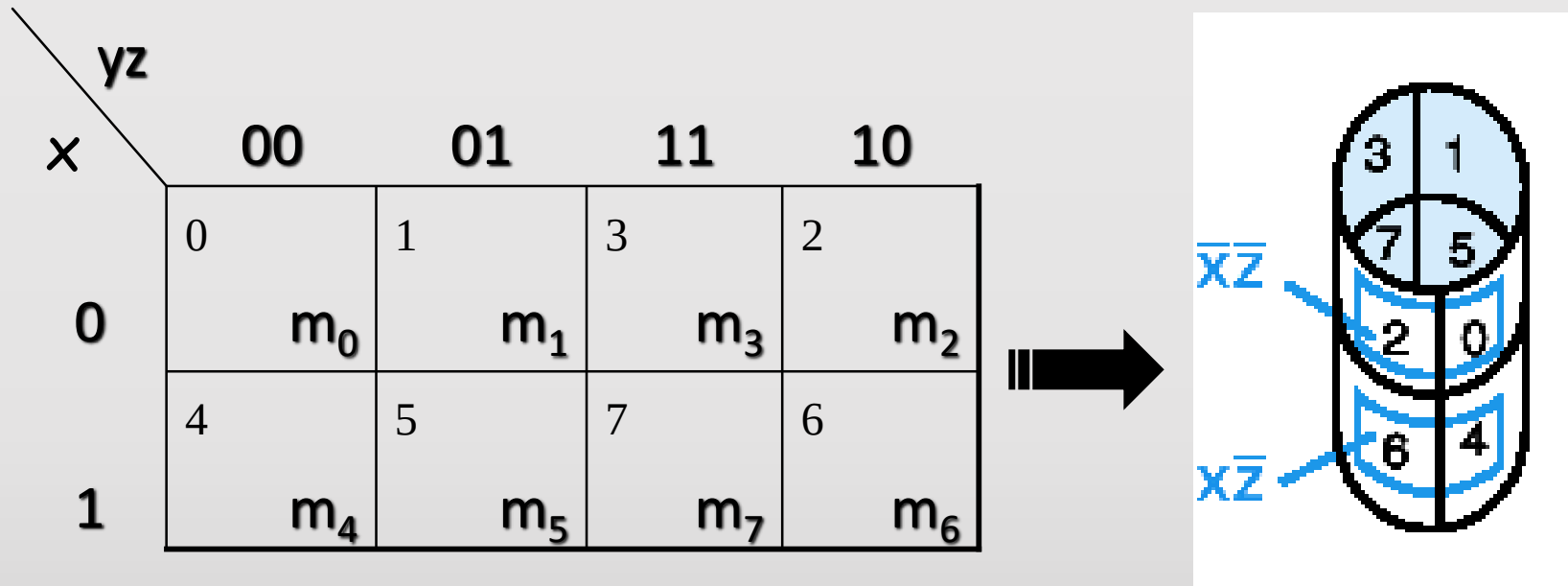
		x_2	
		0	1
x_1	0	0	1
	1	1	0

Detailed description: A 2x2 Karnaugh map for variables x1 and x2. The columns are labeled x2 (0, 1) and the rows are labeled x1 (0, 1). The cells contain values: (0,0)=0, (0,1)=1, (1,0)=1, (1,1)=0. Dashed rectangles group the 1s: one horizontal group covering (0,0) and (0,1), and one vertical group covering (0,0) and (1,0). The cell (0,0) is the intersection of these two groups.

Minimization as SOP using K-map

- Enter 1s in the K-map for each product term in the function
- Group *adjacent* K-map cells containing 1s to obtain a product with fewer variables. Group size must be in power of 2 (2, 4, 8, ...)
- Handle “boundary wrap” for K-maps of 3 or more variables.
- Realize that answer may not be unique

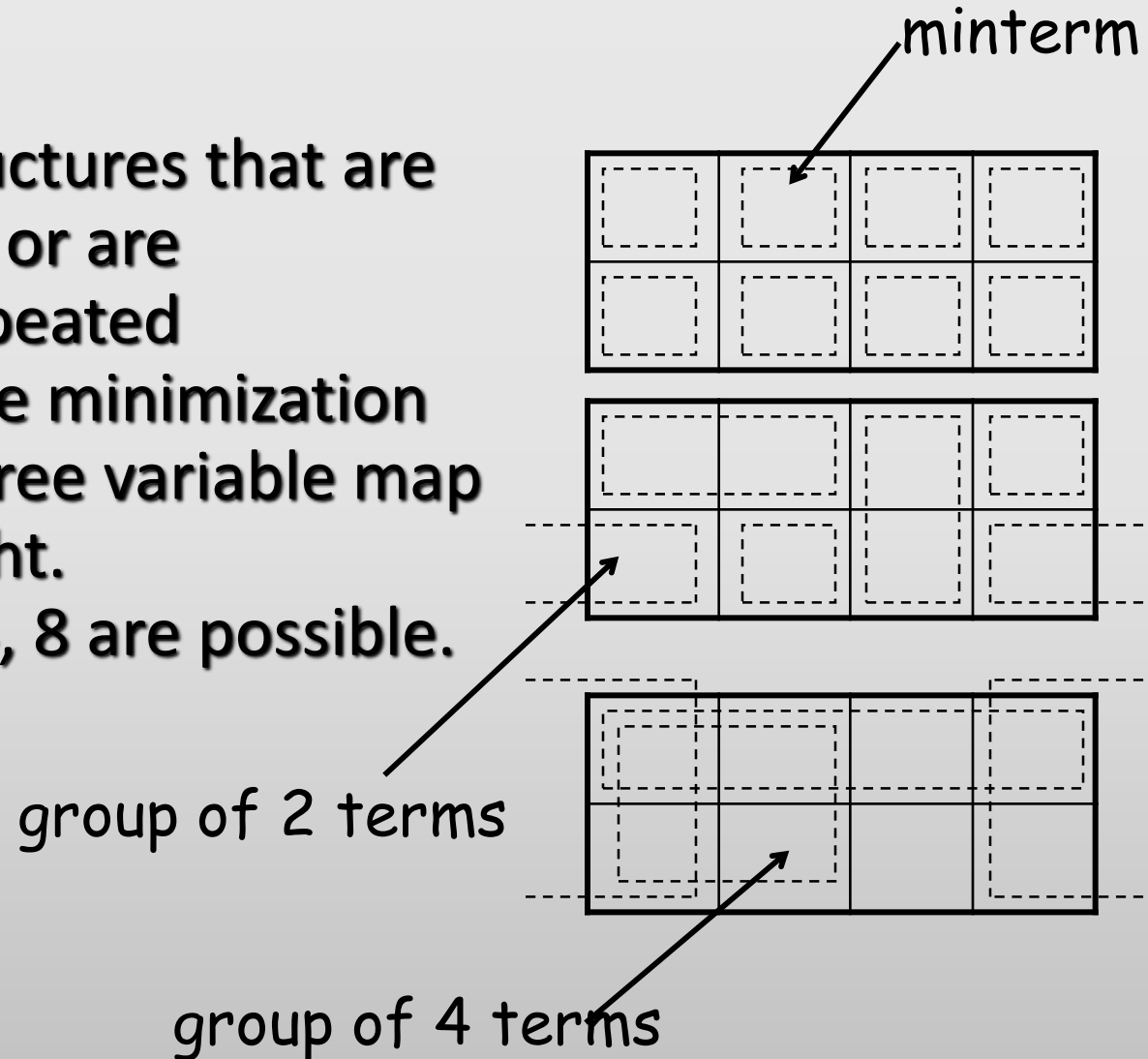
Three-Variable Map



- Note: variable ordering is (x,y,z); yz specifies column, x specifies row.
- Each cell is adjacent to three other cells (left or right or top or bottom or edge wrap)

Three-Variable Map (cont.)

The types of structures that are either minterms or are generated by repeated application of the minimization theorem on a three variable map are shown at right. Groups of 1, 2, 4, 8 are possible.

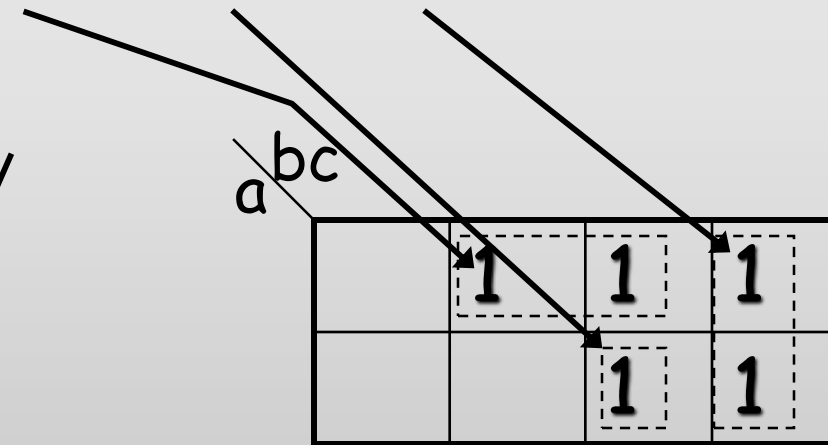
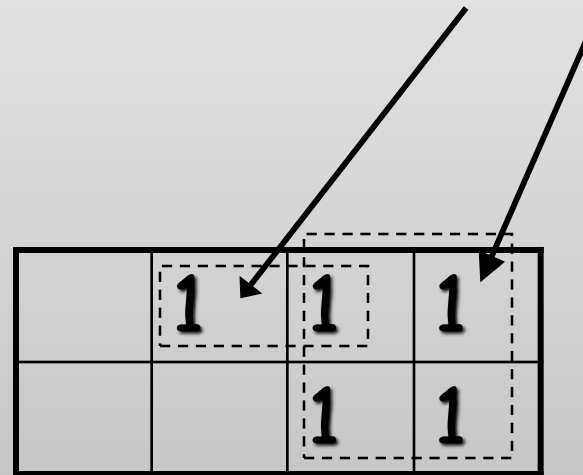


Three-Variable Map

- There is only one possibility of grouping 8 adjacent min terms.
- The possible combinations of grouping 4 adjacent min terms are $\{(m_0, m_1, m_3, m_2), (m_4, m_5, m_7, m_6), (m_0, m_1, m_4, m_5), (m_1, m_3, m_5, m_7), (m_3, m_2, m_7, m_6) \text{ and } (m_2, m_0, m_6, m_4)\}$.
- The possible combinations of grouping 2 adjacent min terms are $\{(m_0, m_1), (m_1, m_3), (m_3, m_2), (m_2, m_0), (m_4, m_5), (m_5, m_7), (m_7, m_6), (m_6, m_4), (m_0, m_4), (m_1, m_5), (m_3, m_7) \text{ and } (m_2, m_6)\}$.
- If $x=0$, then 3 variable K-map becomes 2 variable K-map.

Simplification

- Enter minterms of the Boolean function into the map, then group terms
- Example: $f(a,b,c) = a'c + abc + bc'$
- Result: $f(a,b,c) = a'c + b$



More Examples

- $f_1(x, y, z) = \sum m(2,3,5,7)$

- $f_1(x, y, z) = x'y + xz$

- $f_2(x, y, z) = \sum m(0,1,2,3,6)$

- $f_2(x, y, z) = x' + yz'$

		yz			
		00	01	11	10
x	0			1	1
	1		1	1	

1	1	1	1
			1

Four-Variable Maps

		YZ			
		00	01	11	10
WX	00	m_0	m_1	m_3	m_2
	01	m_4	m_5	m_7	m_6
	11	m_{12}	m_{13}	m_{15}	m_{14}
	10	m_8	m_9	m_{11}	m_{10}



- Top cells are adjacent to bottom cells. Left-edge cells are adjacent to right-edge cells.
- Note variable ordering (WXYZ).

Four-variable Map Simplification

- One square represents a minterm of 4 literals.
- A rectangle of 2 adjacent squares represents a product term of 3 literals.
- A rectangle of 4 squares represents a product term of 2 literals.
- A rectangle of 8 squares represents a product term of 1 literal.
- A rectangle of 16 squares produces a function that is equal to logic 1.

Example

- Simplify the following Boolean function $(A,B,C,D) = \sum m(0,1,2,4,5,7,8,9,10,12,13)$.
- First put the function $g()$ into the map, and then group as many 1s as possible.

ab	cd			
	1	1		1
	1	1	1	
	1	1		
	1	1		1

1	1		1
1	1	1	
1	1		
1	1		1

$$g(A,B,C,D) = c' + b'd' + a'bd$$

Don't Care Conditions in K-Map

- The “Don’t Care” in a [K-Map](#) allows us to form a grouping of the variables. While forming groups of cells, we can consider a “Don’t Care” cell as either 1 or 0 or we can simply ignore that cell. Therefore, “Don’t Care” condition can help us to form a larger group of cells.
- They are denoted with **x** or **—**. Each **x** may be arbitrarily assigned the value 0 or 1 in an implementation.
- Don’t cares can be used to ***further*** simplify a function

Minimization using Don't Cares

- Treat don't cares as if they are 1's to generate SOP and 0's to generate POS.
- Use don't cares for greater combination of cells if possible.
- Keep the remaining don't cares uncombined after all the 1's or 0's are already combined.(1's for SOP and 0's for POS)

Example

ab \ cd				
	00	01	11	10
00		1		1
01	1	1		1
11			x	x
10	1	1	x	x

	1		1
1	1		1
		x	x
1	1	x	x

	1		1
1	1		1
		x	x
1	1	x	x

Another Example

cd
ab

x	1		
1	x		x
1	x	x	1
	x	x	

x	1		
1	x		x
1	x	x	1
	x	x	

x	1		
1	x		x
1	x	x	1
	x	x	

Combinational Logic

Combinational Logic

- Logic circuits for digital systems may be combinational or sequential.
- A combinational circuit consists of input variables, logic gates, and output variables.

Design Procedure

The design procedure for combinational logic circuits are:

- Determine required number of inputs and outputs from the specifications.
- Derive the truth table for each of the outputs based on their relationships to the input.
- Simplify the Boolean expression for each output. Use *Karnaugh Maps* or Boolean algebra.
- Draw a logic diagram that represents the simplified Boolean expression.

Half Adder

- A combinational circuit that performs the addition of two bits is called a **half adder**.
- The truth table for the half adder is listed below:

x	y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$S(x,y) = \sum m(1,2)$$

$$C(x,y) = \sum m(3)$$

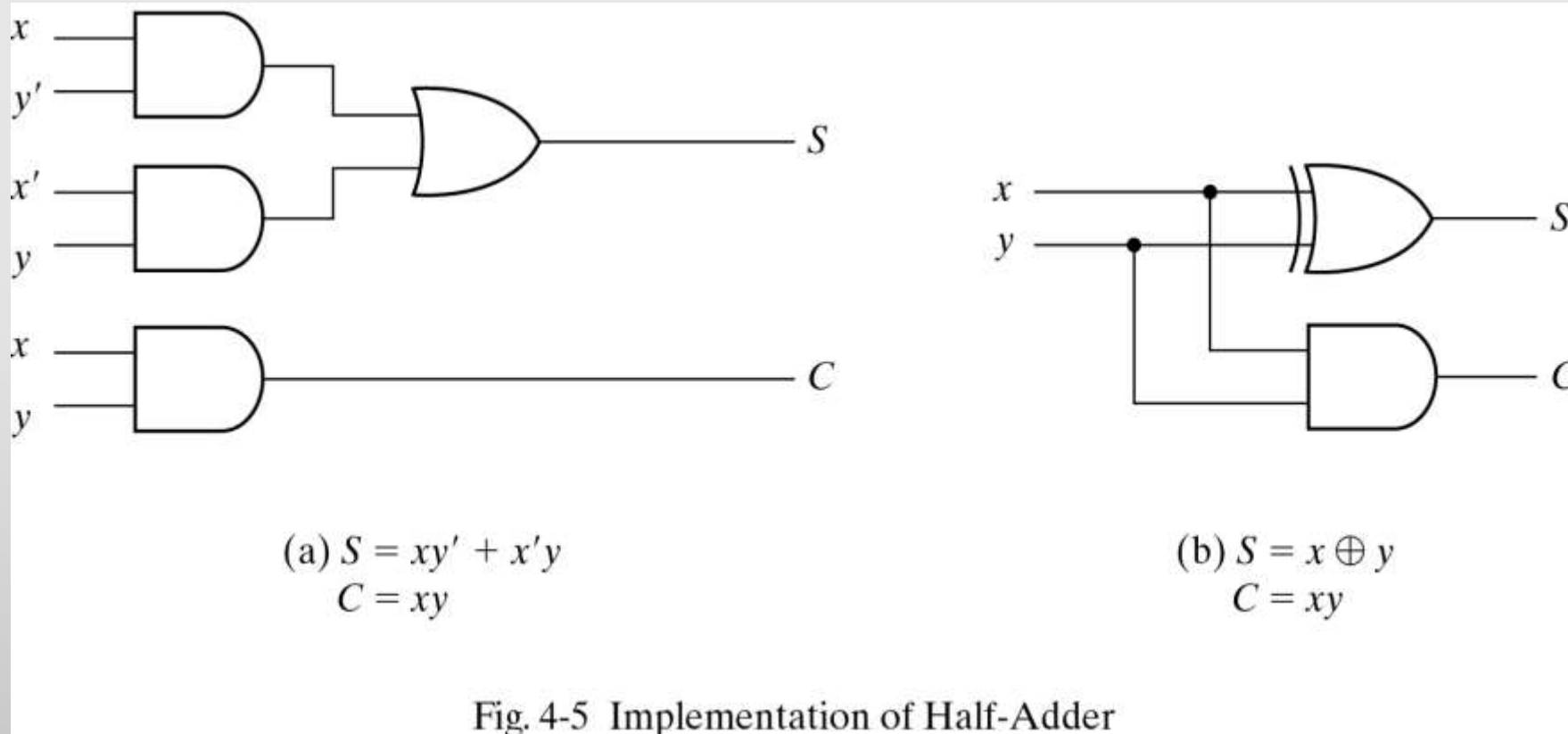
		y
x \	0	1
0	0	1
1	1	0

$$S = x'y + xy'$$

		y
x \	0	1
0	0	0
1	0	1

$$C = xy$$

Implementation of Half-Adder



$$S = x'y + xy' \text{ or } x \oplus y$$

$$C = xy$$

Full-Adder

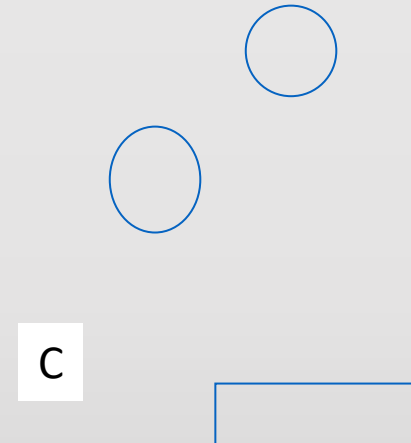
- One that performs the addition of three bits (two significant bits and a previous carry) is a **full adder**.

x	y	z	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$S(x,y,z) = \sum m(1,2,4,7)$$

$$C(x,y,z) = \sum m(3,5,6,7)$$

Simplified Expressions



$$S = x'y'z + x'yz' + xy'z' + xyz \quad \text{or } (x \oplus y \oplus z)$$

$$C = xy + xz + yz$$

Full-Adder

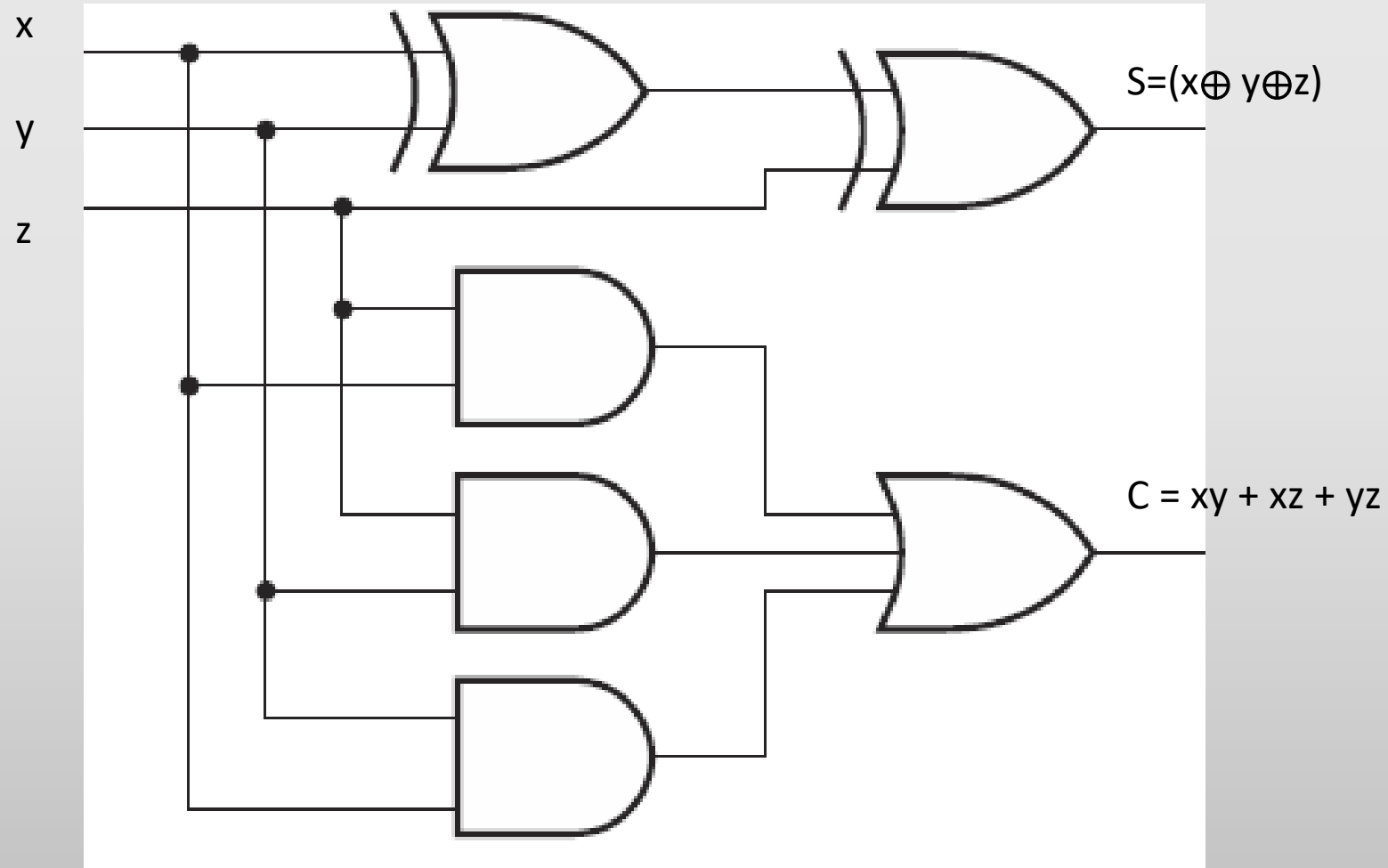
- $S = x'y'z + x'yz' + xy'z' + xyz$
- $= x'(y'z + yz') + x(y'z' + yz)$
- $= x'(y \oplus z) + x(y \oplus z)'$ $[y \oplus z = A]$
- $= x'A + xA'$
- $= x \oplus A$
- $= (x \oplus y \oplus z)$

Full-Adder

$$S = x'y'z + x'yz' + xy'z' + xyz$$

$$C = xy + xz + yz$$

Full-Adder



Half-Subtractor

- One that performs the subtraction of two bits is **half subtractor**.

a	b	D	B
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$D(a,b) = \sum m(1,2)$$

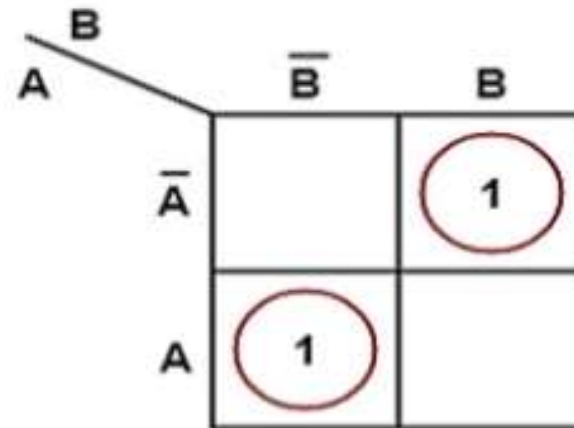
$$B(a,b) = \sum m(1)$$

Half-Subtractor

$$D(a,b) = \sum m(1,2)$$

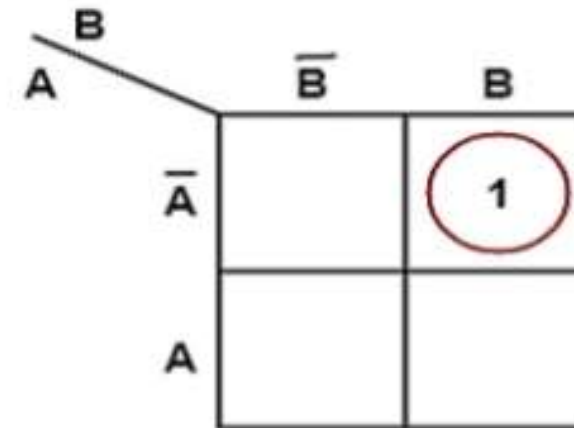
$$B(a,b) = \sum m(1)$$

For D:



$$D = A \oplus B$$

For b:

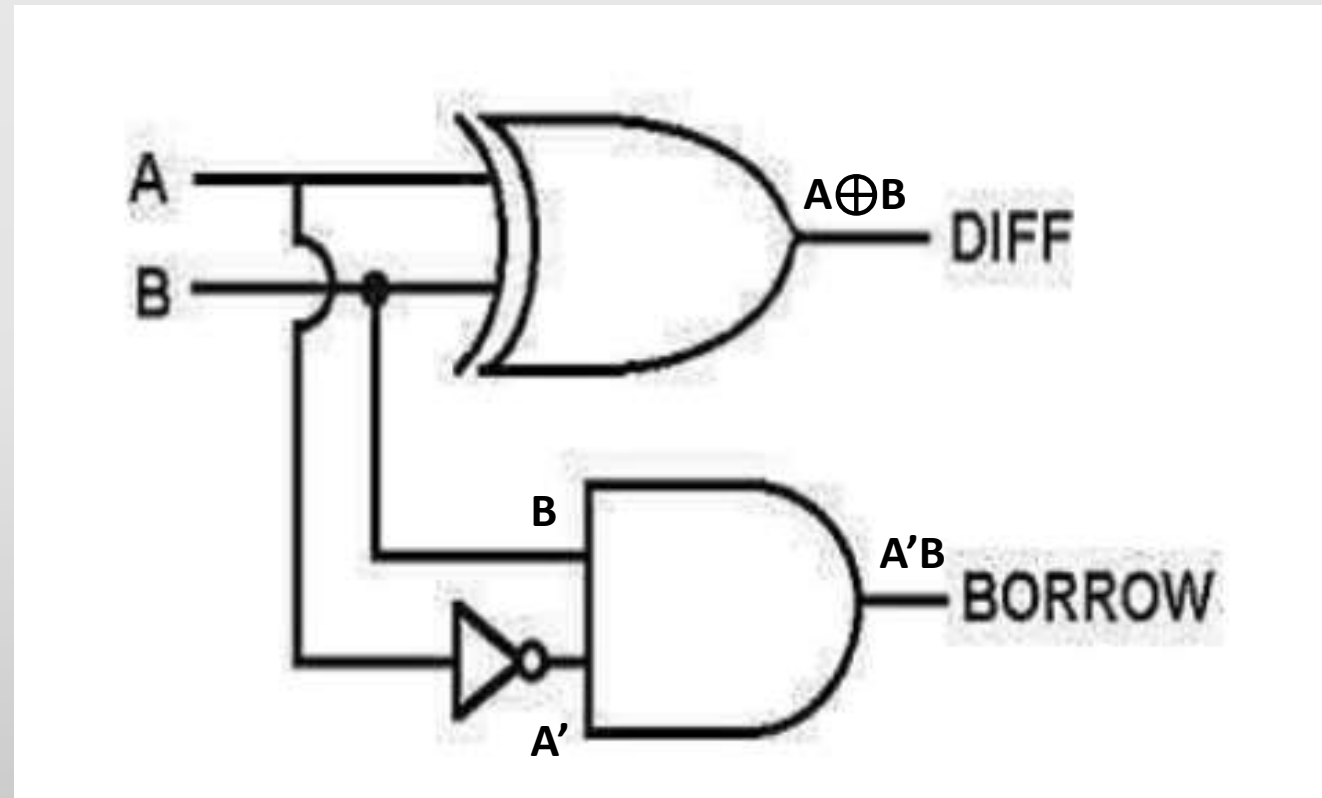


$$b = \bar{A} B$$

$$\text{Borrow} = \bar{A}.B$$

$$\text{Difference} = A \oplus B$$

Half-Subtractor



Full-Subtractor

- One that performs the subtraction of three bits is **Full subtractor**.

a	b	c	D	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$D(a,b,c) = \sum m (1,2,4,7)$$

$$B(a,b,c) = \sum m (1,2,3,7)$$

Full-Subtractor

		bc			
		00	01	11	10
a	0	0	1	0	1
	1	1	0	1	0

$$\text{Difference}(D) = a \oplus b \oplus c$$

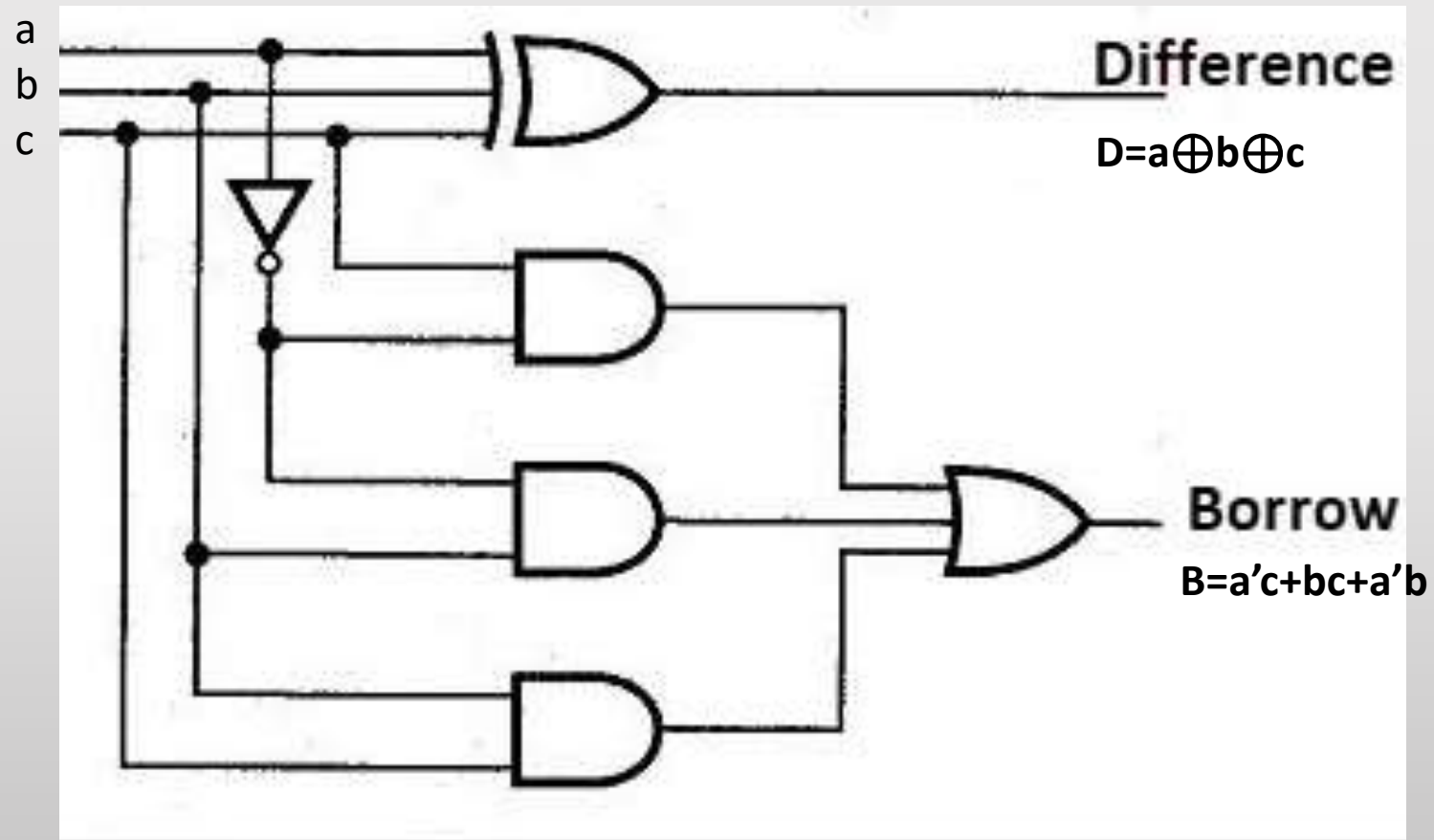
$$D(a,b,c) = \sum m(1,2,4,7)$$

		bc			
		00	01	11	10
a	0	0	1	1	1
	1	0	0	1	0

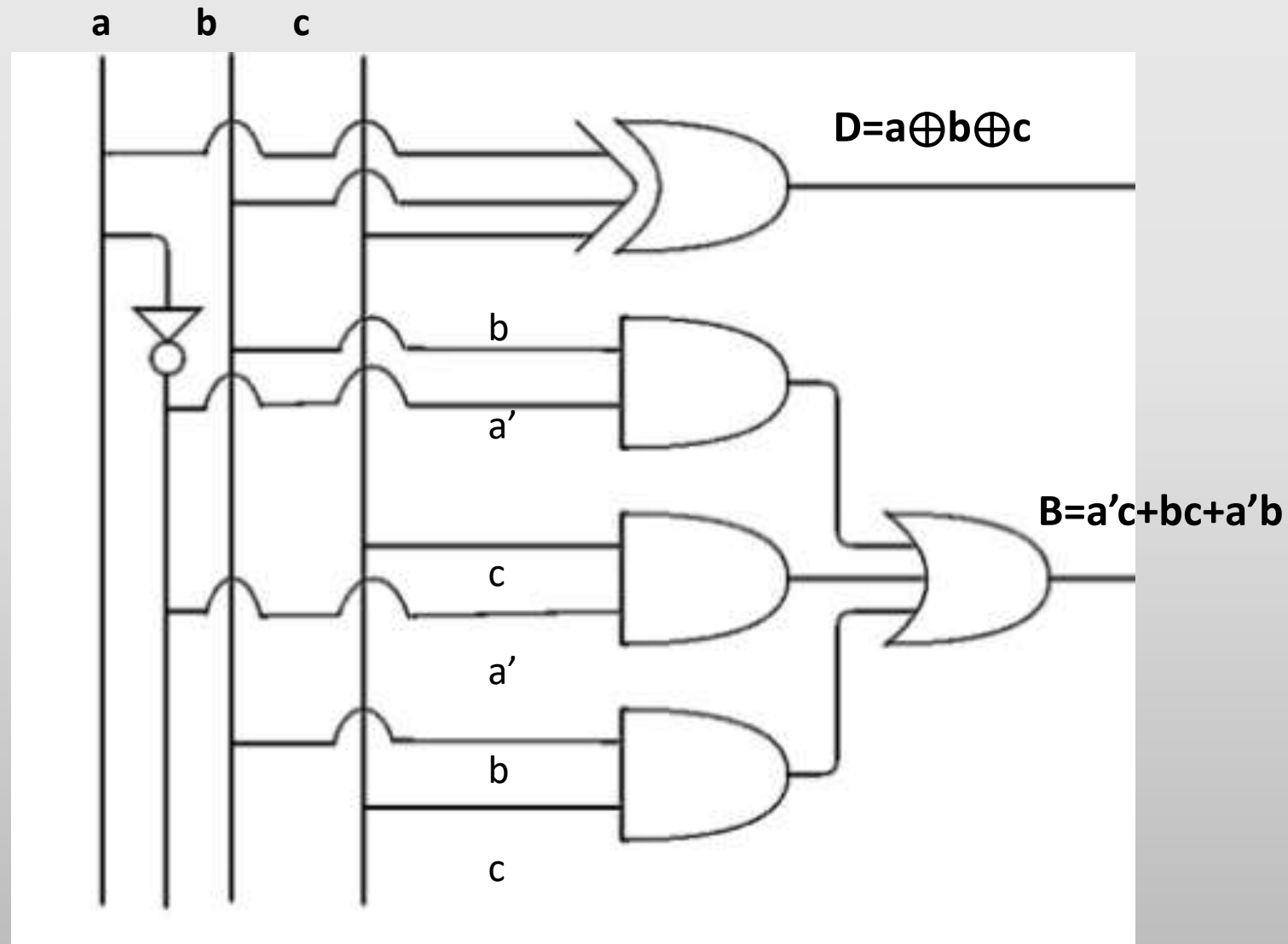
$$\text{Borrow}(B) = a'c + bc + a'b$$

$$B(a,b,c) = \sum m(1,2,3,7)$$

Full-Subtractor



Full-Subtractor



Design a full adder circuit using two half adders. Also verify your answer.

Full Adder

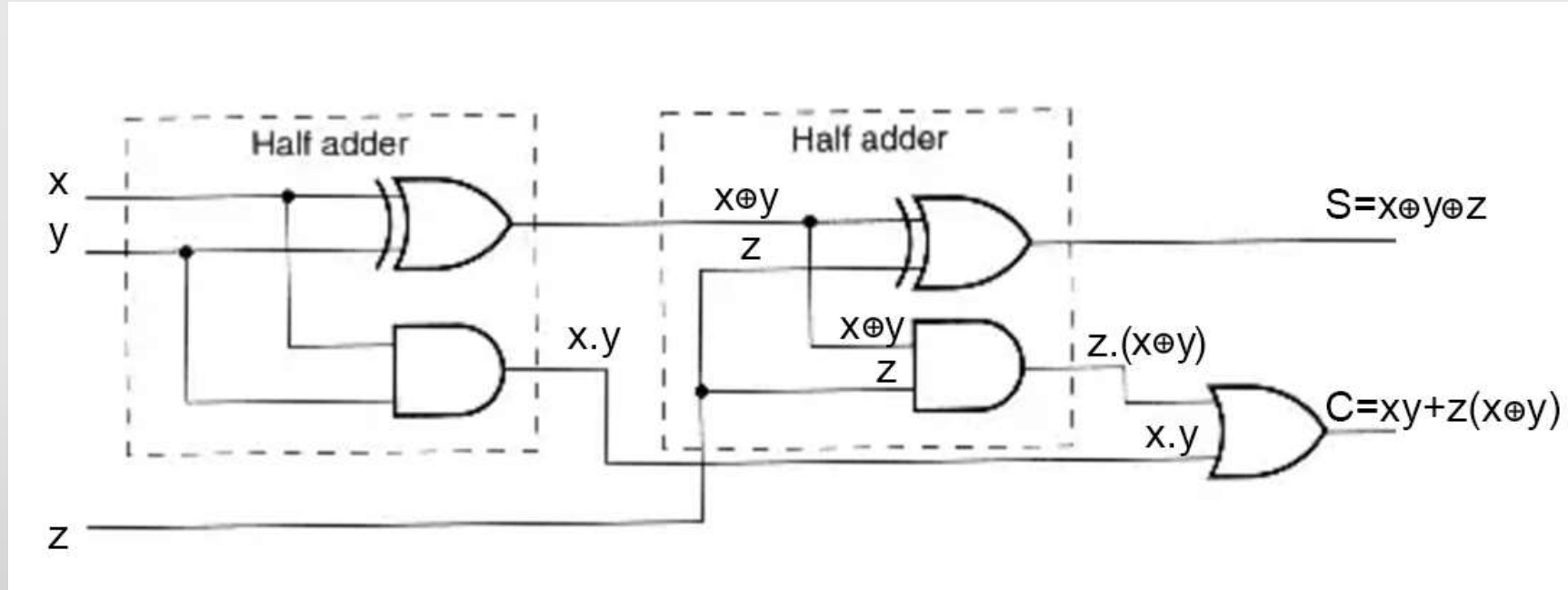
$$S = (x \oplus y \oplus z)$$

$$C = xy + xz + yz$$

Half Adder

$$S = (x \oplus y)$$

$$C = xy$$



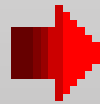
From above circuit:

$$S = (x \oplus y \oplus z)$$

$$C = xy + z(x \oplus y)$$

$$= xy + z(x'y + xy')$$

$$= xy + x'yz + xy'z$$



From actual Full adder circuit:

$$C = xy + xz + yz$$

$$C = xy + xz(y + y') + yz(x + x')$$

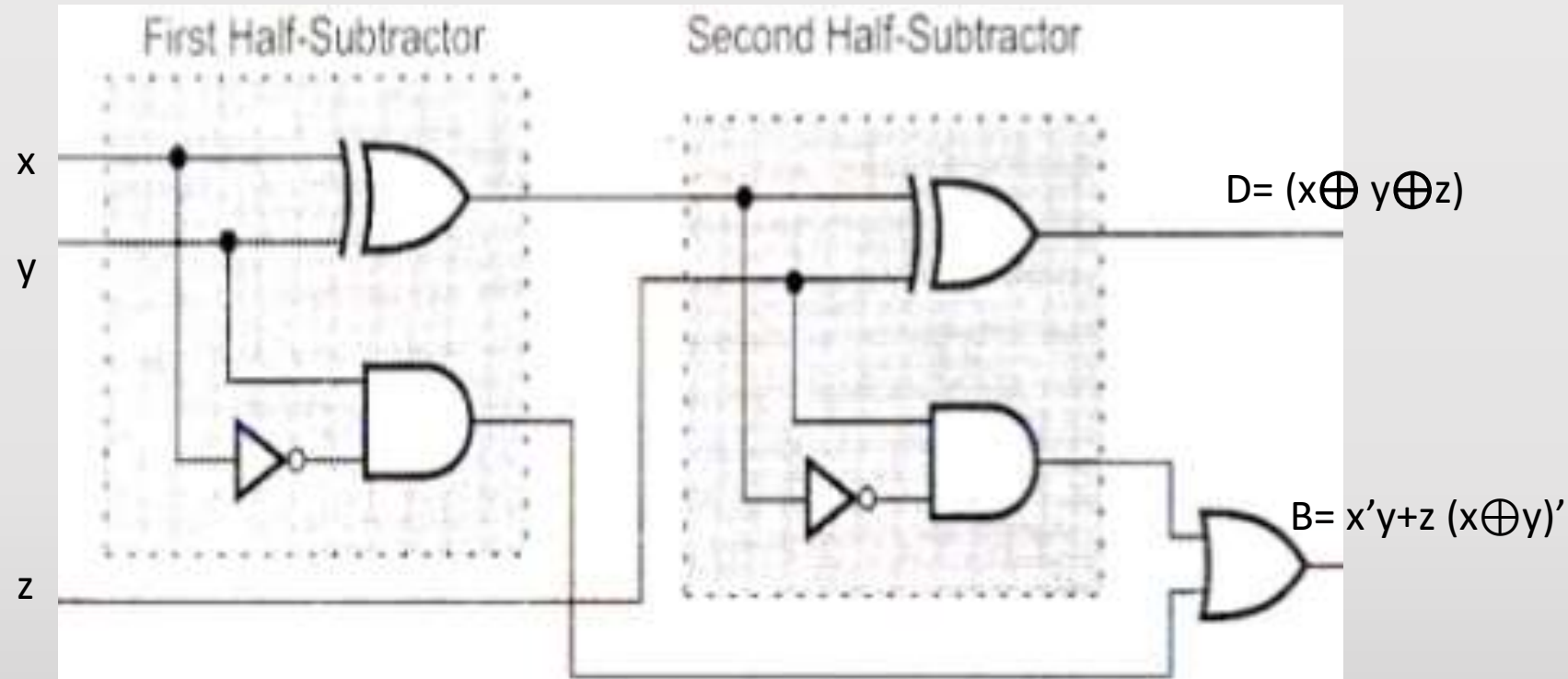
$$C = xy + xyz + xy'z + x'yz + x'yz$$

$$C = xy + xyz + xy'z + x'yz$$

$$C = xy(1 + z) + xy'z + x'yz$$

$$C = xy + x'yz + xy'z$$

Construct a full subtractor circuit using two half subtractors. Also verify your answer.



From above circuit:

$$B = x'y + z(x \oplus y)'$$

$$B = x'y + z(xy + x'y')$$

$$B = x'y + xyz + x'y'z$$

But from the actual Full subtractor circuit:

$$B = x'y + yz + x'z$$

$$B = x'y + yz(x + x') + x'z(y + y')$$

$$B = x'y + xyz + x'yz + x'y'z$$

$$B = x'y + xyz + x'yz + x'y'z$$

$$B = x'y + x'yz + xyz + x'y'z$$

$$B = x'y(1 + z) + xyz + x'y'z$$

$$B = x'y + xyz + x'y'z \text{ verified.}$$

Code Conversion Circuit

Design a converter circuit which converts BCD to excess 3.

BCD Code (Input)				Excess-3 Code (Output)			
<i>A</i>	<i>B</i>	<i>C</i>	<i>D</i>	<i>W</i>	<i>X</i>	<i>Y</i>	<i>Z</i>
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0

Don't care conditions:

$m_{10}-m_{15}$

$$W(A,B,C,D) = \sum m(5,6,7,8,9)$$

$$X(A,B,C,D) = \sum m(1,2,3,4,9)$$

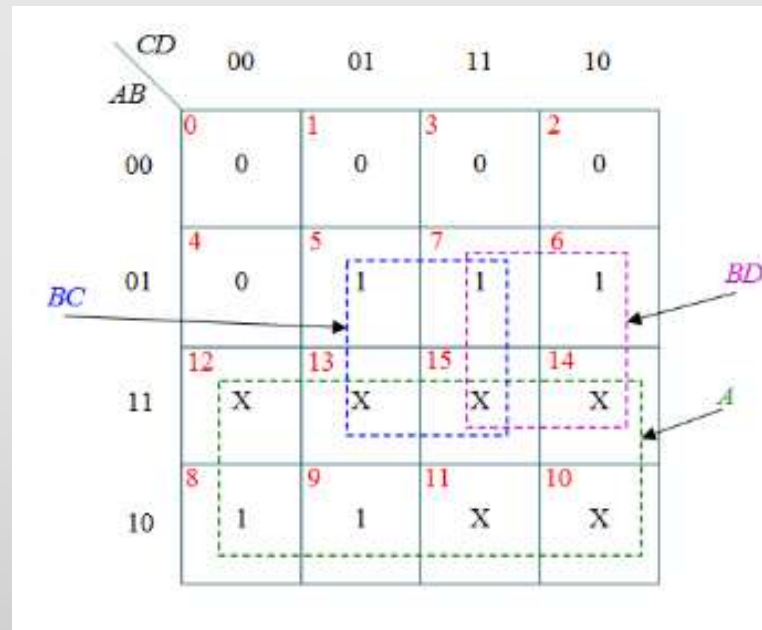
$$Y(A,B,C,D) = \sum m(0,3,4,7,8)$$

$$Z(A,B,C,D) = \sum m(0,2,4,6,8)$$

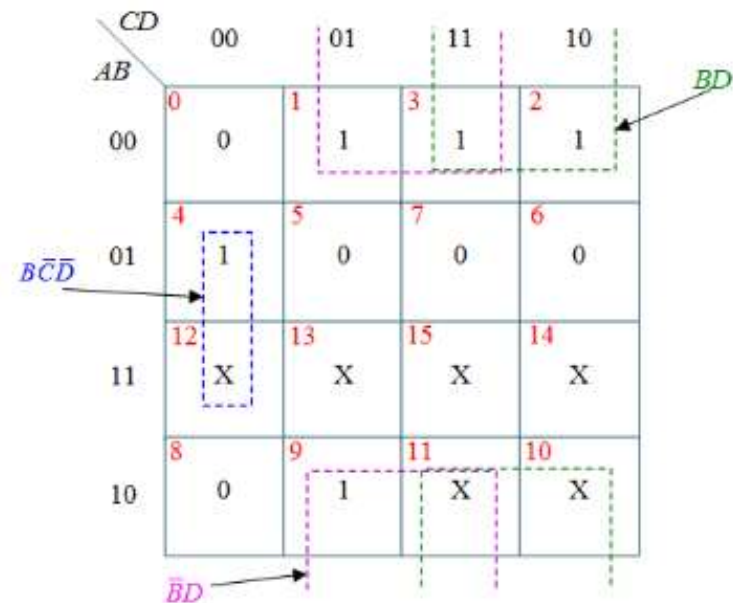
Code Conversion Circuit

$$W(A,B,C,D) = \sum m(5,6,7,8,9)$$

$$X(A,B,C,D) = \sum m(1,2,3,4,9)$$



$$W = A + BC + BD$$



$$X = B'C + B'D + BC'D'$$

Code Conversion Circuit

$$Y(A,B,C,D) = \sum m(0,3,4,7,8)$$

		CD			
		00	01	11	10
AB	00	0 1	1 0	3 1	2 0
	01	4 1	5 0	7 1	6 0
	11	12 X	13 X	15 X	14 X
	10	8 1	9 0	11 X	10 X

Annotations: A blue dashed box highlights the 1s in the first column (CD=00), labeled $\bar{C}\bar{D}$. A green dashed box highlights the 1s in the third column (CD=11), labeled CD .

$$Y = CD + C'D'$$

$$Z(A,B,C,D) = \sum m(0,2,4,6,8)$$

		CD			
		00	01	11	10
AB	00	0 1	1 0	3 0	2 1
	01	4 1	5 0	7 0	6 1
	11	12 X	13 X	15 X	14 X
	10	8 1	9 0	11 X	10 X

Annotation: A green dashed box highlights the 1s in the first column (CD=00), labeled $\bar{D}$.

$$Z = D'$$



Code Conversion Circuit

*Design a converter circuit
 which converts Binary to Gray.*

$$W(A,B,C,D) = \sum m (8,9,10,11,12,13,14,15)$$

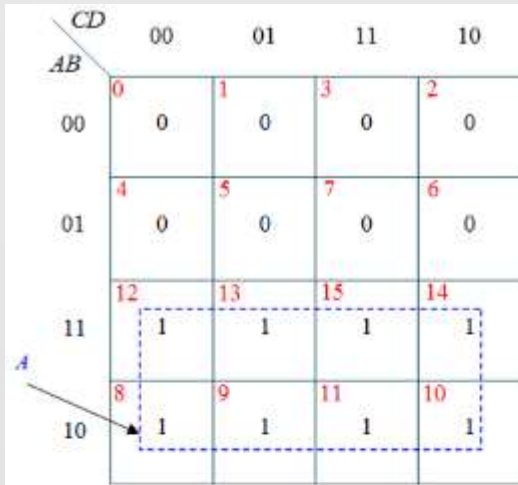
$$X(A,B,C,D) = \sum m (4,5,6,7,8,9,10,11)$$

$$Y(A,B,C,D) = \sum m (2,3,4,5,10,11,12,13)$$

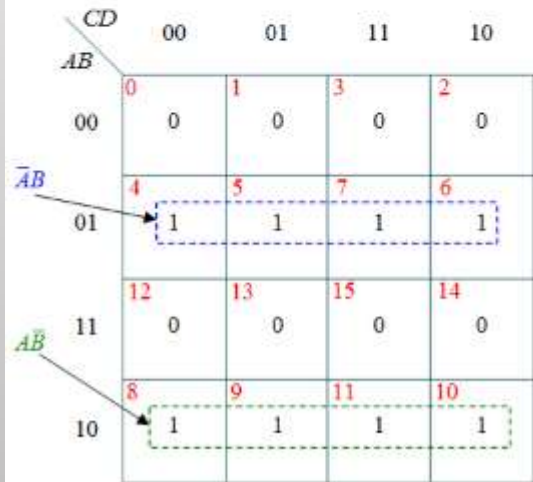
$$Z(A,B,C,D) = \sum m (1,2,5,6,9,10,13,14)$$

Binary Code (Input)				Gray Code (Output)			
A	B	C	D	W	X	Y	Z
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	1
0	0	1	0	0	0	1	1
0	0	1	1	0	0	1	0
0	1	0	0	0	1	1	0
0	1	0	1	0	1	1	1
0	1	1	0	0	1	0	1
0	1	1	1	0	1	0	0
1	0	0	0	1	1	0	0
1	0	0	1	1	1	0	1
1	0	1	0	1	1	1	1
1	0	1	1	1	1	1	0
1	1	0	0	1	0	1	0
1	1	0	1	1	0	1	1
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

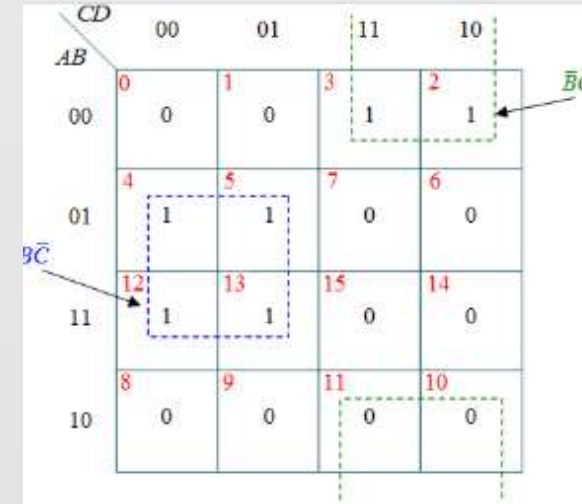
Code Conversion Circuit



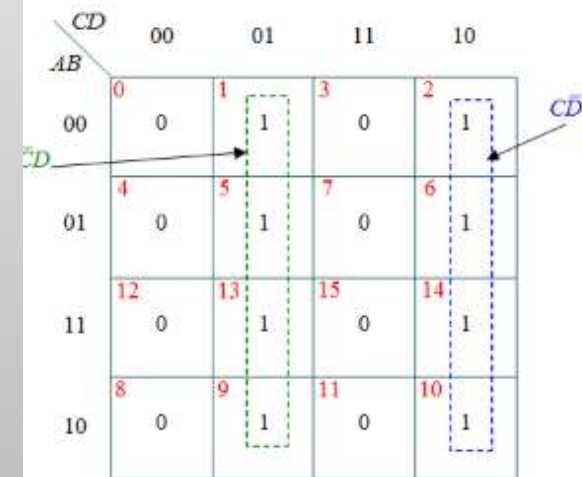
(a) *k-map for W*



(b) *k-map for X*



(c) *k-map for Y*



(d) *k-map for Z*

Parity Generator Circuit

EVEN PARITY

3 Bit Message			Even Parity Generator
A	B	C	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

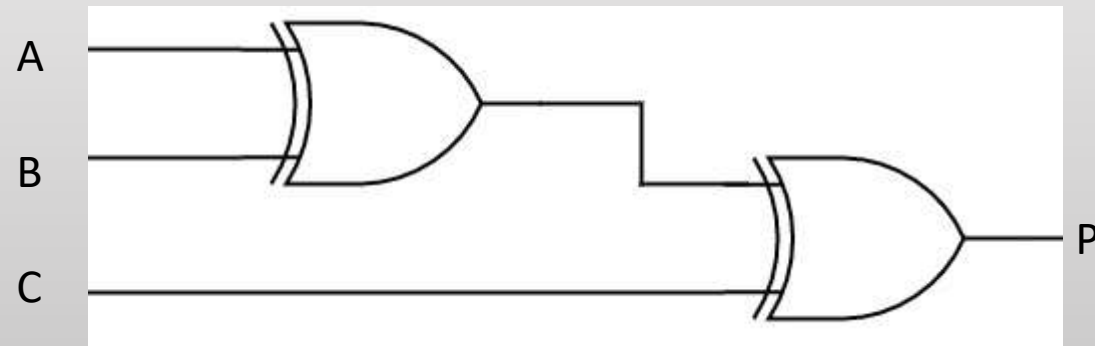
$$P(A,B,C) = \sum m (1,2,4,7)$$

Parity Generator Circuit

$$P(A,B,C) = \sum m(1,2,4,7)$$

BC	00	01	11	10
A				
0	0	1	3	2
1	4	5	7	6

$$\begin{aligned}
 P &= \bar{A} \bar{B} C + \bar{A} B \bar{C} + A \bar{B} \bar{C} + A B C \\
 &= \bar{A} (\bar{B} C + B \bar{C}) + A (\bar{B} \bar{C} + B C) \\
 &= \bar{A} (B \oplus C) + A (\overline{B \oplus C}) \\
 P &= A \oplus B \oplus C
 \end{aligned}$$



3 bit even parity generator circuit

Parity Generator Circuit

ODD PARITY

3 Bit Message			Odd Parity Bit Generator
A	B	C	P
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

$$P(A,B,C) = \sum m (0,3,5,6)$$

Parity Generator Circuit

$$P(A,B,C) = \sum m (0,3,5,6)$$

BC	00	01	11	10
A				
0	1 ⁰	1 ¹	1 ³	2 ²
1	4 ⁴	1 ⁵	7 ⁷	1 ⁶

$$P = A'B'C' + A'BC + AB'C + ABC'$$

$$P = C'(A'B' + AB) + C(A'B + AB')$$

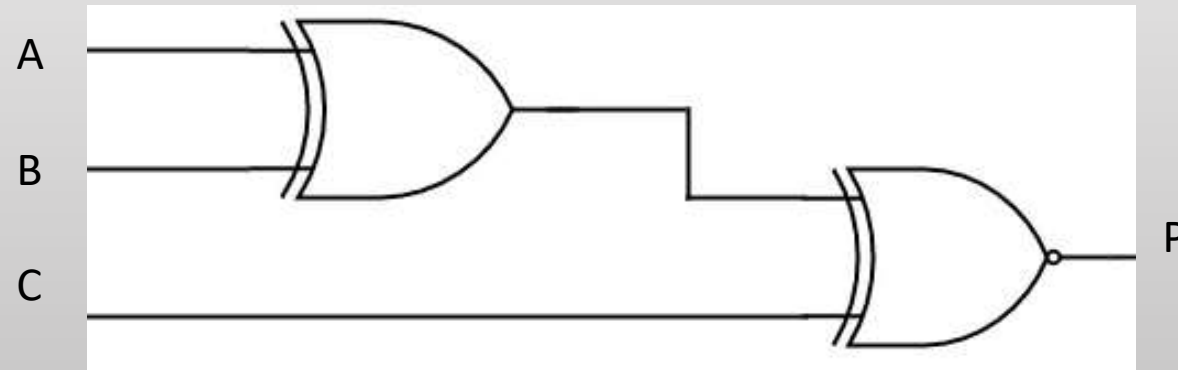
$$P = C'(A \oplus B)' + C(A \oplus B)$$

$$P = C'X' + CX \quad [X = (A \oplus B)]$$

$$P = C \odot X$$

$$P = C \odot (A \oplus B)$$

$$P = (A \oplus B) \odot C$$



3 bit odd parity generator circuit

Parity Checker Circuit

Even Parity
Check

$$C_p(A,B,C,P) = \sum m(1,2,4,7,8,11,13,14)$$

$C_p=1$ for error

4 Bit Received Message				Parity error check C_p
A	B	C	P	C_p
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

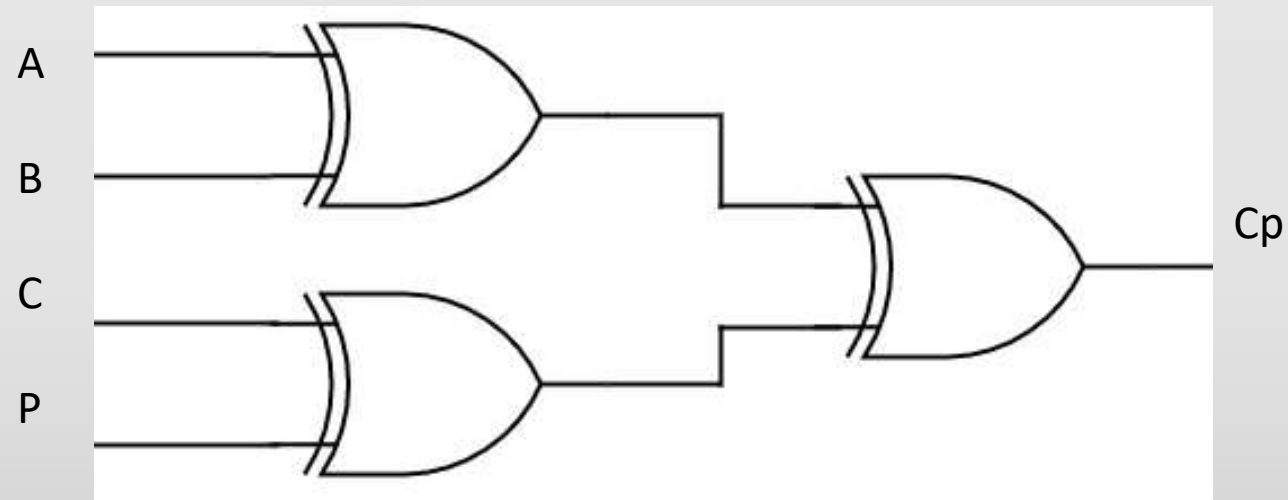
Parity Checker Circuit

$$C_p(A,B,C,P) = \sum m(1,2,4,7,8,11,13,14)$$

AB \ CP	00	01	11	10
00	0 ⁰	1 ¹	0 ³	1 ²
01	1 ⁴	0 ⁵	1 ⁷	0 ⁶
11	0 ¹²	1 ¹³	0 ¹⁵	1 ¹⁴
10	1 ⁸	0 ⁹	1 ¹¹	0 ¹⁰

$$C_p = (A \oplus B) \oplus (C \oplus P)$$

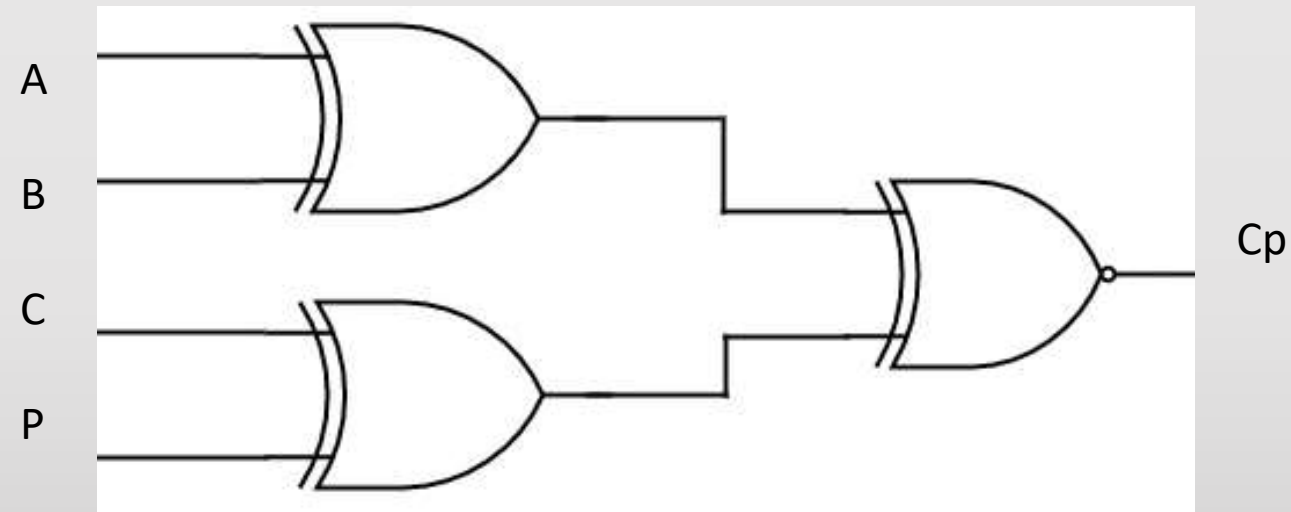
Parity Checker Circuit



3 bit message even parity checker circuit

Parity Checker Circuit

Odd Parity
Check



3 bit message odd parity checker circuit

$$C_p = (A \oplus B) \odot (C \oplus P)$$

Code Conversion Circuit

Design a converter circuit which converts 84 -2 -1 to 2421 code.

Don't care conditions:

$m_1, m_2, m_3, m_{12}, m_{13}, m_{14}$

$$W(A,B,C,D) = \sum m(8,9,10,11,15)$$

$$X(A,B,C,D) = \sum m(4,8,9,10,15)$$

$$Y(A,B,C,D) = \sum m(5,6,8,11,15)$$

$$Z(A,B,C,D) = \sum m(5,7,9,11,15)$$

Decimal	8 4 -2 -1	2 4 2 1
	A B C D	W X Y Z
0	0 0 0 0 (m_0)	0 0 0 0
1	0 1 1 1 (m_7)	0 0 0 1
2	0 1 1 0 (m_6)	0 0 1 0
3	0 1 0 1 (m_5)	0 0 1 1
4	0 1 0 0 (m_4)	0 1 0 0
5	1 0 1 1 (m_{11})	1 0 1 1
6	1 0 1 0 (m_{10})	1 1 0 0
7	1 0 0 1 (m_9)	1 1 0 1
8	1 0 0 0 (m_8)	1 1 1 0
9	1 1 1 1 (m_{15})	1 1 1 1

Code Conversion Circuit

*Design a converter
circuit which converts
84 -2 -1 to 2421 code.*

$$W(A,B,C,D) = \sum m (8,9,10,11,15) \quad W = A$$

		CD			
		00	01	11	10
AB	00	0000	X 0001	X 0011	X 0010
	01	0100	0101	0111	0110
	11	X 1100	X 1101	1 1111	X 1110
	10	1 1000	1 1001	1 1011	1 1010

Code Conversion Circuit

*Design a converter
circuit which converts
84 -2 -1 to 2421 code.*

$$X(A,B,C,D) = \sum m(4,8,9,10,15) \quad X = AB + AC' + AD' + BC'D'$$

CD \ AB	00	01	11	10
00		X	X	X
01	1			
11	X	X	1	X
10	1	1		1

0000 0001 0011 0010
0100 0101 0111 0110
1100 1101 1111 1110
1000 1001 1011 1010

Code Conversion Circuit

*Design a converter
circuit which converts
84 -2 -1 to 2421 code.*

$$Y(A,B,C,D) = \sum m (5,6,8,11,15)$$

$$Y = AB + A'C'D + AC'D' + A'CD' + ACD$$

CD \ AB	00	01	11	10
00	0000	X 0001	X 0011	X 0010
01	0100	1 0101	0111	1 0110
11	X 1100	X 1101	1 1111	X 1110
10	1 1000	1001	1 1011	1010

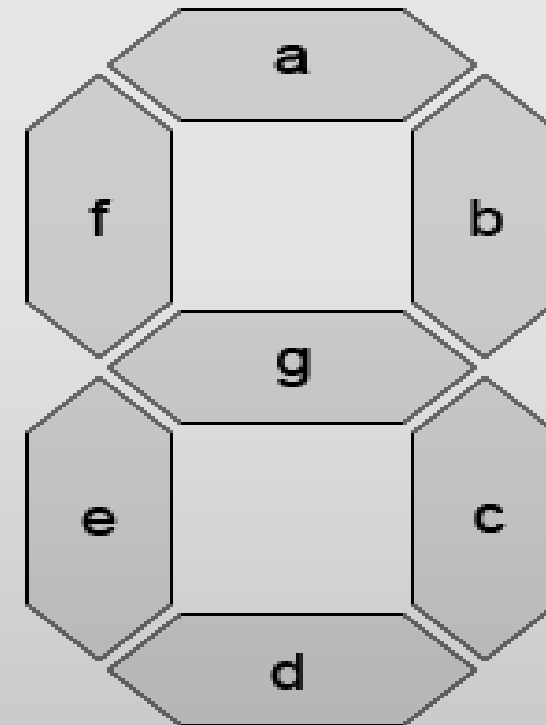
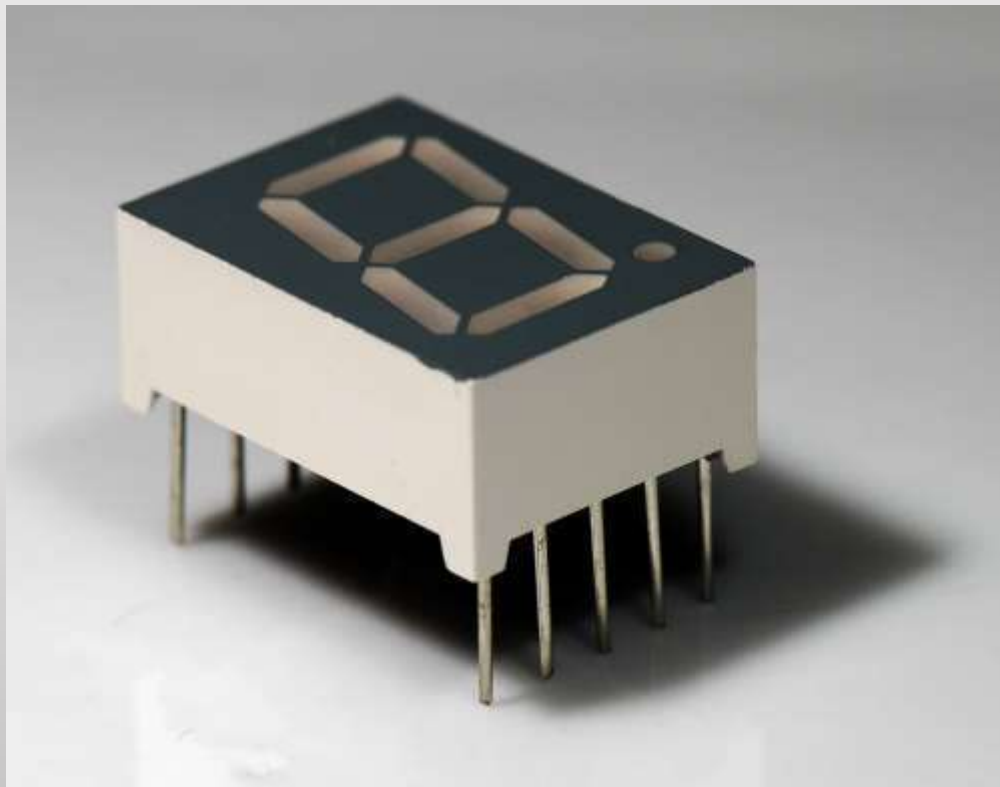
Code Conversion Circuit

*Design a converter
circuit which converts
84 -2 -1 to 2421 code.*

$$Z(A,B,C,D) = \sum m(5,7,9,11,15) \quad Z = D$$

CD \ AB	00	01	11	10
00	0000	X 0001	X 0011	X 0010
01	0100	1 0101	1 0111	0110
11	X 1100	X 1101	1 1111	X 1110
10	1000	1 1001	1 1011	1010

Seven Segment Display



Seven Segment Display

- 7 LEDs (light emitting diodes), each one controlled by an input.
- A seven-segment decoders can be used to display the decimal numbers 0-9 and some alpha characters.

Display Possibilities

0 1 2 3 4 5 6 7 8 9

Select Alpha Characters

A B C D E F G H I
 J K L M N O P Q R

Simple Messages

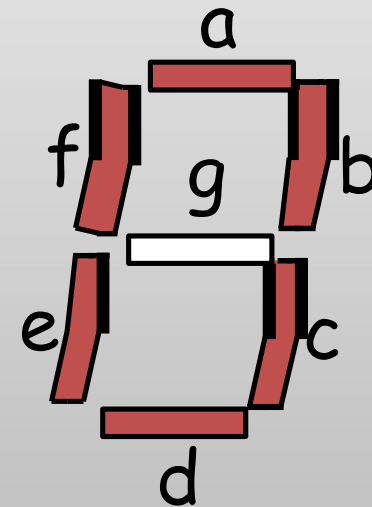
1234 5678

Seven Segment Display

- Input is a 4-bit BCD code $\rightarrow$ 4 inputs (A, B, C, D)
- Output is a 7-bit code (a,b,c,d,e,f,g) that allows for the decimal equivalent to be

Displayed Example:

- Input: $(0000)_{\text{BCD}}$
- Output: 1111110 ($a=b=c=d=e=f=1, g=0$)



Applications



Truth Table for 7 Segment Decoder

Decimal Digit	Input lines				Output lines							Display pattern
	A	B	C	D	a	b	c	d	e	f	g	
0	0	0	0	0	1	1	1	1	1	1	0	
1	0	0	0	1	0	1	1	0	0	0	0	
2	0	0	1	0	1	1	0	1	1	0	1	
3	0	0	1	1	1	1	1	1	0	0	1	
4	0	1	0	0	0	1	1	0	0	1	1	
5	0	1	0	1	1	0	1	1	0	1	1	
6	0	1	1	0	1	0	1	1	1	1	1	
7	0	1	1	1	1	1	1	0	0	0	0	
8	1	0	0	0	1	1	1	1	1	1	1	
9	1	0	0	1	1	1	1	1	0	1	1	

Expression for segment g

$$g(A,B,C,D) = \sum m(2, 3, 4, 5, 6, 8, 9) \text{ and } \sum d(10,11,12,13,14,15)$$

AB \ CD	00	01	11	10
00	0	0	1	1
01	1	1	0	1
11	×	×	×	×
10	1	1	×	×

$$g = \overline{B}C + C\overline{D} + B\overline{C} + A$$

Draw K MAP for all the 7 expressions:
a,b,c,d,e,f,g and draw the logic diagram

Universality of NAND & NOR Gates

What are a Universal Gate ? Why NAND and NOR are known as universal gates?

A gate which can be used to create any Logic gate is called Universal Gate.

NAND and NOR are called Universal Gates because all the other gates can be created by using these gates.

NAND as Universal Gates

Implementation of NOT using NAND

Desired [not gate]

$$F = A'$$

Available [Nand gate]

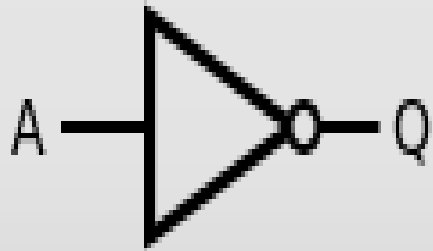
$$F = (A)'$$

$$F = A'$$

NAND as Universal Gates

Implementation of NOT using NAND

A NOT gate is made by joining the inputs of a NAND gate together.



Desired NOT Gate



NOT using NAND

NAND as Universal Gates

Implementation of AND using NAND

Desired [AND gate]

$$F = A.B$$

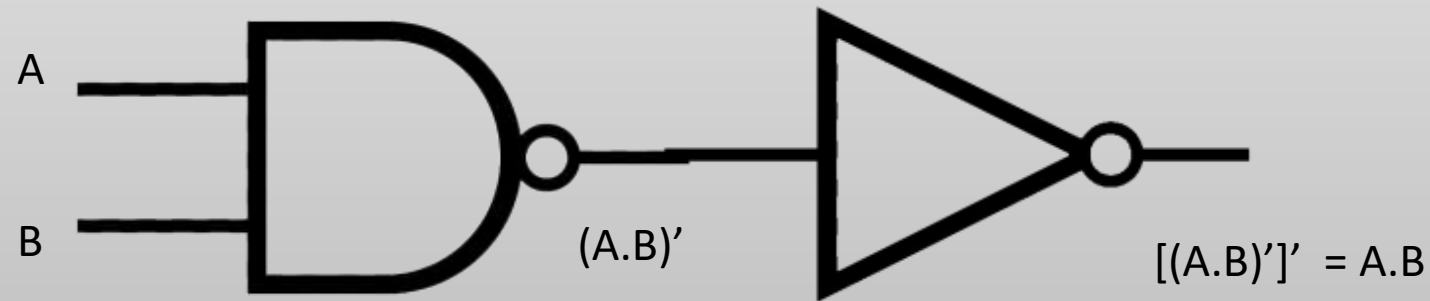
Available [NAND gate]

$$F = (A.B)'$$

To achieve AND gate;

$$F = [(A.B)']'$$

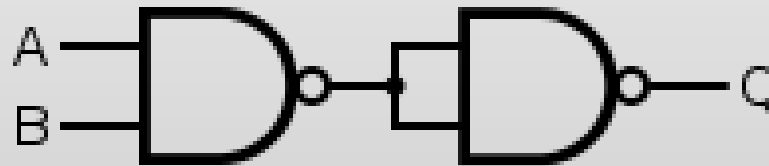
$$F = A.B$$



NAND as Universal Gates

Implementation of AND using NAND

An AND gate is made by following a NAND gate with a NOT gate(using NAND)



AND using NAND

NAND as Universal Gates

Implementation of OR using NAND

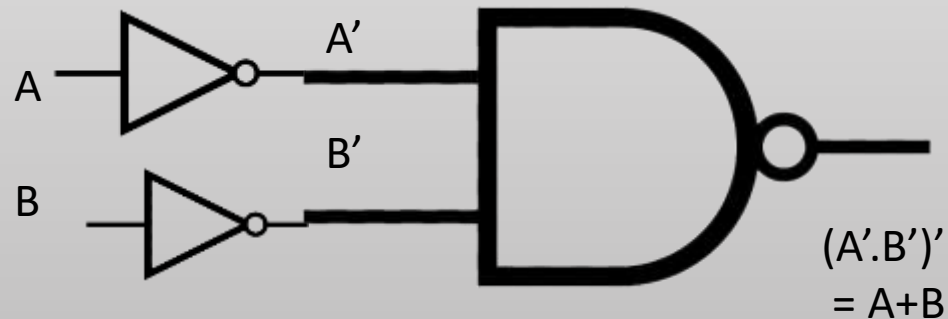
Desired [OR gate]

$$F = A + B$$

Available [NAND gate]

$$\begin{aligned} F(A, B) &= (A \cdot B)' \\ &= A' + B' \end{aligned}$$

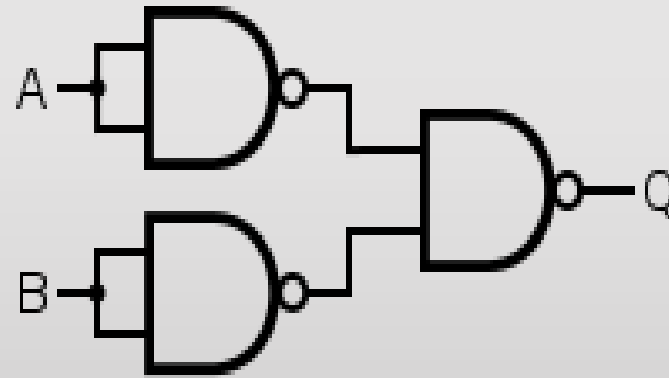
$$\begin{aligned} F(A', B') &= (A' \cdot B')' \\ &= A + B \end{aligned}$$



NAND as Universal Gates

Implementation of OR using NAND

- An OR gate is made by inverting the inputs to a NAND gate.

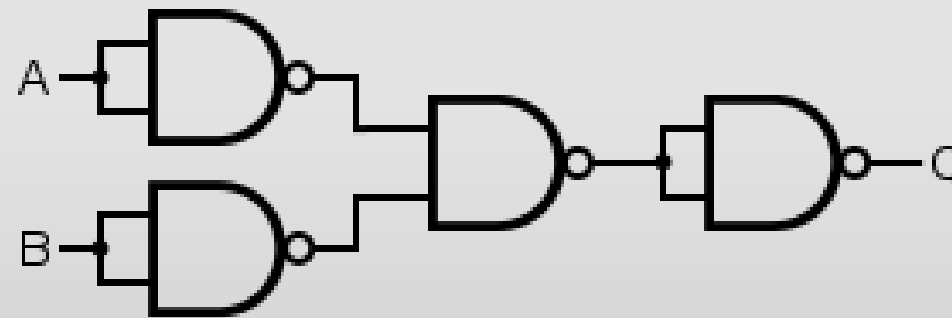


OR using NAND

NAND as Universal Gates

Implementation of NOR using NAND

A NOR gate is simply an inverted OR gate. So, we add NOT gate(using NAND) to OR gate made using NAND gates.



NOR using NAND

NOR as Universal Gates

Implementation of NOT using NOR

Desired [not gate]

$$F = A'$$

Available [Nor gate]

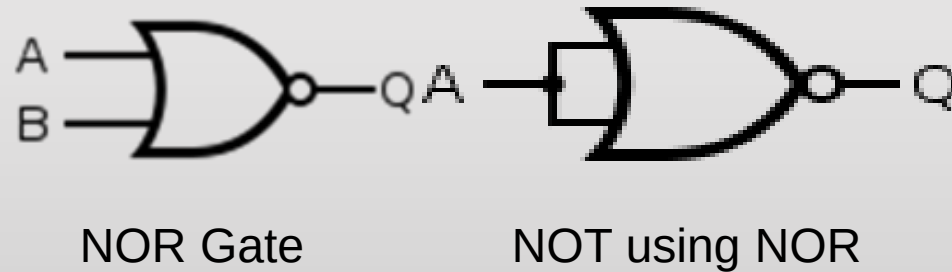
$$F(A,A) = (A+A)'$$

$$F = A'$$

NOR as Universal Gates

Implementation of NOT using NOR

NOT made by joining the inputs of a NOR gate.



NOR as Universal Gates

Implementation of OR using NOR

Desired [Or gate]

$$F = A+B$$

Available [NOR gate]

$$F = (A+B)'$$

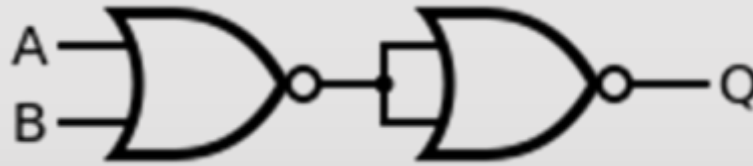
$$F = [(A+B)']'$$

$$F = (A+B)$$

NOR as Universal Gates

Implementation of OR using NOR

The OR gate is simply a NOR gate followed by another NOR gate



OR using NOR

NOR as Universal Gates

Implementation of AND using NOR

Desired [AND gate]

$$F = A.B$$

Available [NOR gate]

$$F(A,B) = (A+B)'$$

$$F = A'.B'$$

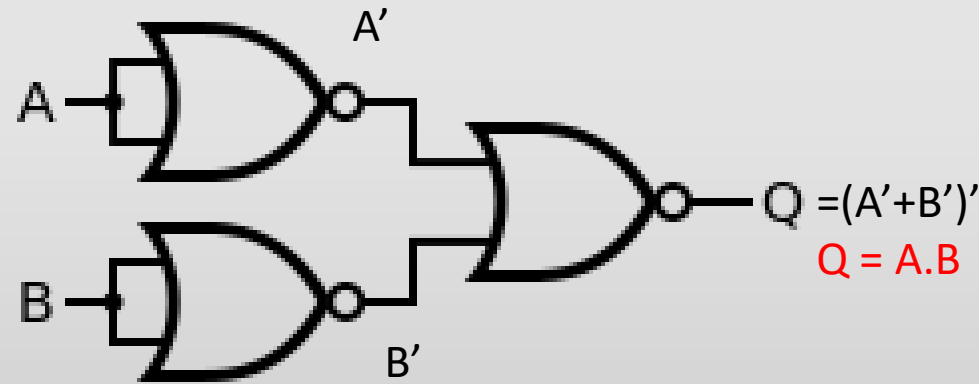
$$F(A',B') = (A'+B')'$$

$$F = A.B$$

NOR as Universal Gates

Implementation of AND using NOR

An AND gate is made by inverting the inputs to a NOR gate.

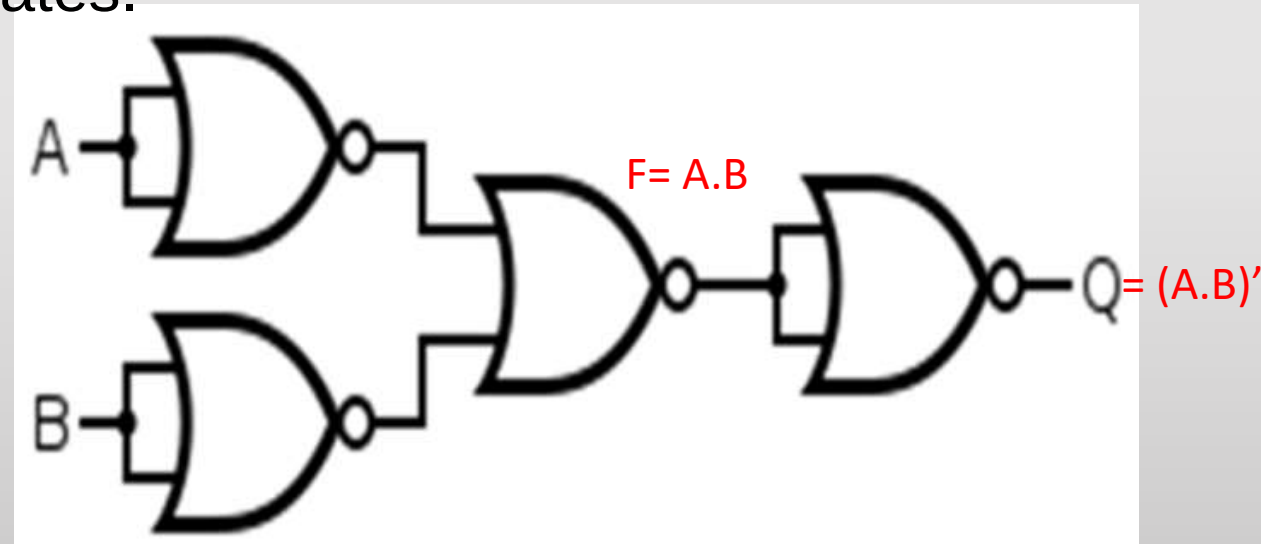


AND using NOR

NOR as Universal Gates

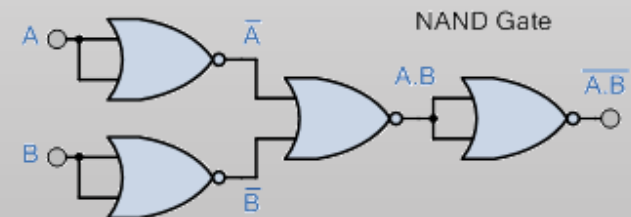
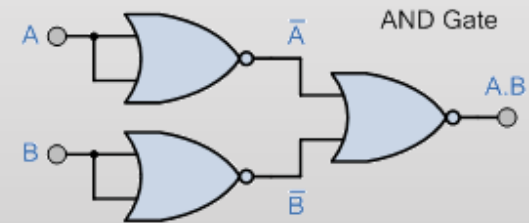
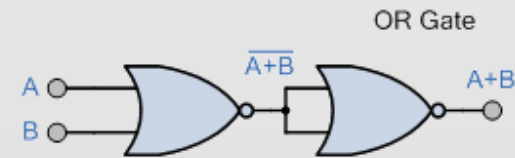
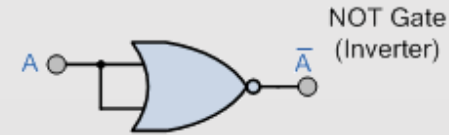
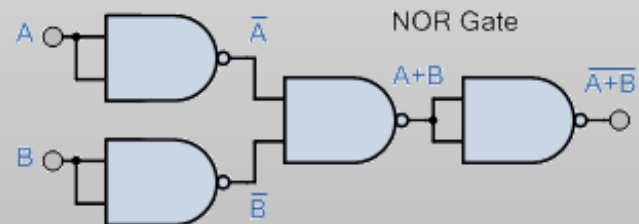
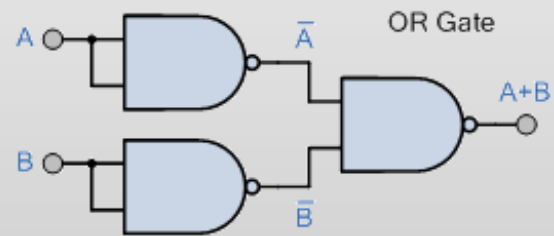
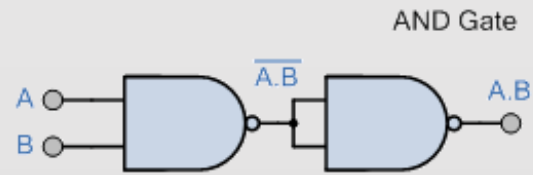
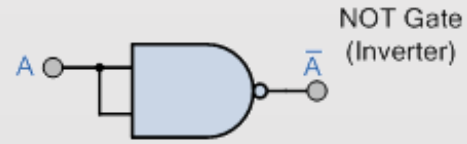
Implementation of NAND using NOR

A NAND gate is simply an inverted AND gate. So, we add NOT gate(using NOR) to AND gate made using NOR gates.



NAND using NOR

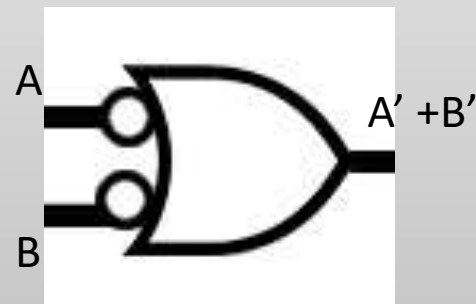
Summary



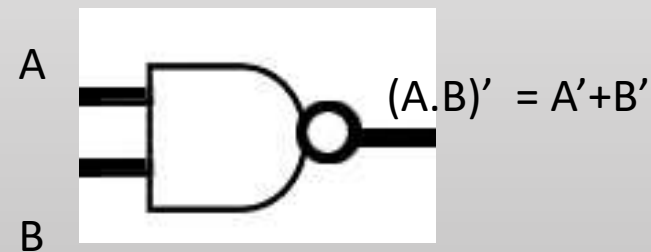
NAND and NOR Implementation

NAND Implementation

1. Draw the logic diagram using basic gates. Assume both forms of wires are available.
2. Replace all the the AND gates using AND-invert symbol.
3. Replace all the OR gates using invert-OR symbol.
4. Cancel all the circles(inverters) in the same line. If any circle is remaining, replace it using Inverter of NAND gate.
5. Replace all the remaining gates with AND-invert symbol.



Invert-OR



AND-invert

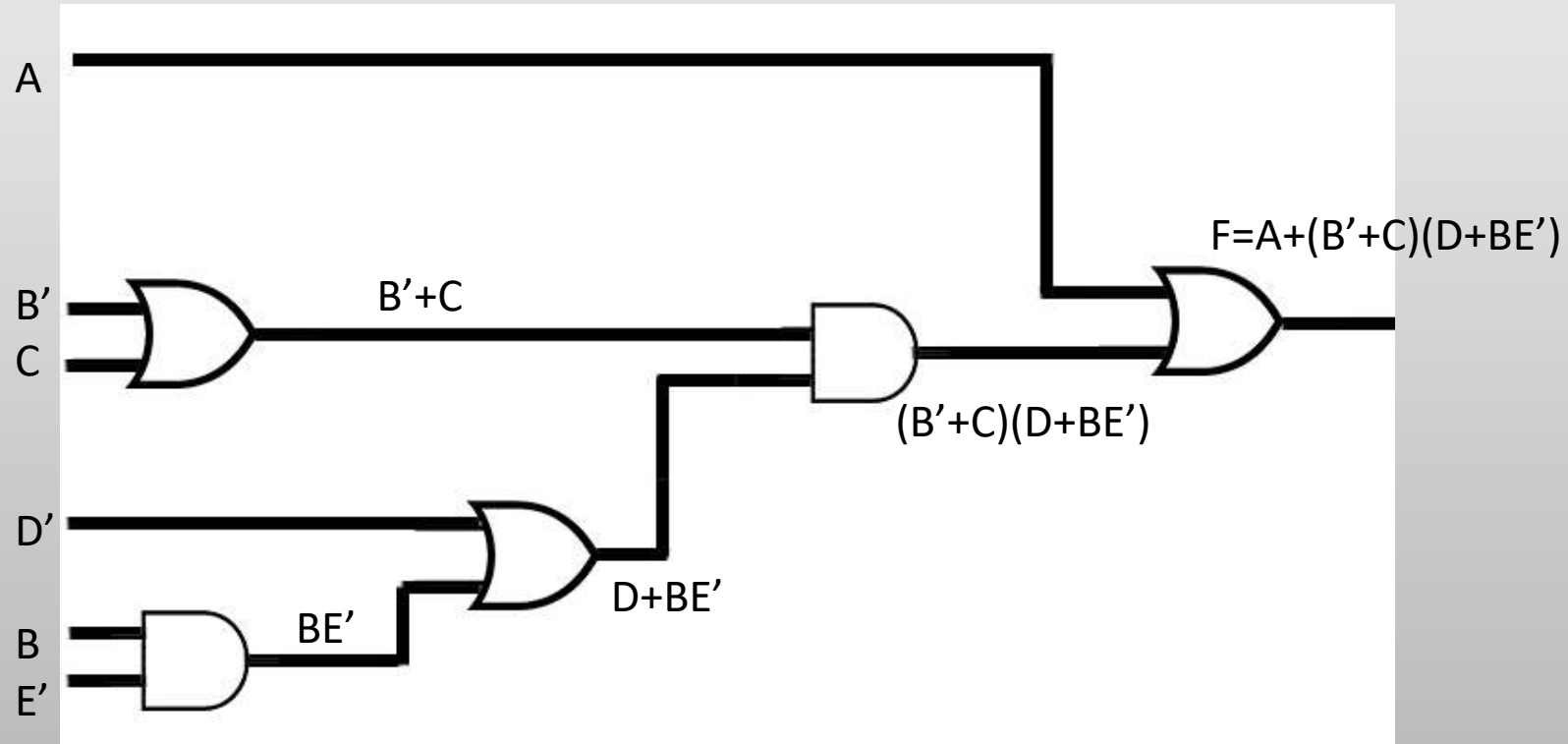
NAND and NOR Implementation

NAND Implementation

- Implement the Boolean function $F = A + (B' + C)(D' + BE')$ using NAND gates only.

Step 1

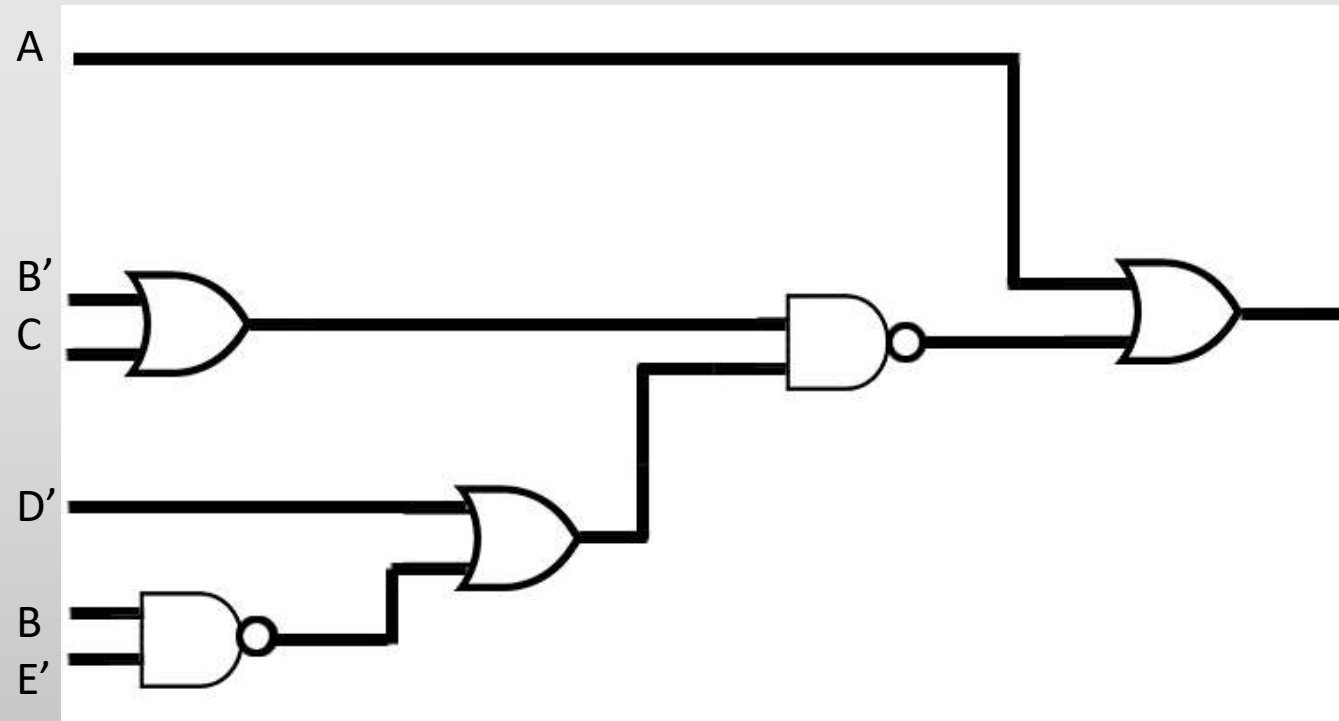
Draw the logic diagram using basic gates. Assume both forms of wires are available.



NAND and NOR Implementation

Step 2

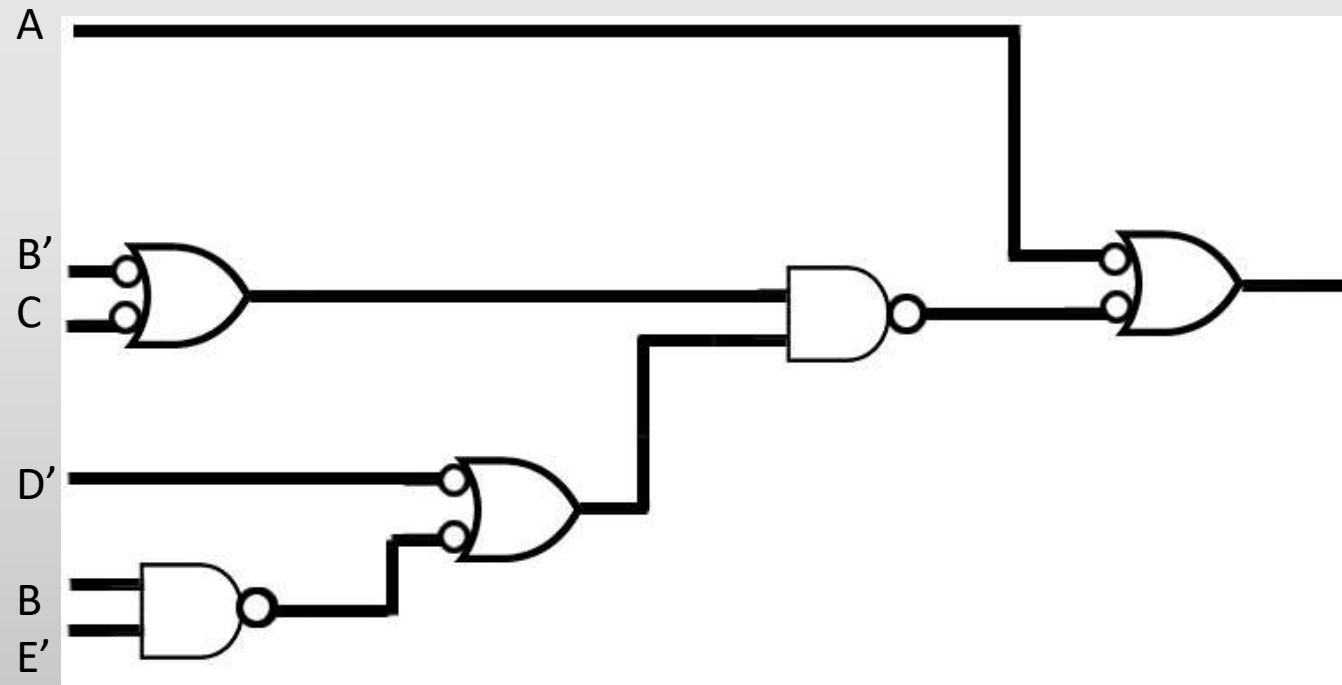
Replace all the AND gates using AND-invert symbol.



NAND and NOR Implementation

Step 3

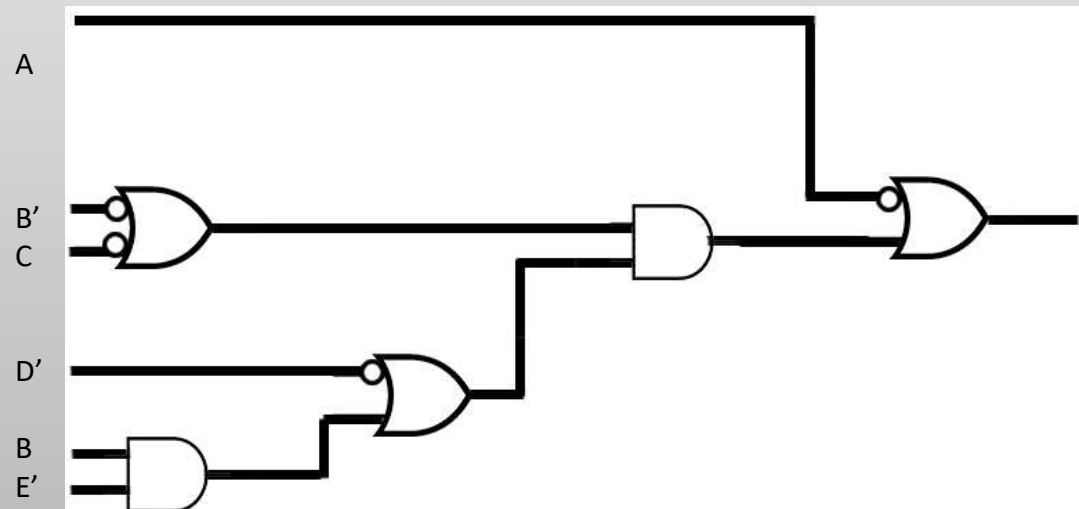
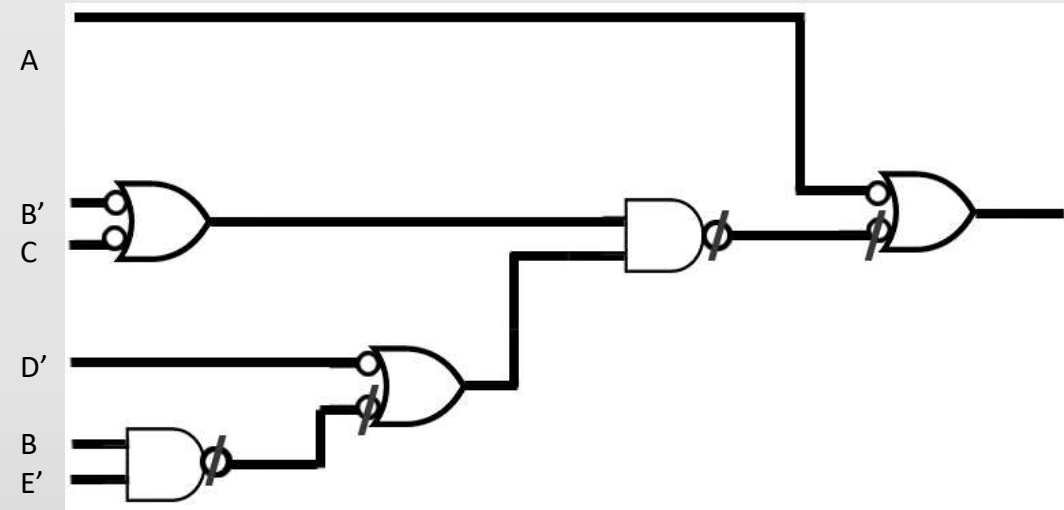
Replace all the OR gates using invert-OR symbol.



NAND and NOR Implementation

Step 4

Cancel all the circles in the same line.

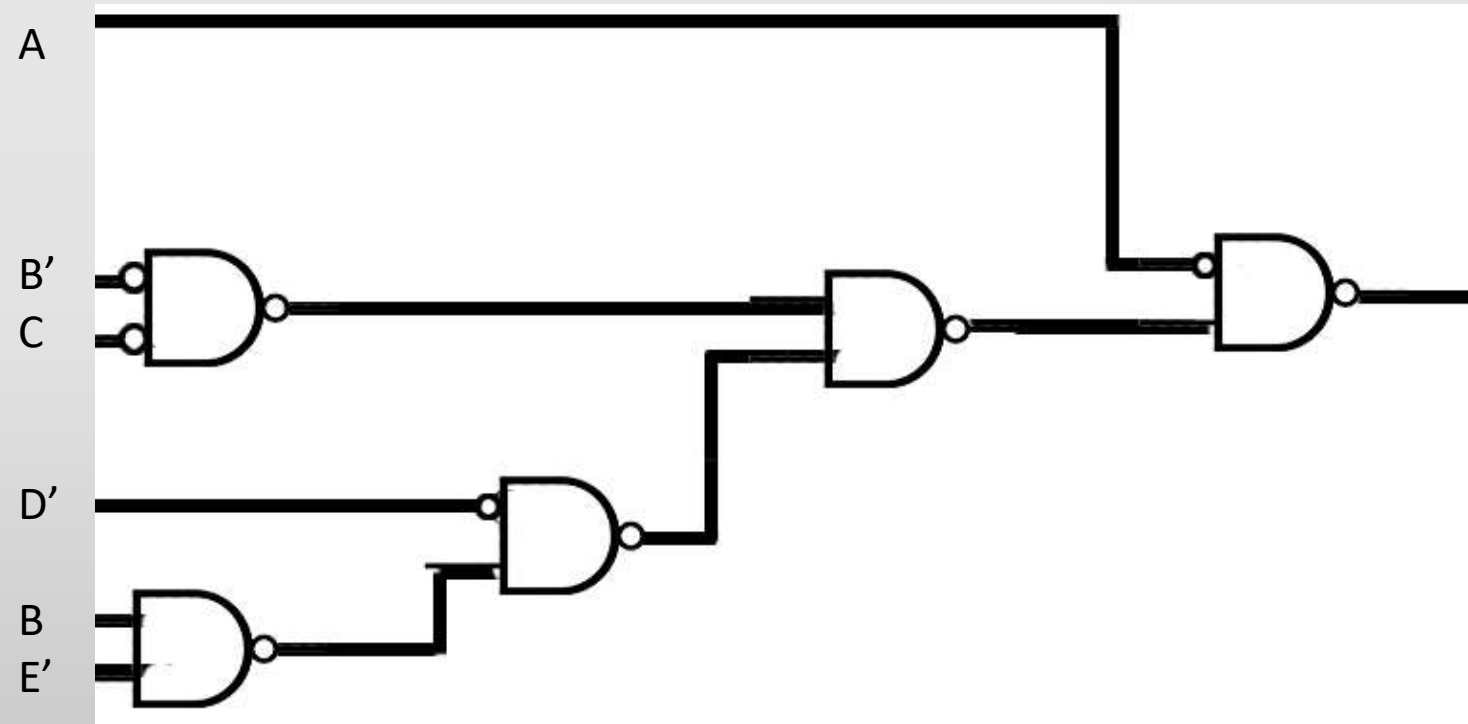


NAND and NOR Implementation

Step 5

Replace all the remaining gates with AND-invert gates.

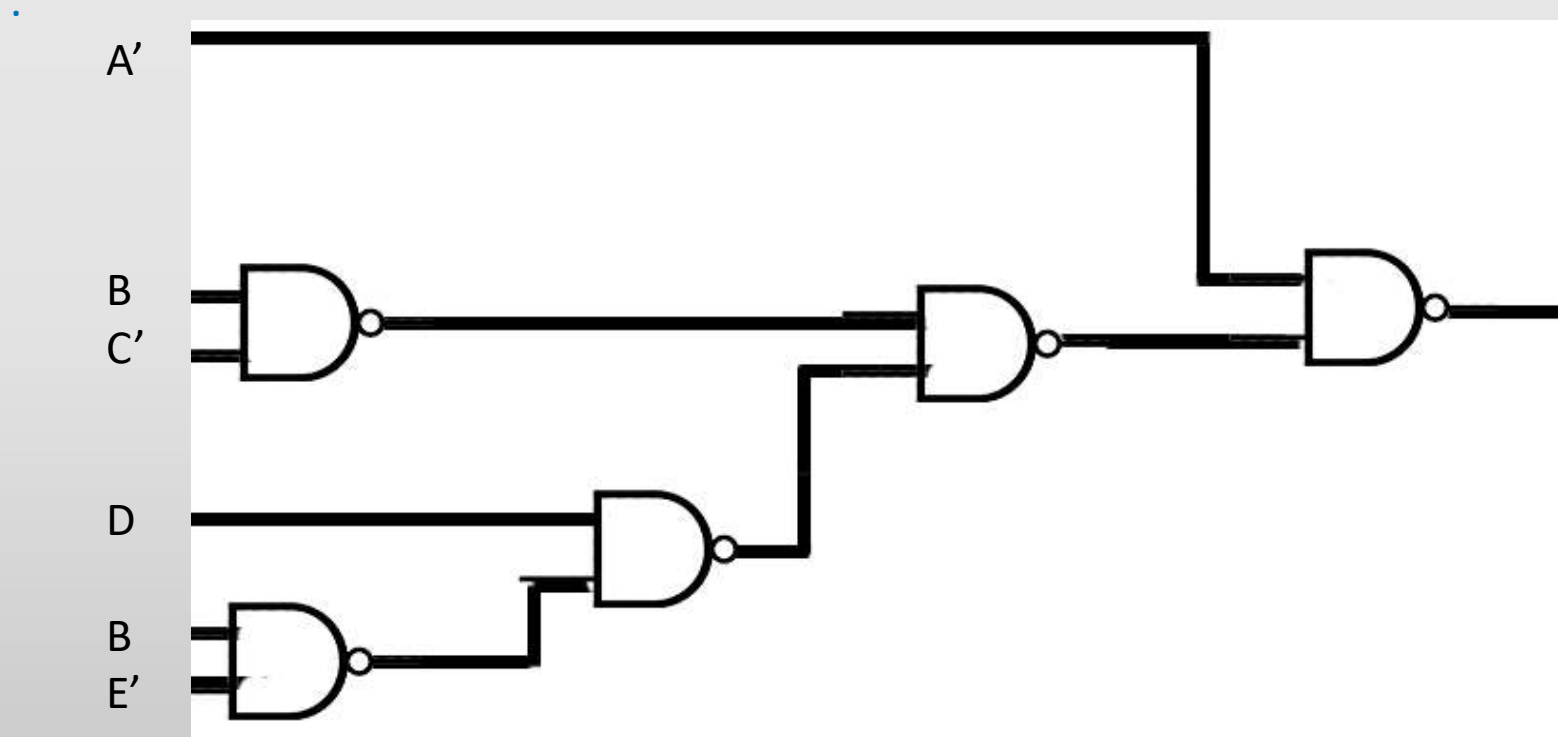
.



NAND and NOR Implementation

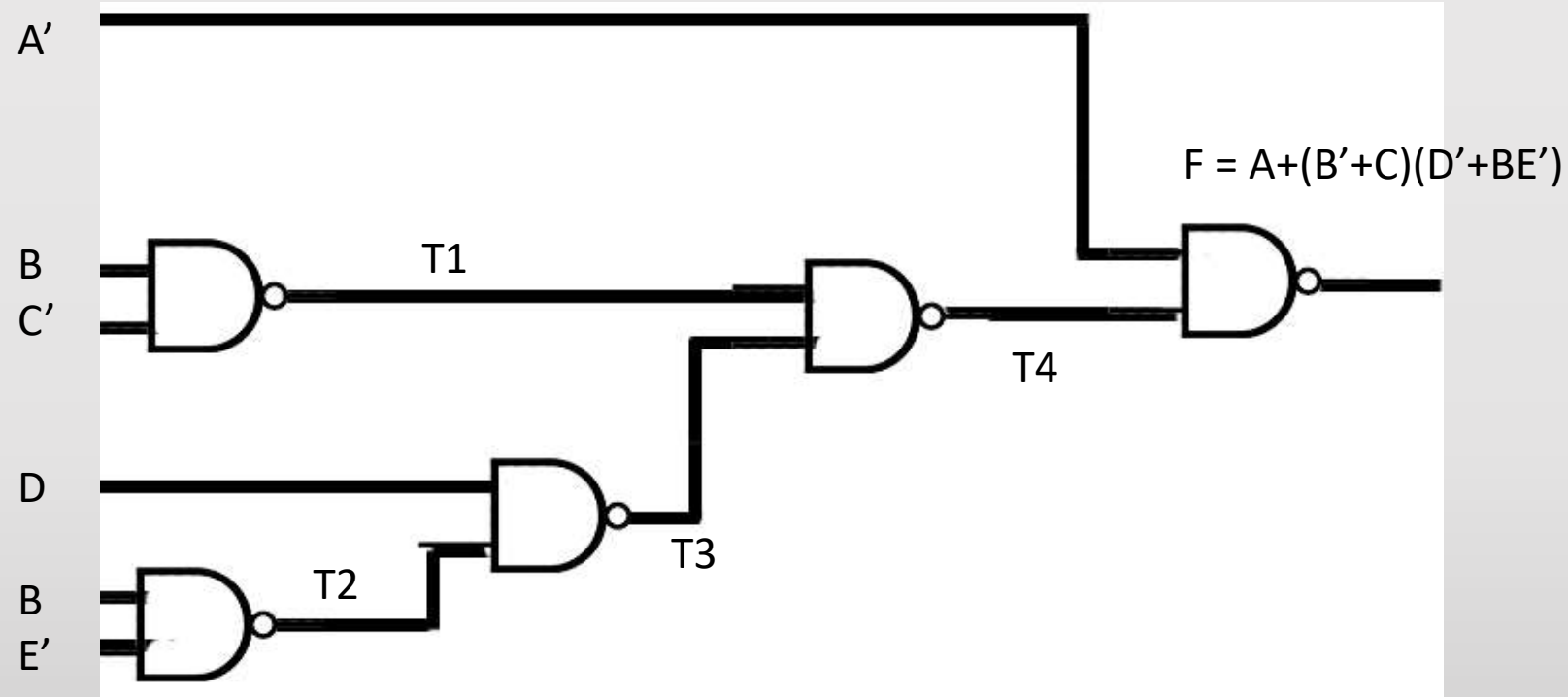
Step 6

Simply, replace remaining circles with complementing the input or replace it with an inverter using NAND gate.



NAND and NOR Implementation

Finally, verify the output.



NAND and NOR Implementation

Finally, verify the output.

$$T1 = (B.C')' = B' + C$$

$$T2 = (B.E')' = B' + E$$

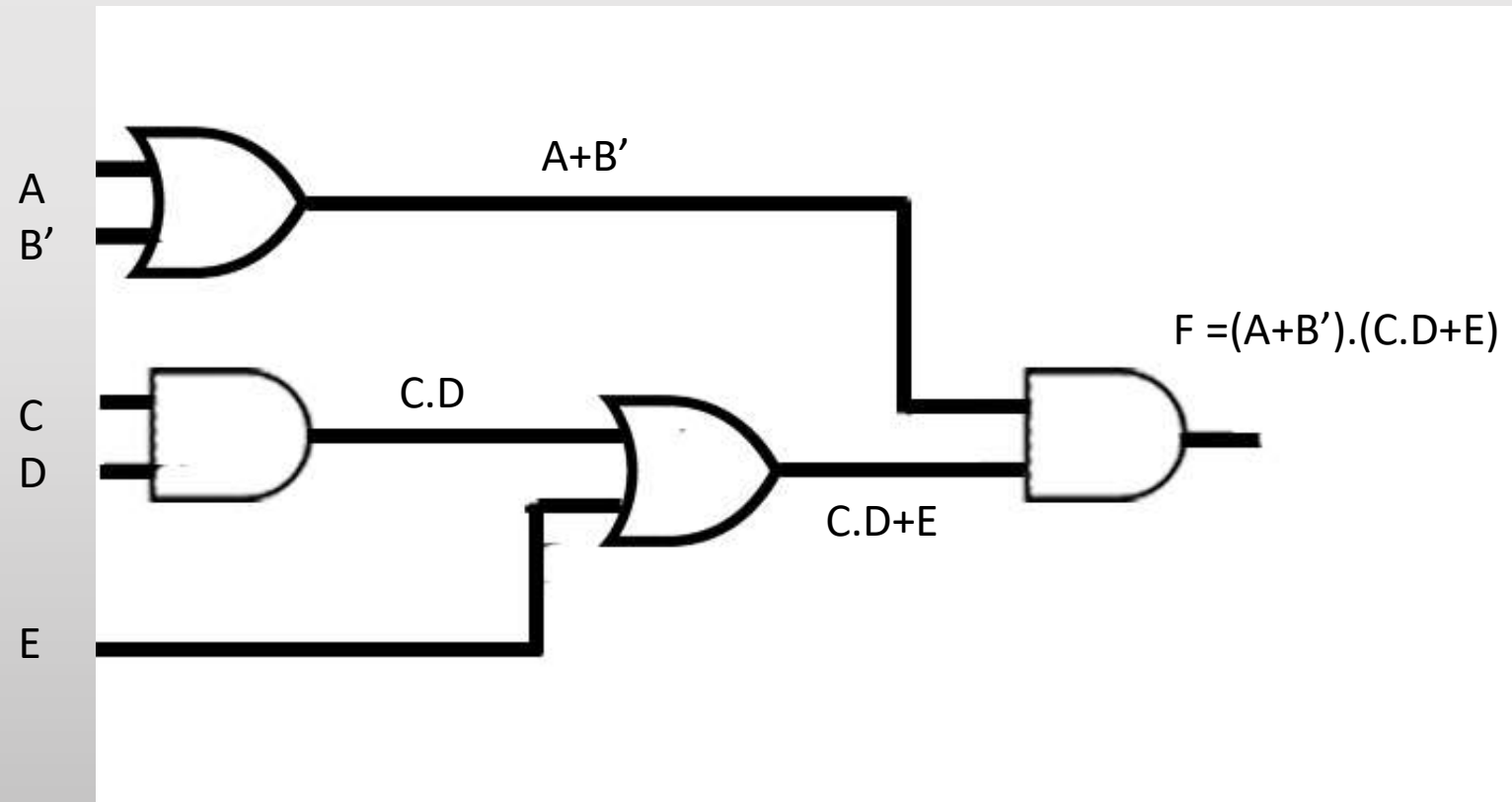
$$T3 = (D.T2)' = D' + T2'$$

$$\begin{aligned} T4 &= (T1.T3)' = T1' + T3' = (B' + C)' + (D' + T2')' \\ &= (B' + C)' + (D' + (B' + E)')' \\ &= (B' + C)' + (D' + B.E')' \end{aligned}$$

$$\begin{aligned} F &= (A' . T4)' = A + T4' \\ &= A + [(B' + C)' + (D' + B.E')']' \\ &= A + (B' + C). (D' + B.E') \end{aligned}$$

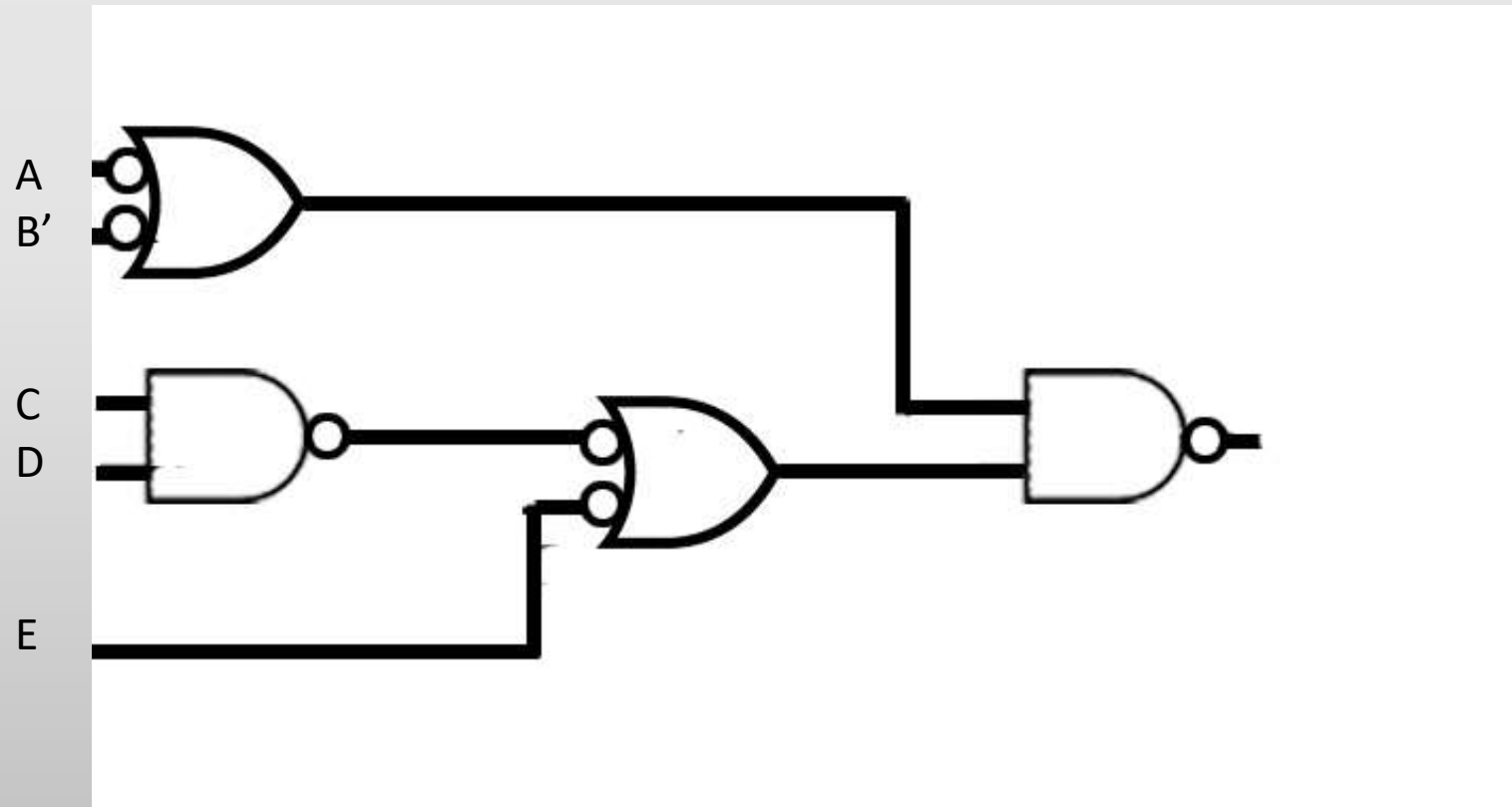
NAND and NOR Implementation

- Implement the Boolean function $F = (A+B')(CD+E)$ using NAND gates only.



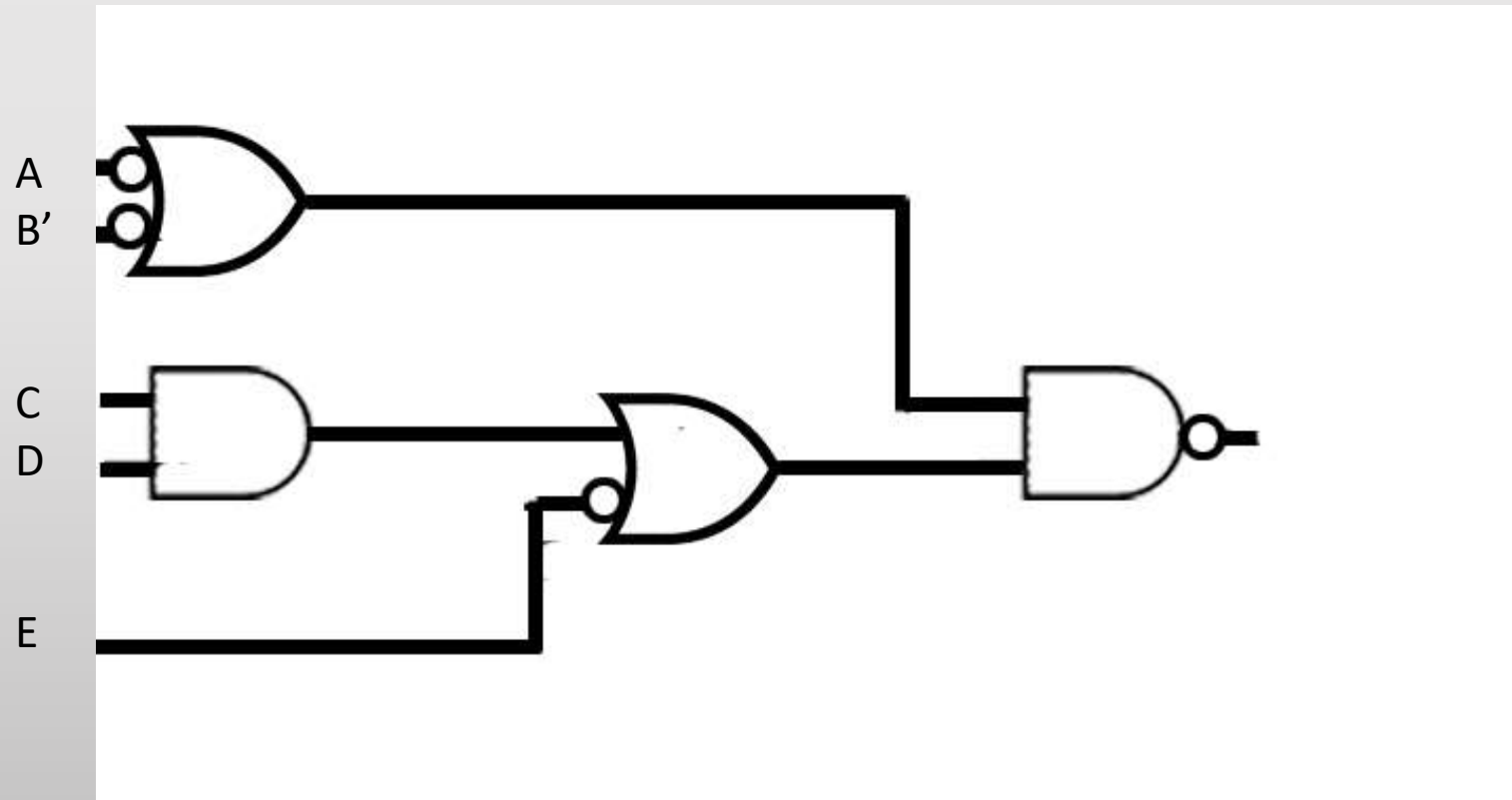
NAND and NOR Implementation

- Implement the Boolean function $F = (A+B')(CD+E)$ using NAND gates only.



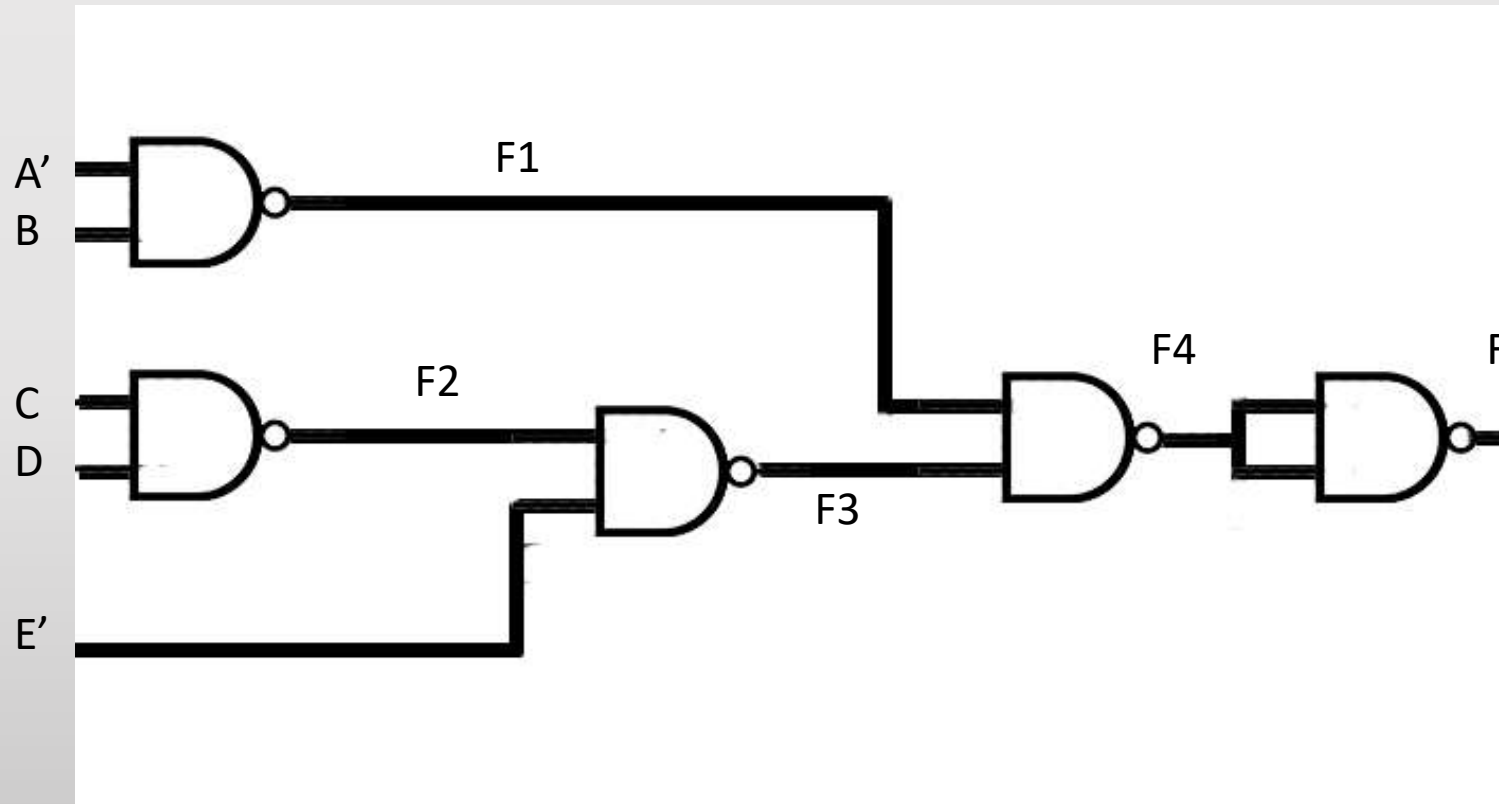
NAND and NOR Implementation

- Implement the Boolean function $F = (A+B')(CD+E)$ using NAND gates only.



NAND and NOR Implementation

- Implement the Boolean function $F = (A+B')(CD+E)$ using NAND gates only.



NAND and NOR Implementation

- Implement the Boolean function $F = (A+B')(CD+E)$ using NAND gates only.

$$F1 = (A'.B)' = A+B'$$

$$F2 = (C.D)' = C'+D'$$

$$\begin{aligned} F3 &= (F2.E')' = F2'+E \\ &= C.D+E \end{aligned}$$

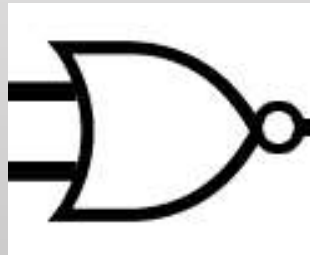
$$\begin{aligned} F4 &= (F1.F3)' = F1' + F3' \\ &= (A'.B) + (C.D+E)' \\ &= (A'.B) + ((C.D)' . E') \\ &= (A'.B) + ((C'+D') . E') \end{aligned}$$

$$\begin{aligned} F &= F4' = [(A'.B) + ((C'+D') . E')] \\ &= (A'.B)' . (C'+D')' + E \\ &= A+B' . (C.D) + E \\ &= (A+B')(CD+E) \end{aligned}$$

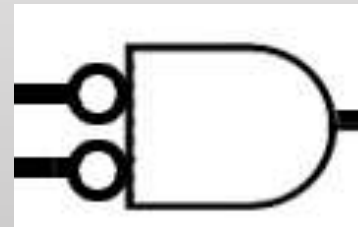
NAND and NOR Implementation

NOR Implementation

1. Draw the logic diagram using basic gates. Assume both forms of wires are available.
2. Replace all the the OR gates using OR-invert symbol.
3. Replace all the AND gates using invert-AND symbol.
4. Cancel all the circles in the same line. If any circle is remaining, replace it using Inverter or NOR gate.
5. Replace all the remaining gates with OR-invert signal.



OR-invert



invert-AND

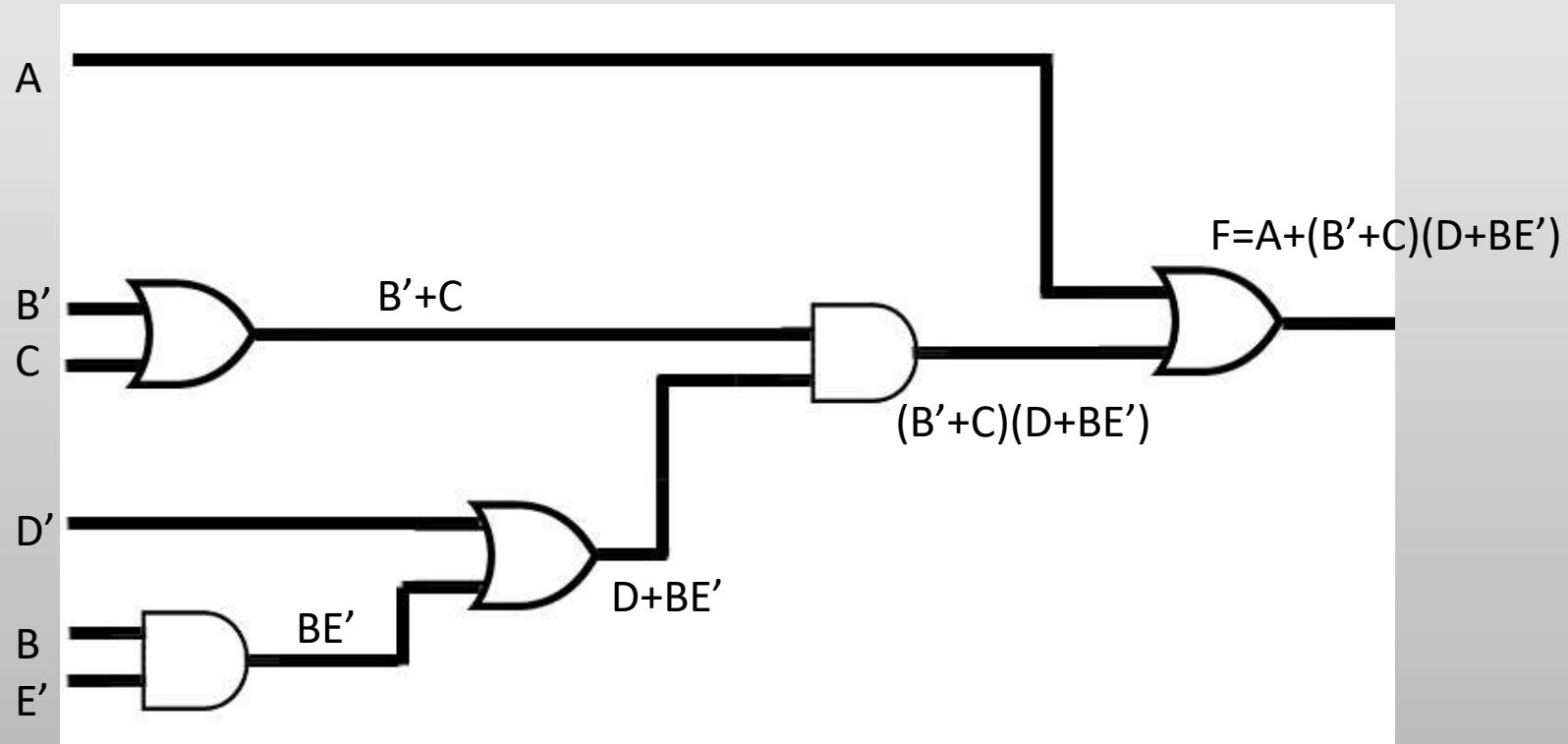
NAND and NOR Implementation

NOR Implementation

- Implement the Boolean function $F = A + (B' + C)(D' + BE')$ using NOR gates only.

Step 1

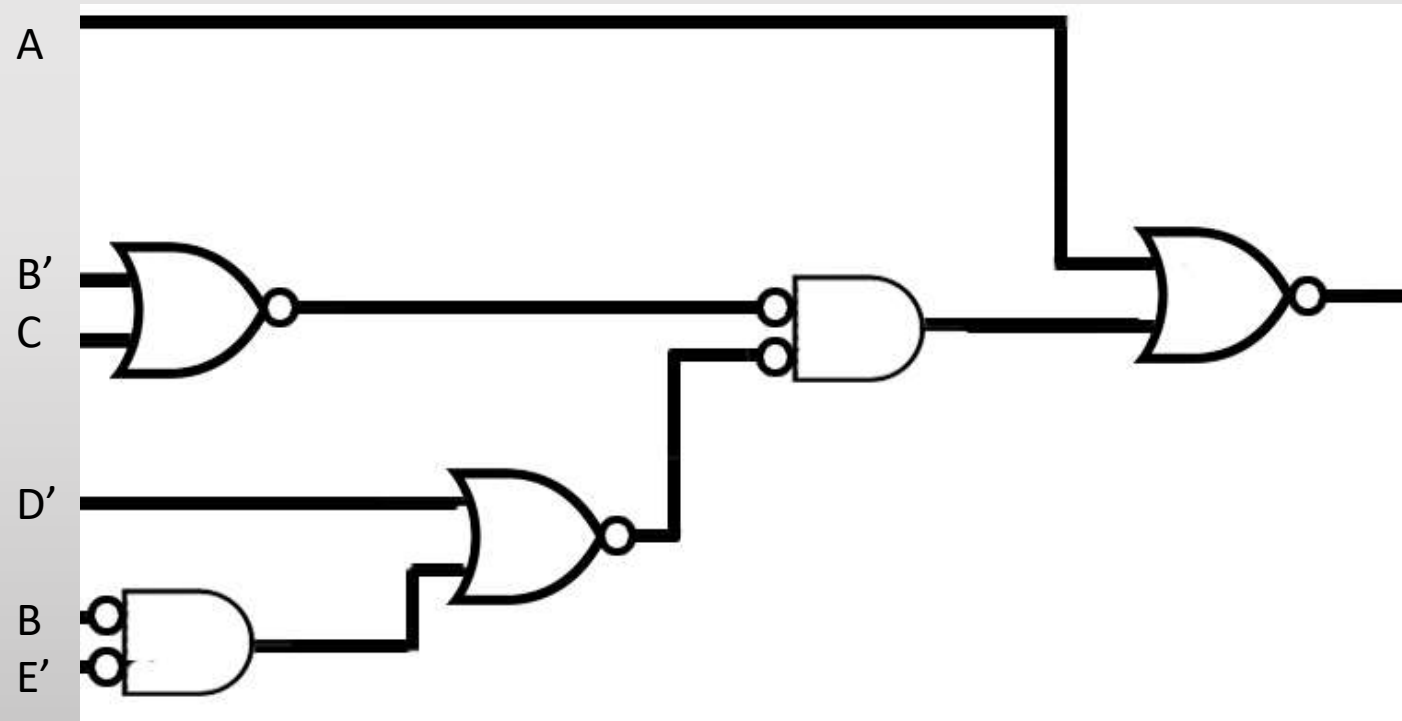
Draw the logic diagram using basic gates. Assume both forms of wires are available.



NAND and NOR Implementation

Step 2

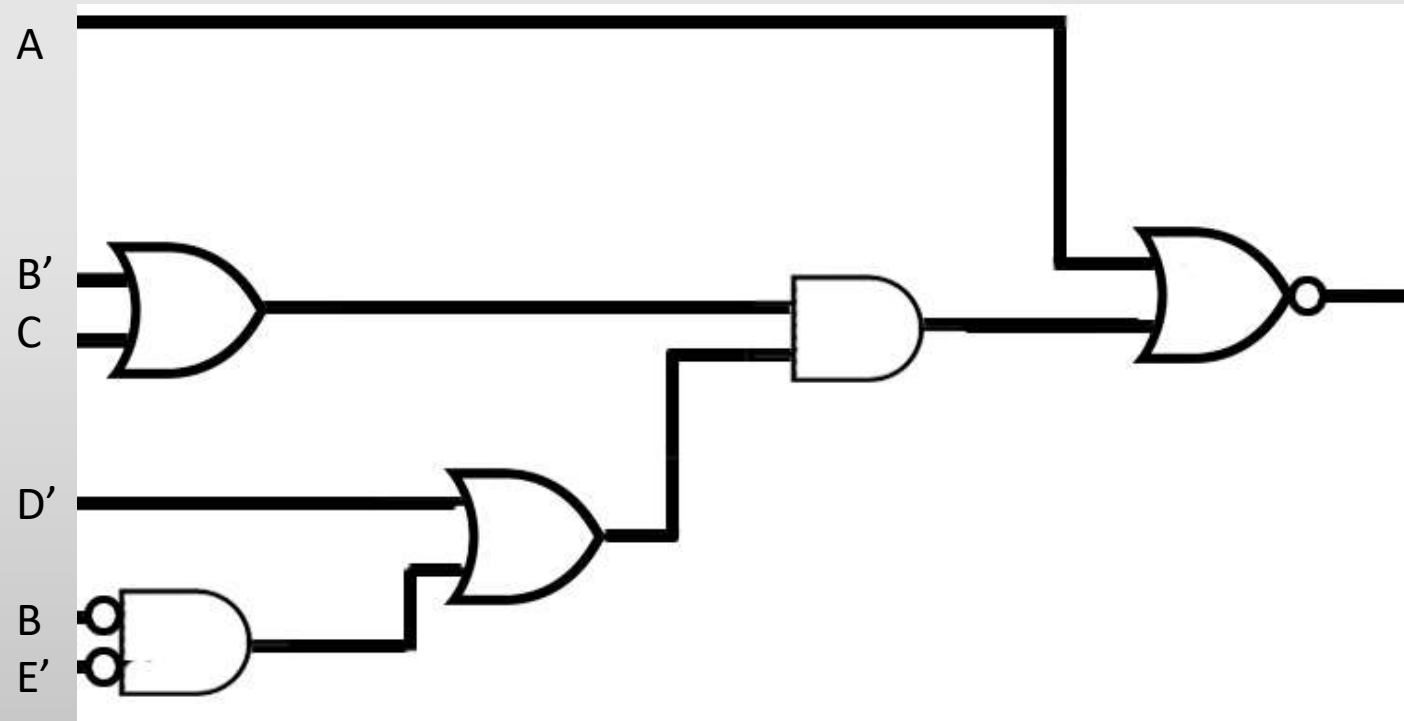
Replace all the AND gates using invert-AND symbol & OR with OR-invert symbol



NAND and NOR Implementation

Step 3

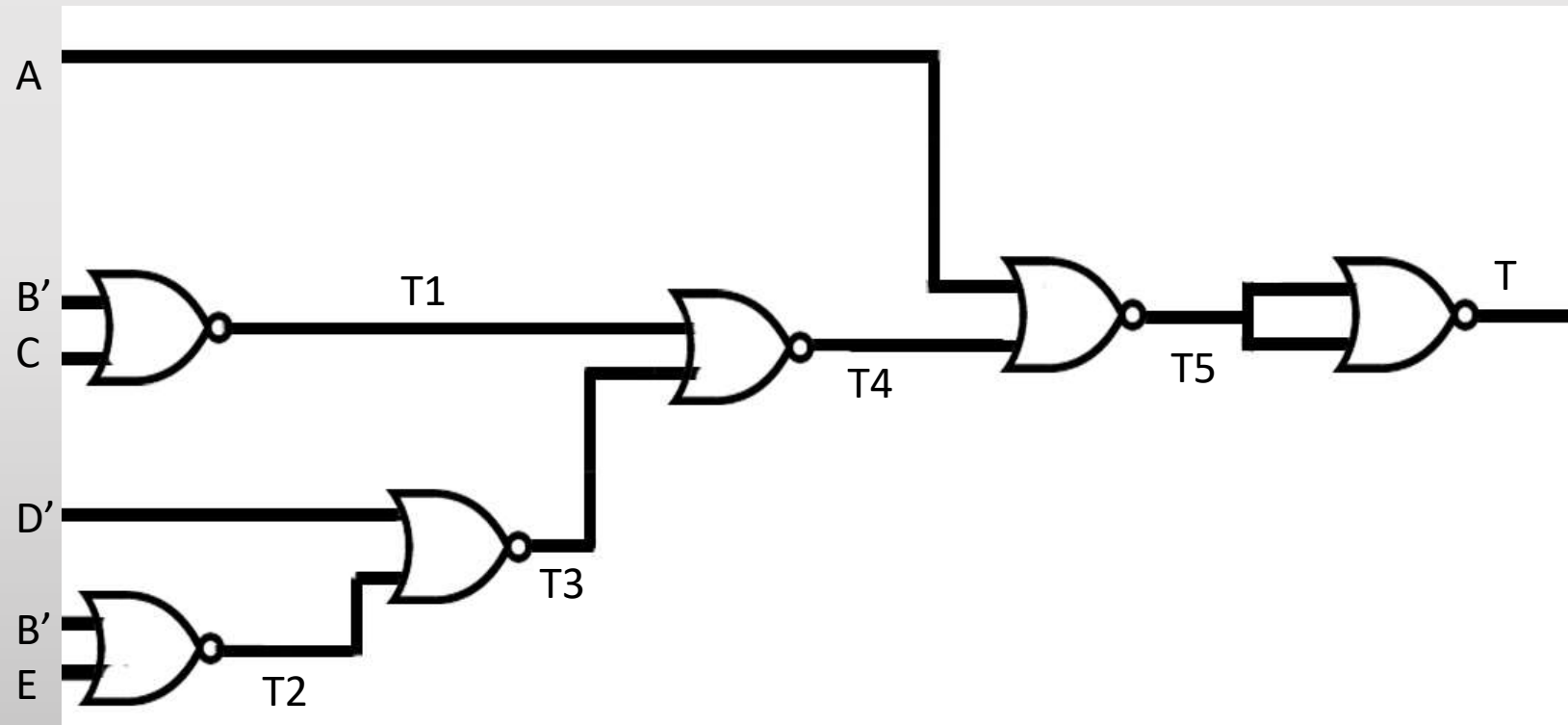
Cancel all the circles in the same line.



NAND and NOR Implementation

Step 4

Remove the remaining circles using inverters(using NOR) and replace all the gates with OR-invert.



NAND and NOR Implementation

- Implement the Boolean function $F = (A+B')(CD+E)$ using NOR gates only.
- Implement the Boolean function $F = A'B + AB'$ using NOR only and NAND only.
- Implement the Boolean function $F = AB + A'B'$ using NOR only and NAND only.

Combinational Logic with MSI & LSI

MSI & LSI Components in Combinational Logic

- SSI – Small Scale Integration

No. of gates – up to 10. For example: IC7400, IC7404 ,etc

- MSI – Medium Scale Integration

No. of gates – 10-100. For example: MUX, DEMUX, Encoder, etc

- LSI – Large Scale Integration

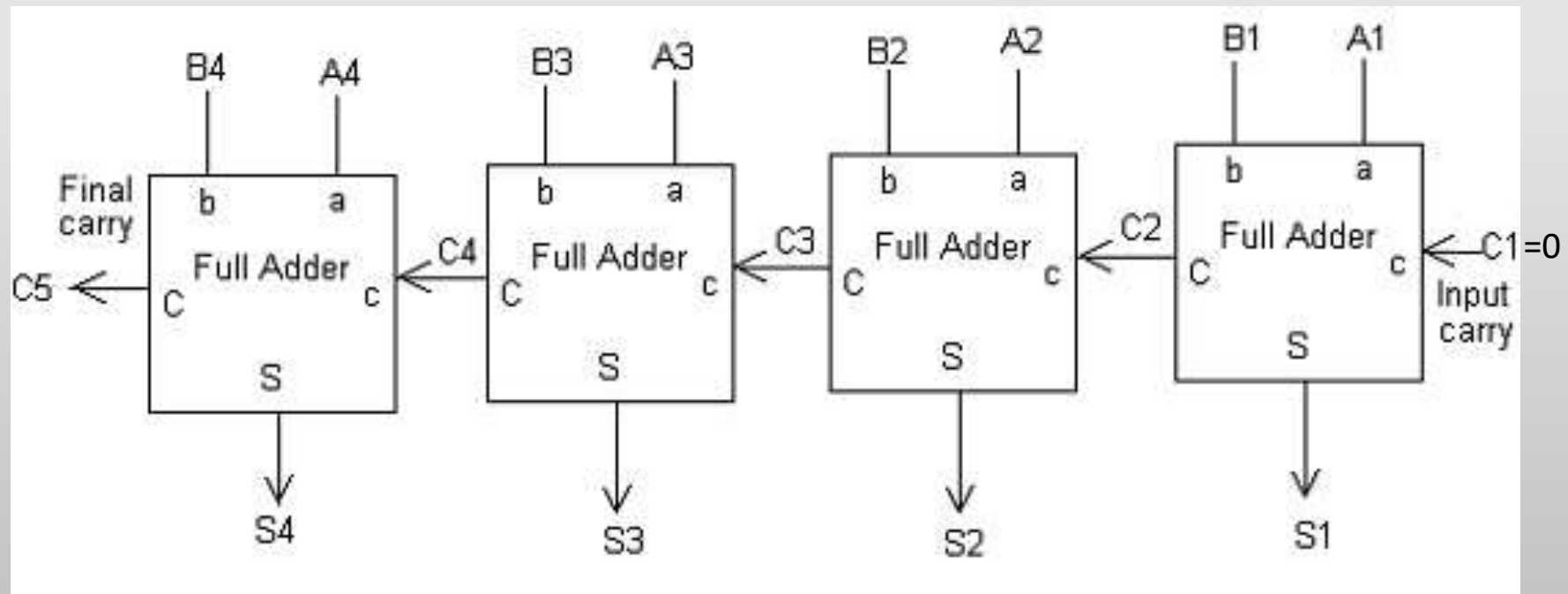
No. of gates – 100-1000. For example: ROM,PLA, etc

- VLSI – Very Large Scale Integration

No. of gates – more than 1000. For example: Microprocessor

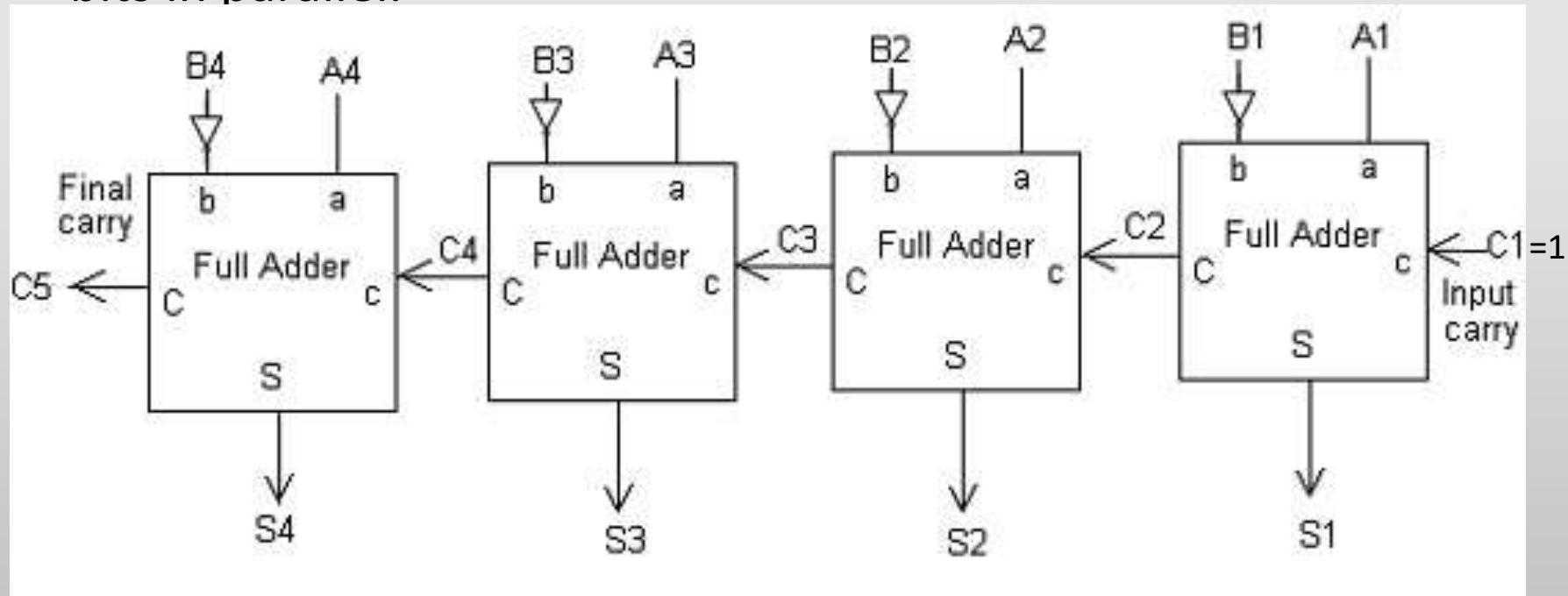
4-bit Binary Parallel Adder

Parallel Adder is a digital circuit capable of finding the arithmetic sum of two binary numbers that is greater than one bit in length by operating on corresponding pairs of bits in parallel.



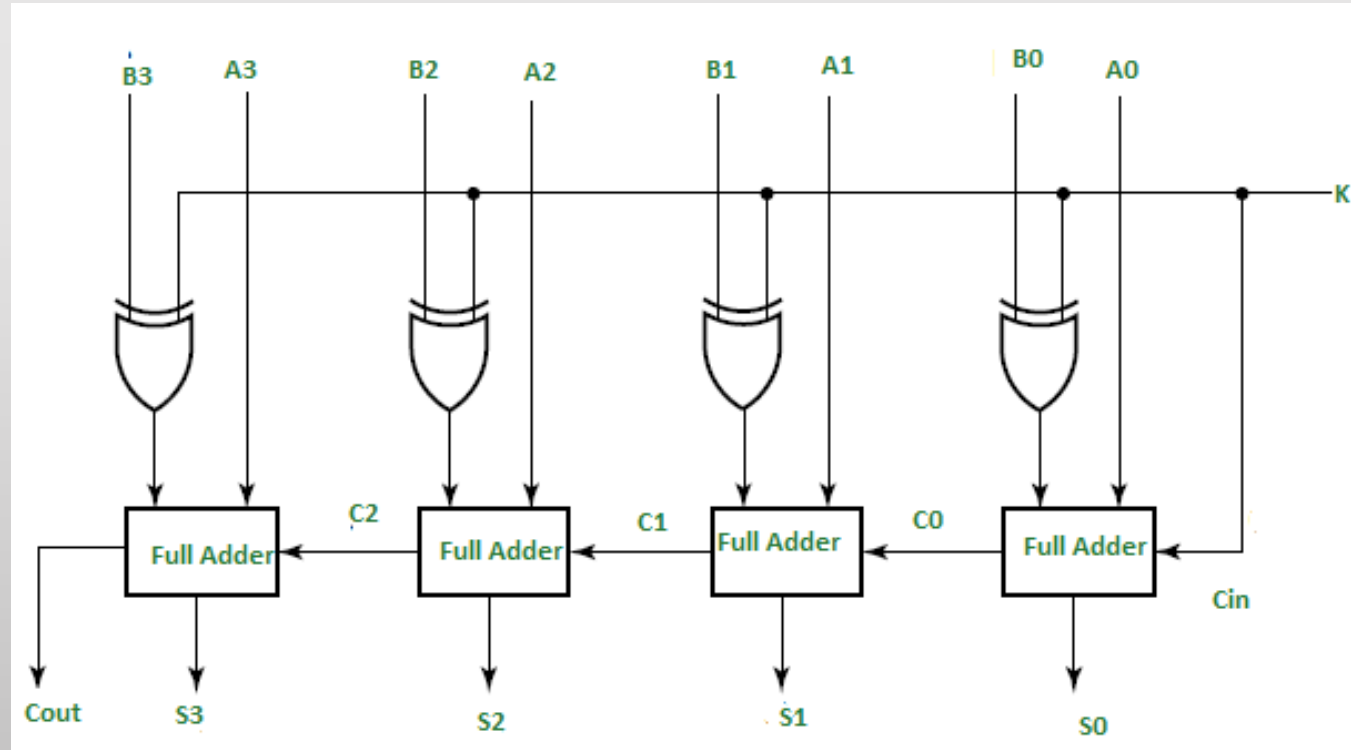
4-bit Binary Parallel Subtractor

A Parallel Subtractor is a digital circuit capable of finding the arithmetic difference of two binary numbers that is greater than one bit in length by operating on corresponding pairs of bits in parallel.



4-bit Binary Parallel Adder/Subtractor

A Binary Adder-Subtractor is circuit which is capable of both addition and subtraction of binary numbers in one circuit itself.



X-OR Gate

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

The above circuit works as adder when $k=0$ and subtractor when $k=1$

BCD Adder

BCD adder (sum up to 9)

Dec					
0	0	0	0	0	0
1	0	0	0	0	1
2	0	0	0	1	0
3	0	0	0	1	1
4	0	0	1	0	0
5	0	0	1	0	1
6	0	0	1	1	0
7	0	0	1	1	1
8	0	1	0	0	0
9	0	1	0	0	1

BCD Adder

BCD adder (sum is 10 to 19)

10	1	0	0	0	0
11	1	0	0	0	1
12	1	0	0	1	0
13	1	0	0	1	1
14	1	0	1	0	0
15	1	0	1	0	1
16	1	0	1	1	0
17	1	0	1	1	1
18	1	1	0	0	0
19	1	1	0	0	1

BCD Adder

Algorithm for BCD adder

- If sum is up to 9
 - ✓ Use the regular Adder.

- If the sum > 9
 - ✓ Use the regular adder and add 6 to the result

BCD Adder

Decimal	Binary Sum					BCD sum				
	C'	S' ₃	S' ₂	S' ₁	S' ₀	C	S ₃	S ₂	S ₁	S ₀
0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	1
2	0	0	0	1	0	0	0	0	1	0
3	0	0	0	1	1	0	0	0	1	1
4	0	0	1	0	0	0	0	1	0	0
5	0	0	1	0	1	0	0	1	0	1
6	0	0	1	1	0	0	0	1	1	0
7	0	0	1	1	1	0	0	1	1	1
8	0	1	0	0	0	0	1	0	0	0
9	0	1	0	0	1	0	1	0	0	1
10	0	1	0	1	0	1	0	0	0	0
11	0	1	0	1	1	1	0	0	0	1
12	0	1	1	0	0	1	0	0	1	0
13	0	1	1	0	1	1	0	0	1	1
14	0	1	1	1	0	1	0	1	0	0
15	0	1	1	1	1	1	0	1	0	1
16	1	0	0	0	0	1	0	1	1	0
17	1	0	0	0	1	1	0	1	1	1
18	1	0	0	1	0	1	1	0	0	0

BCD Adder

Dec		C'	S3 ,	S2'	S1 ,	S0'		C	S3	S2	S1	S0
10		0	1	0	1	0		1	0	0	0	0
11		0	1	0	1	1		1	0	0	0	1
12		0	1	1	0	0		1	0	0	1	0
13		0	1	1	0	1		1	0	0	1	1
14		0	1	1	1	0		1	0	1	0	0
15		0	1	1	1	1		1	0	1	0	1
16		1	0	0	0	0		1	0	1	1	0
17		1	0	0	0	1		1	0	1	1	1
18		1	0	0	1	0		1	1	0	0	0

Add '0110' if :

$$C' = 1$$

$$S3' \cdot S1' = 1$$

$$S3' \cdot (S2' + S1') = 1$$

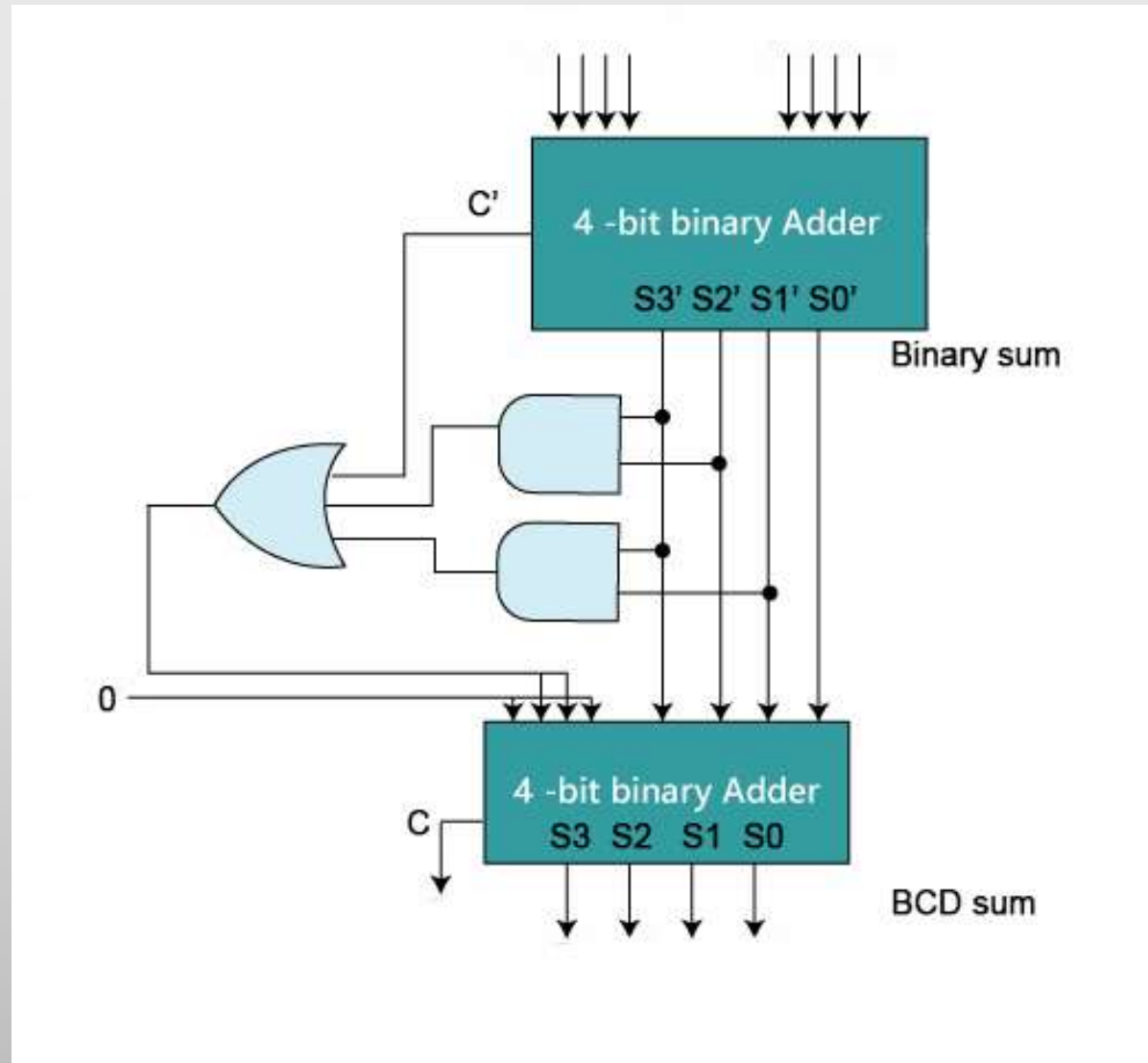
$$C' + S3' \cdot S1' + S3' \cdot (S2' + S1')$$

$$= C' + S3' \cdot S1' + S3' \cdot S2' + S3' \cdot S1'$$

$$= C' + S3' \cdot S1' + S3' \cdot S2$$

If $[C' + S3' \cdot S1' + S3' \cdot S2] = 1$; add '0110'

BCD Adder



Magnitude Comparator

A magnitude comparator is a hardware electronic device that takes two numbers as input in binary form and determines whether one number

- is greater than,
- less than or
- equal to the other number.

It has three output terminals, one each for equality

$A = B$

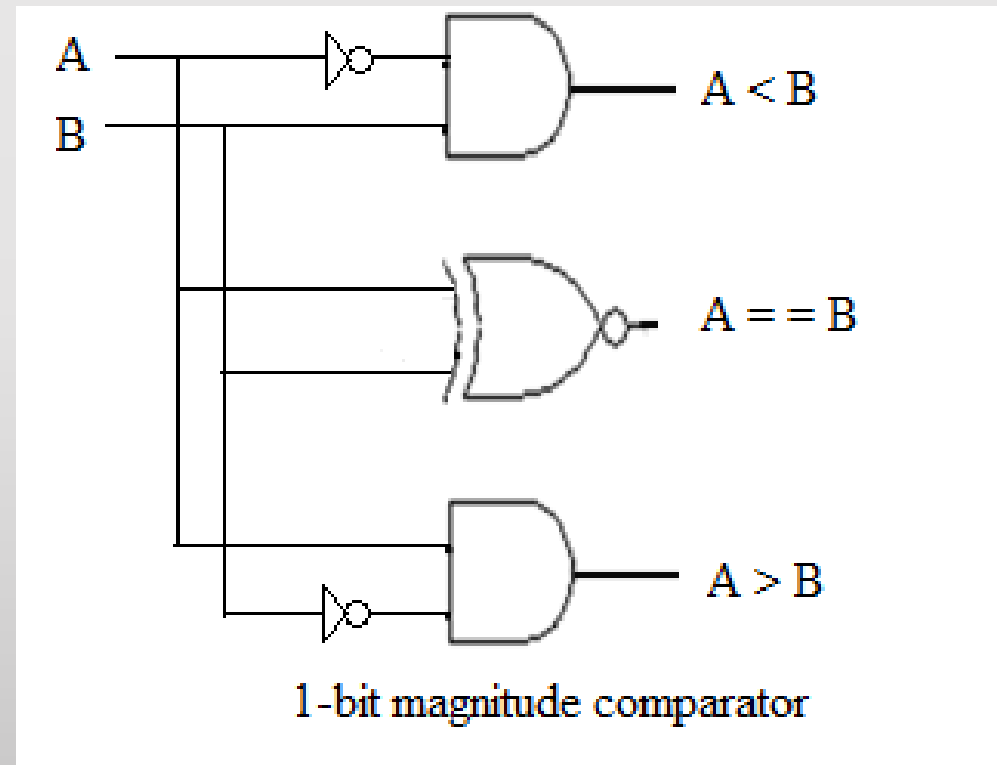
$A > B$

$A < B$

1-bit Magnitude Comparator

Inputs		Outputs		
A	B	A>B	A=B	A<B
0	0	0	1	0
0	1	0	0	1
1	0	1	0	0
1	1	0	1	0

1-bit Magnitude Comparator



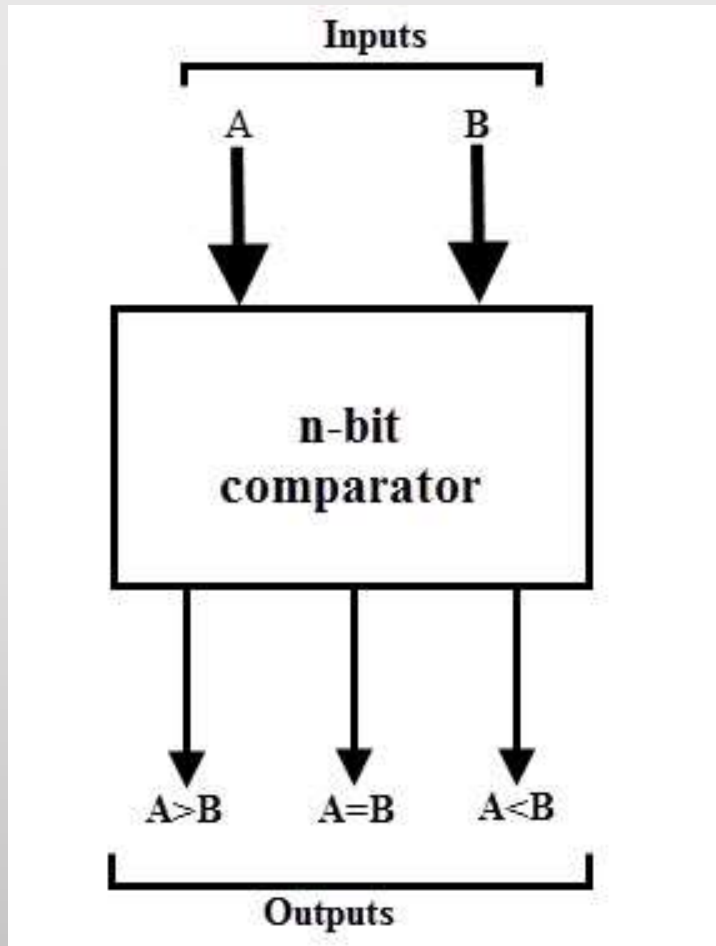
2-bit Magnitude Comparator

Inputs				Outputs		
A		B				
A1	A0	B1	B0	A>B	A=B	A<B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1

Inputs				Outputs		
A		B				
A1	A0	B1	B0	A>B	A=B	A<B
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	1

2-bit Magnitude Comparator

4-bit Magnitude Comparator



A → A3 A2 A1 A0

B → B3 B2 B1 B0

A → 1001

B → 0111

X-Nor Gate

A	B	F
0	0	1
0	1	0
1	0	0
1	1	1

4-bit Magnitude Comparator

$F1 \rightarrow A=B$

$A3=B3; A2=B2; A1=B1; A0=B0$

i.e. $A_i=B_i$

Using X-Nor gate:

$$X_i = AB + A'B' = (A'B + AB')'$$

If $F1 = x0.x1.x2.x3. = 1; A=B$

$F2 \rightarrow A>B$

$A3>B3 \quad [A3=1, B3=0]$

$A3=B3; A2>B2 \quad [A2=1, B2=0]$

$A3=B3; A2=B2; A1>B1 \quad [A1=1, B1=0]$

$A3=B3; A2=B2; A1=B1; A0>B0 \quad [A0=1, B0=0]$

If $F2 = A3.B3' + x3.A2.B2' + x3.x2.A1.B1' + x3.x2.x1.A0.B0' = 1; A>B$

4-bit Magnitude Comparator

$F3 \rightarrow A < B$

$A3 < B3$ [$A3=0, B3=1$]

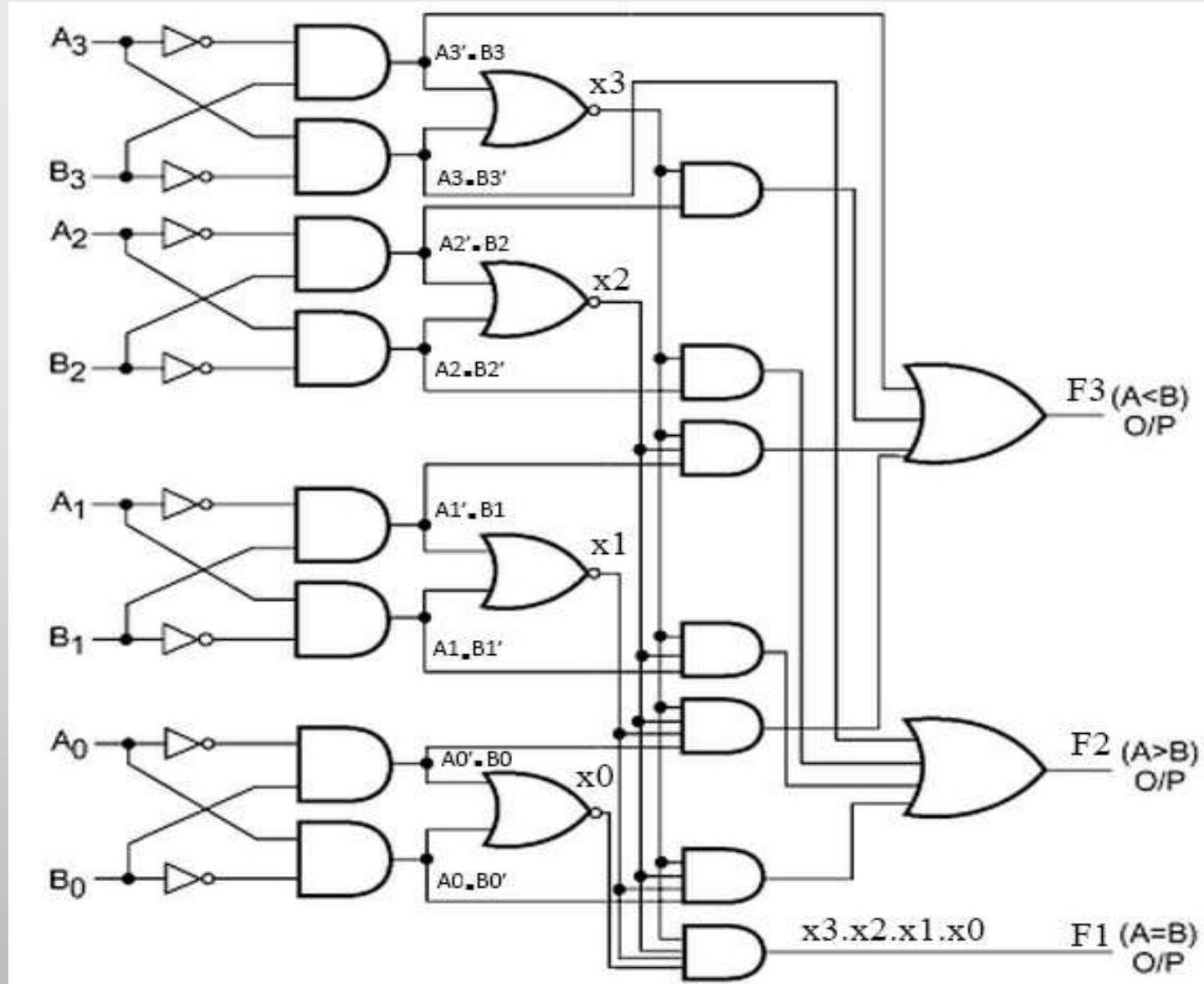
$A3=B3; A2 < B2$ [$A2=0, B2=1$]

$A3=B3; A2=B2; A1 < B1$ [$A1=0, B1=1$]

$A3=B3; A2=B2; A1=B1; A0 < B0$ [$A0=0, B0=1$]

If $F2 = A3'.B3 + x3.A2'.B2 + x3.x2.A1'.B1 + x3.x2.x1.A0'.B0 = 1; A > B$

4-bit Magnitude Comparator



Multiplexer (MUX)

A Multiplexer is a digital circuit that has multiple inputs and a single output.

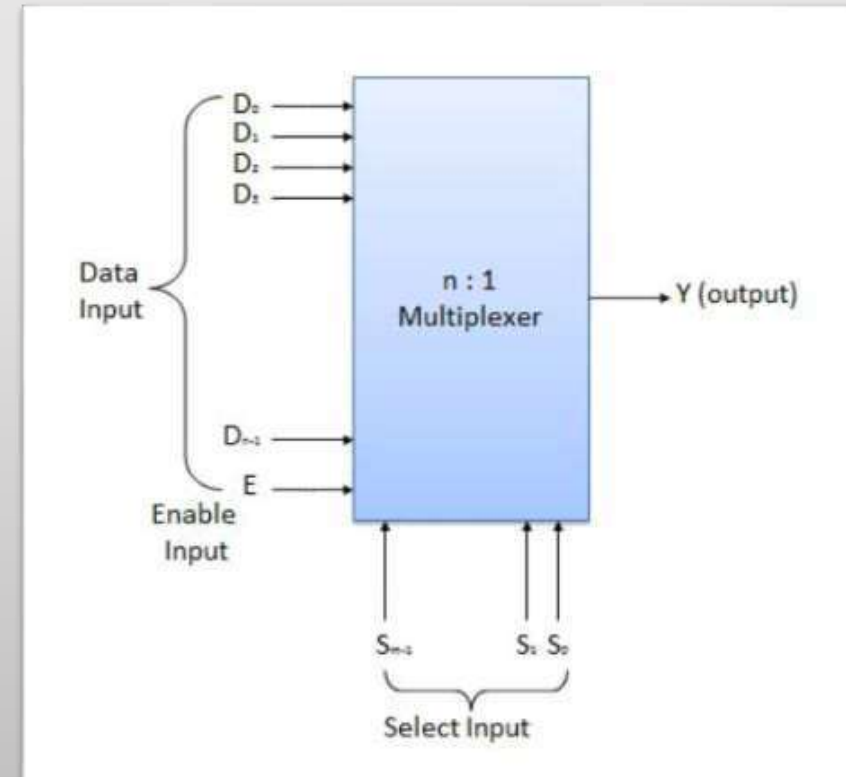
The selection of one of the n inputs is done by the select inputs

It has one output selected at a time.

It is also known as Data Selector.

Size of MUX = $2^n \times 1$

where n is number of select inputs



2:1 MUX

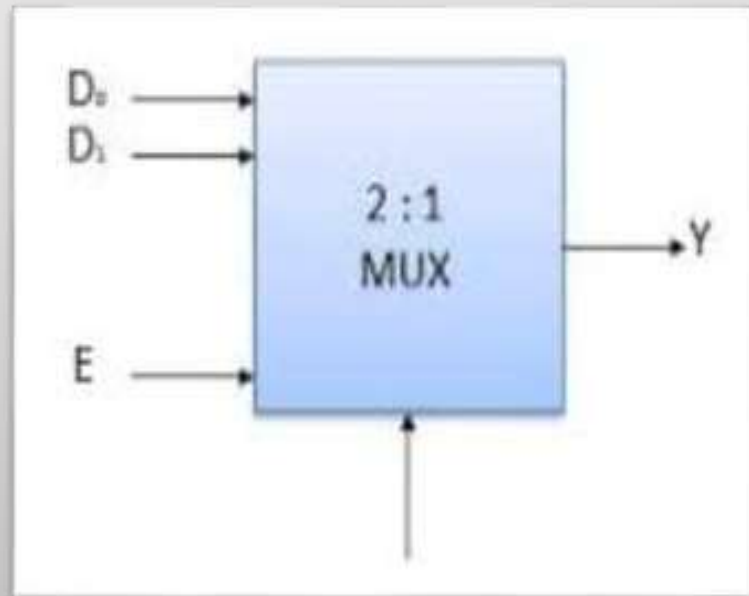
Number of Inputs = 2

Number of Output = 1

Select inputs = 1

Size of MUX = $2^1 \times 1$

BLOCK DIAGRAM



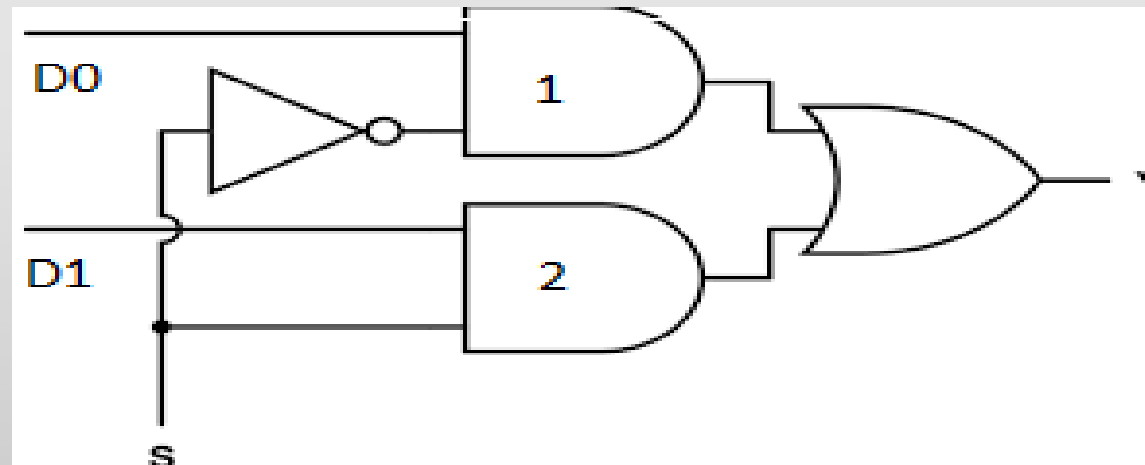
TRUTH TABLE

S	OUTPUT Y
0	D_0
1	D_1

$$Y = S'D_0 + SD_1$$

2:1 MUX

$$Y = S'D_0 + SD_1$$



4:1 MUX

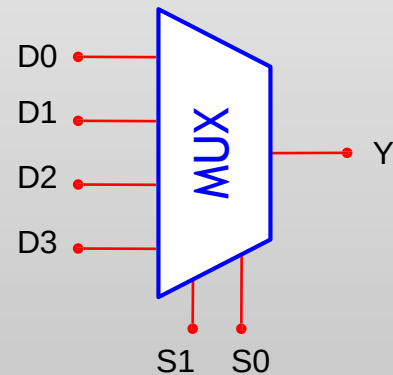
Number of Inputs = 4

Number of Output = 1

Select inputs = 2

Size of MUX = $2^2 \times 1$

BLOCK DIAGRAM



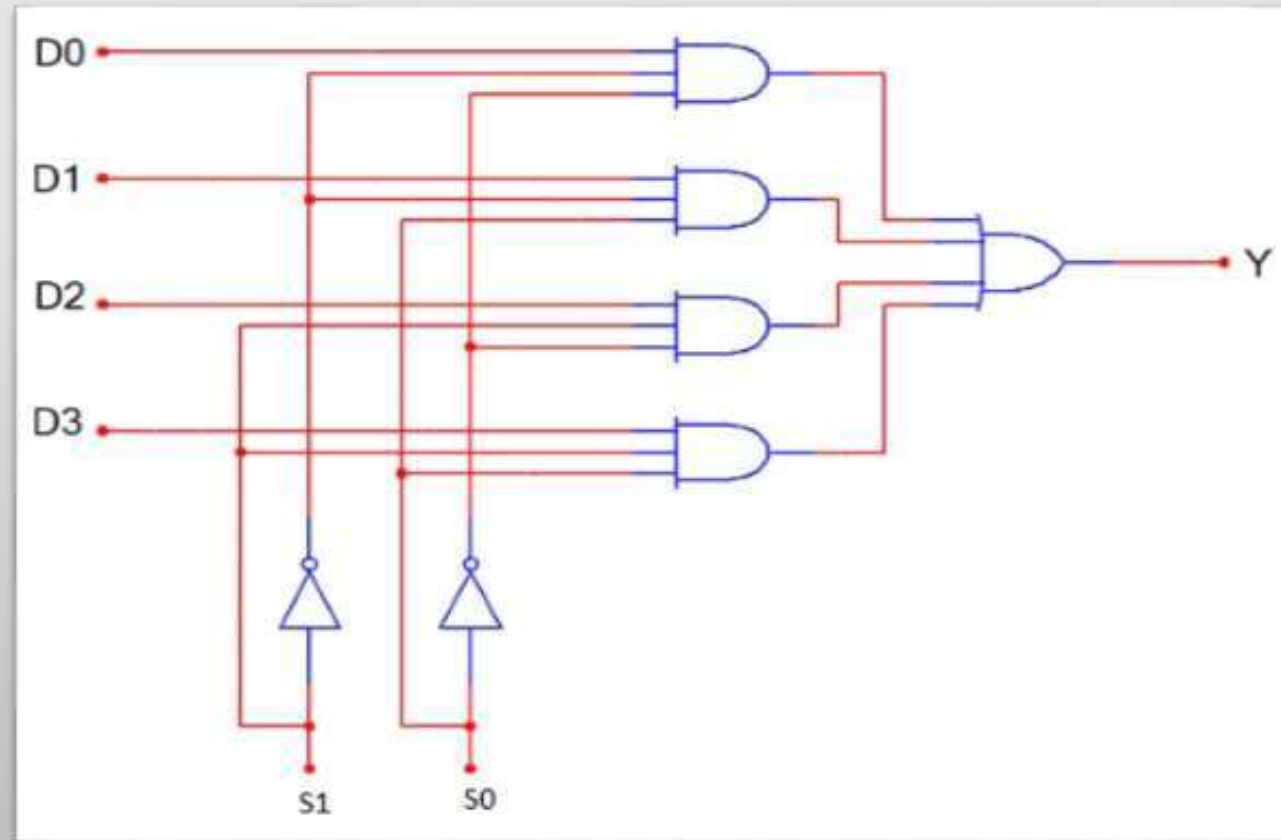
TRUTH TABLE

S1	S0	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

$$Y = S1'S0'D0 + S1'S0D1 + S1S0'D2 + S1S0D3$$

4:1 MUX

$$Y = S_1'S_0'D_0 + S_1'S_0D_1 + S_1S_0'D_2 + S_1S_0D_3$$



8:1 MUX

Number of Inputs = 8

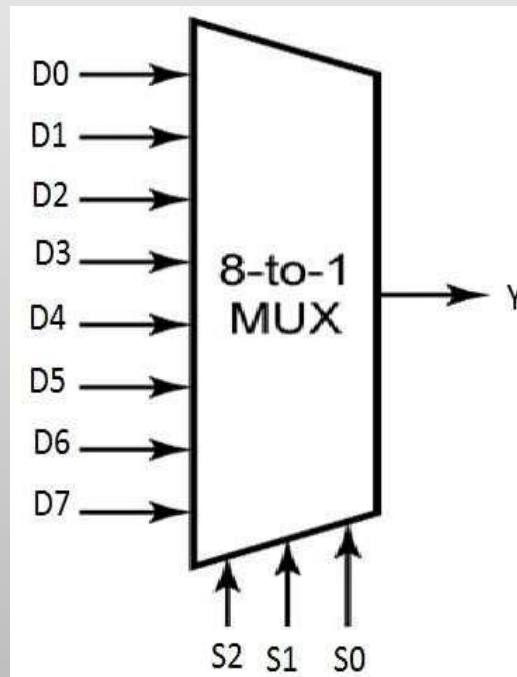
Number of Output = 1

Select inputs = 3

Size of MUX = $2^3 \times 1$

Y = ?

BLOCK DIAGRAM



TRUTH TABLE

S2	S1	S0	Y
0	0	0	D0
0	0	1	D1
0	1	0	D2
0	1	1	D3
1	0	0	D4
1	0	1	D5
1	1	0	D6
1	1	1	D7

Implementation of Boolean Function using MUX

Most important thing during this implementation is the implementation table which is derived from following rules:

List the inputs of the multiplexer and under them list all the minterms in two rows. The first row lists all those minterms where A is complemented, and the second row all the minterms with A uncomplemented, as shown in above example. Circle all the minterms of the function and inspect each column separately.

- If the two minterms in a column are not circled, apply 0 to the corresponding multiplexer input.
- If the two minterms are circled, apply 1 to the corresponding multiplexer input.
- If the bottom minterm is circled and the top is not circled, apply A to the corresponding multiplexer input.
- If the top minterm is circled and the bottom is not circled, apply A' to the corresponding multiplexer input

Implementation of Boolean Function using MUX

Implement the Boolean function $F(A,B,C) = \sum(1,3,5,6)$ using MUX.

Number of inputs(n) = 3 (A,B,C)

Data input = 1; A

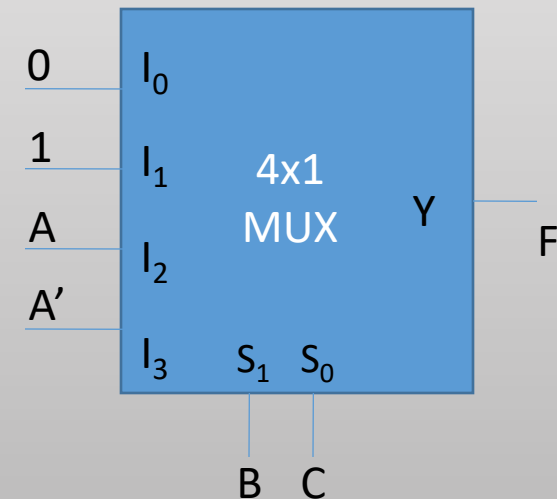
Select inputs = 2 ; B and C

[($n-1$) is the no of select inputs]

Size of MUX = $2^2 \times 1 = 4 \times 1$

Implementation table for input A to be connected,

	I0	I1	I2	I3
A'	0	1	2	3
A	4	5	6	7
inputs	0	1	A	A'



Implementation of Boolean Function using MUX

Implement the Boolean function $F(A,B,C) = \sum(1,3,5,6)$ using MUX.

Number of inputs(n) = 3 (A,B,C)

Data input = 1; B

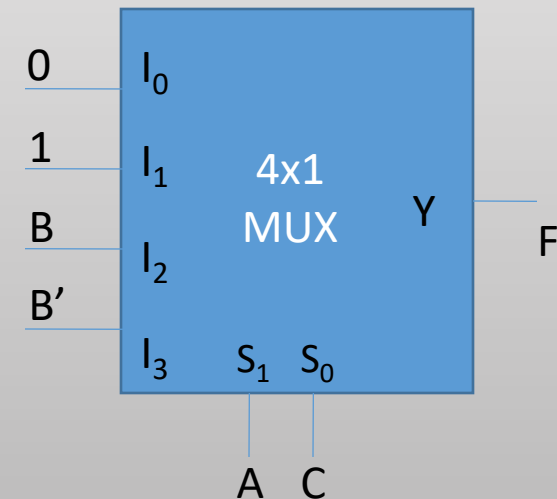
Select inputs = 2 ; A and C

[($n-1$) is the no of select inputs]

Size of MUX = $2^2 \times 1 = 4 \times 1$

Implementation table for input B to be connected,

	I0	I1	I2	I3
B'	0	1	4	5
B	2	3	6	7
inputs	0	1	B	B'



Implementation of Boolean Function using MUX

Implement the Boolean function $F(A,B,C) = \sum(1,3,5,6)$ using MUX.

Number of inputs(n) = 3 (A,B,C)

Data input = 1; C

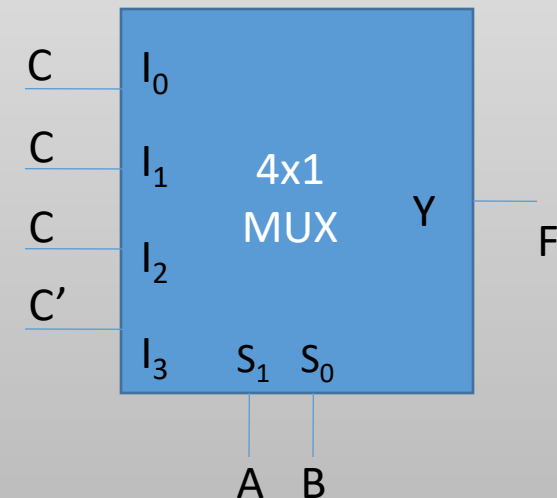
Select inputs = 2 ; A and B

[($n-1$) is the no of select inputs]

Size of MUX = $2^2 \times 1 = 4 \times 1$

Implementation table for input C to be connected,

	I ₀	I ₁	I ₂	I ₃
C'	0	2	4	6
C	1	3	5	7
inputs	C	C	C	C'



Implementation of Boolean Function using MUX

Implement the Full adder circuit using MUX.

From the truth table of Full Adder

$$S(x,y,z) = \sum(1,2,4,7)$$

$$C(x,y,z) = \sum(3,5,6,7)$$

Number of inputs = 3 (x,y,z)

Data input = 1; x

Select inputs = 2 ; y and z

Size of MUX = $2^2 \times 1 = 4 \times 1$

Implementation table for input x to be connected, *For sum*

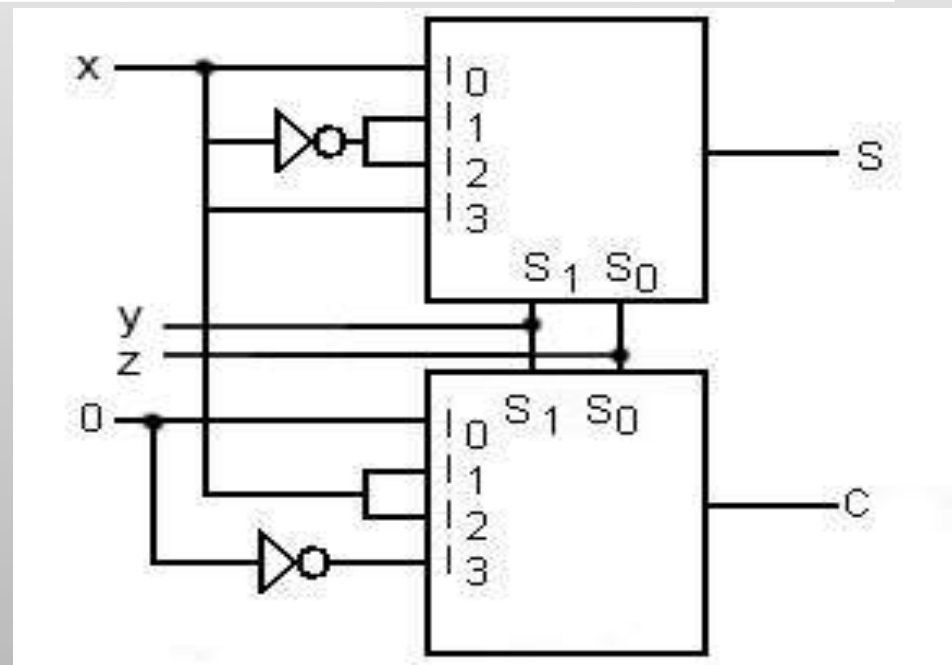
	I0	I1	I2	I3
x'	0	1	2	3
x	4	5	6	7
inputs	x	x'	x'	x

Implementation of Boolean Function using MUX

Implement the Full adder circuit using MUX.

Implementation table for input x to be connected, *For carry*

	I0	I1	I2	I3
x'	0	1	2	3
x	4	5	6	7
inputs	0	x	x	1



Implementation of Boolean Function using MUX

Implement the Boolean function $F(A,B,C,D) = \sum(0,1,3,4,8,9,15)$ using MUX

Number of inputs = 4 (A,B,C,D)

Data input = 1; A

Select inputs = 3 ; B,C and D

Size of MUX = $2^3 \times 1 = 8 \times 1$

Implementation table for input A to be connected,

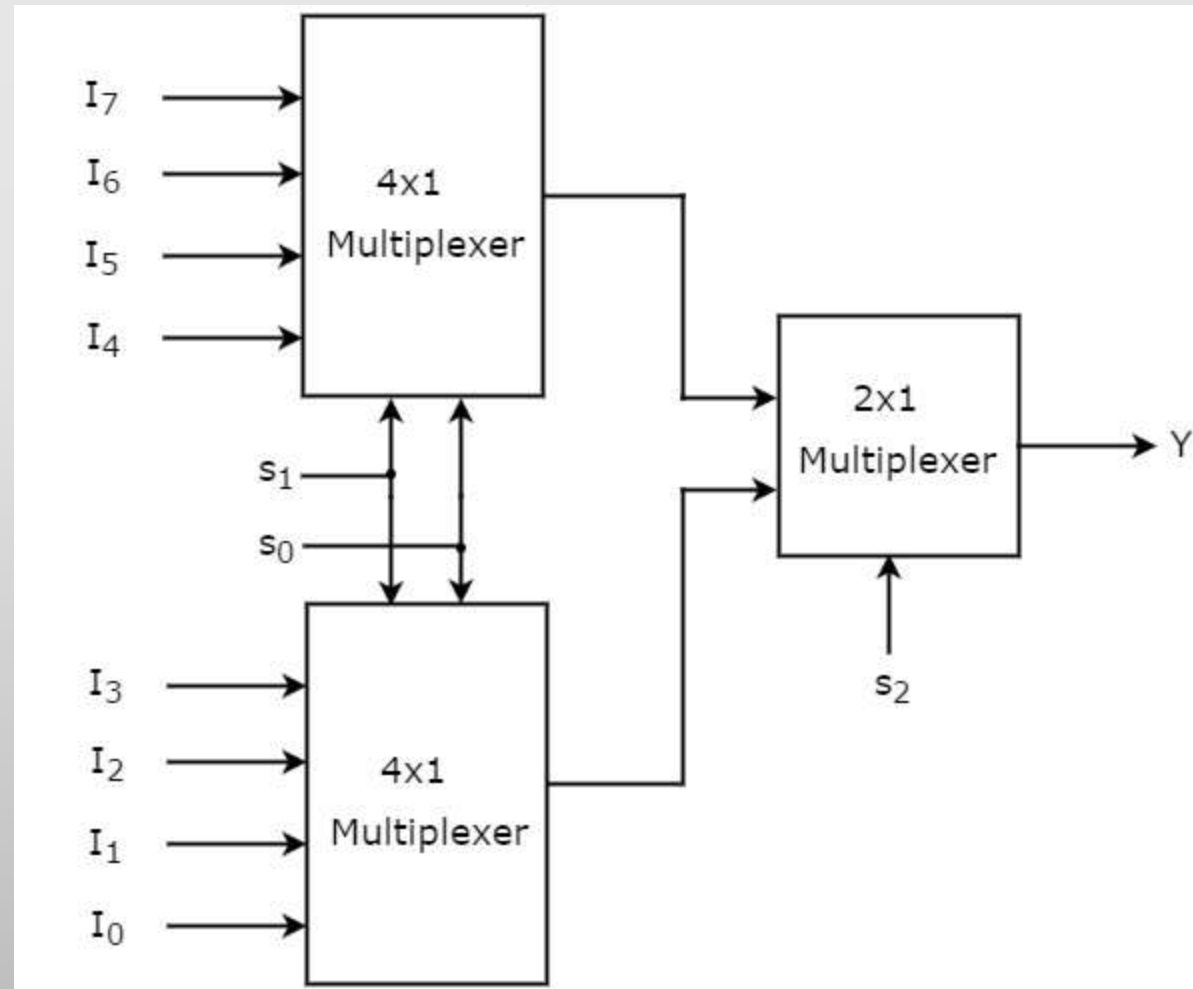
	I0	I1	I2	I3	I4	I5	I6	I7
A'	0	1	2	3	4	5	6	7
A	8	9	10	11	12	13	14	15
i/p	1	1	0	A'	A'	0	0	A

Implementation of Boolean Function using MUX

Implement the Boolean function $F(A,B,C,D) = \sum(0,1,3,4,8,9,15)$ using MUX

Multiplexer (MUX)

*Design **8 x 1 MUX** using 4 x 1 and 2 x 1 MUX.*



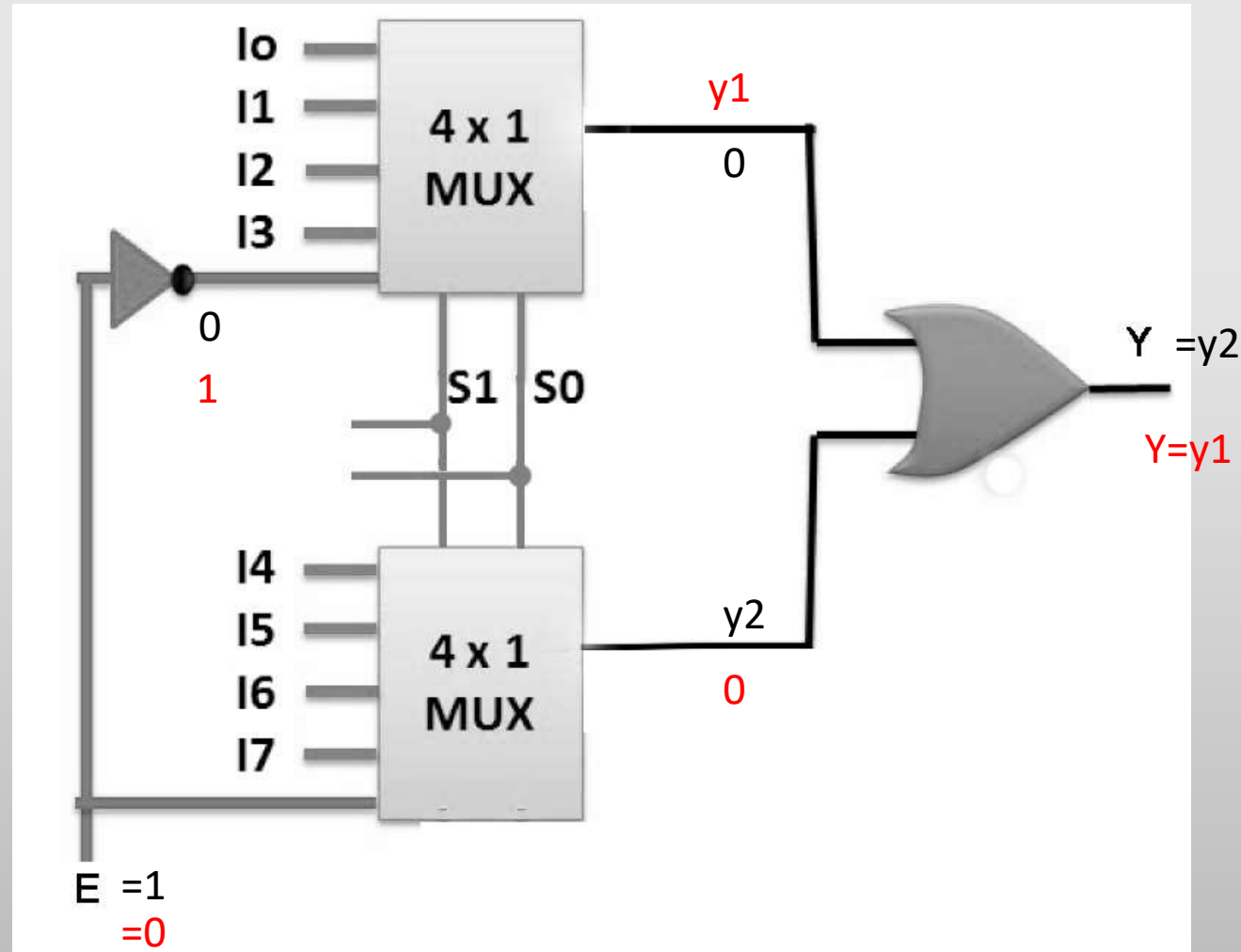
Multiplexer (MUX)

*Design **32 x 1** MUX using 16 x 1 and 2 x 1 MUX.*

*Design **16 x 1** MUX using 8 x 1 MUX.*

Multiplexer (MUX)

Design **8 x 1** MUX using 4 x 1 MUX.

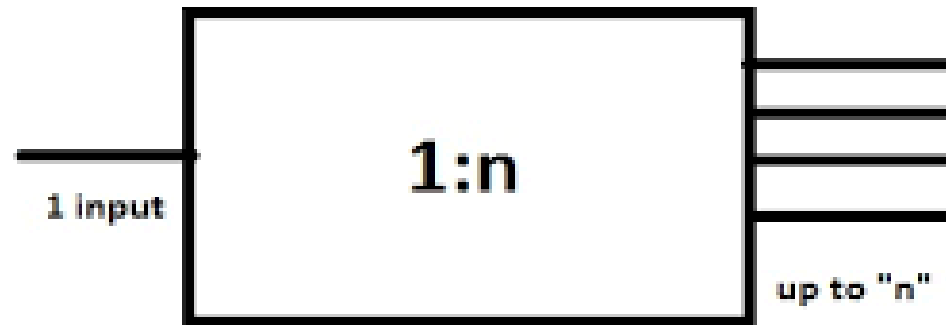


Demultiplexer (DEMUX)

DEMUX is exactly opposite to MUX. DEMUX always has single input and multiple outputs unlike MUX.

It can be called as Data Distributor.

Size of DEMUX = 1×2^n where n is select inputs.



1:2 DEMUX

No of input = 1

No of outputs = 2

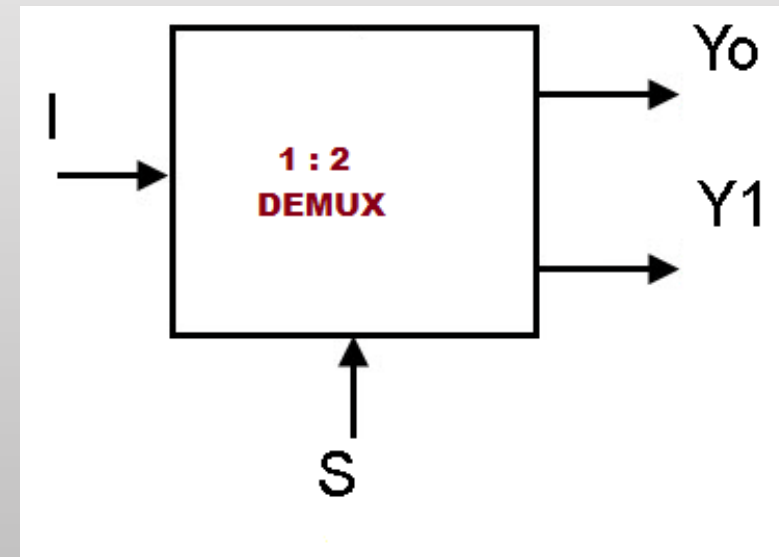
Size of DEMUX = 1×2^1

No of select input = 1

S	Y0	Y1
0	1	0
1	0	1

$$Y0 = S'.I$$

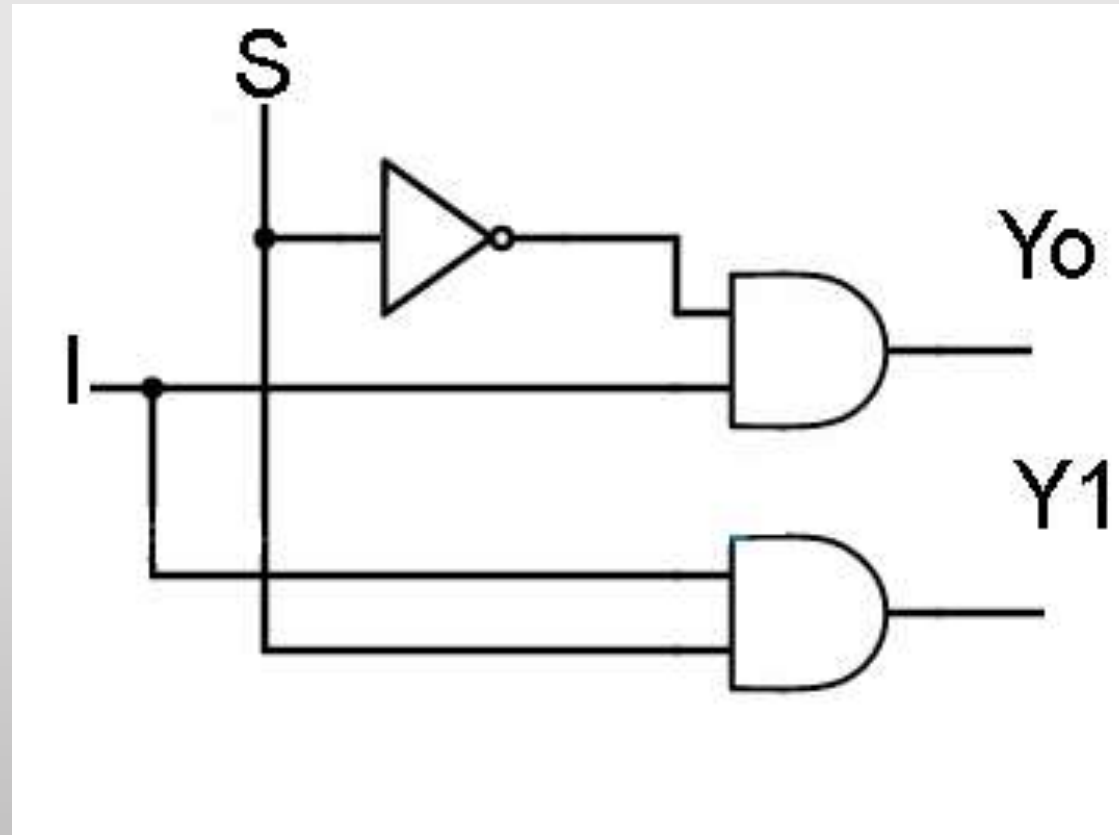
$$Y1 = S.I$$



1:2 DEMUX

$$Y_0 = S'I$$

$$Y_1 = SI$$



1:4 DEMUX

No of input = 1

No of outputs = 4

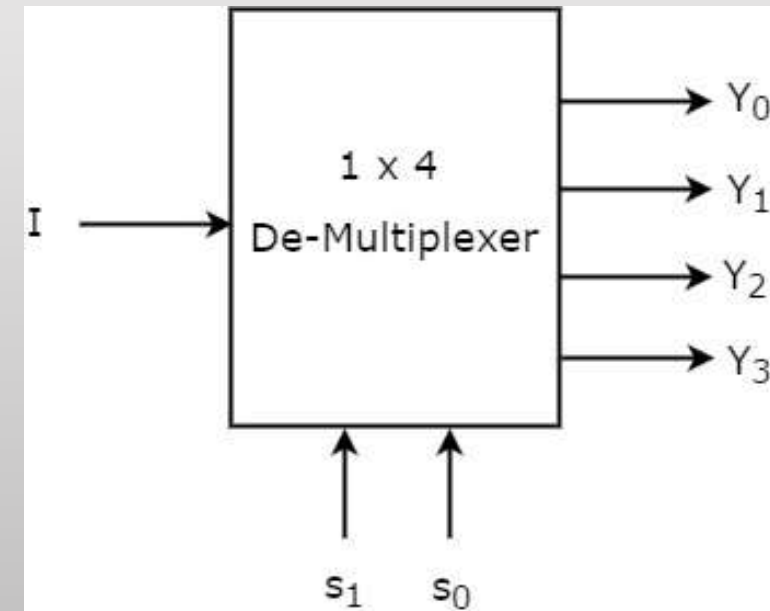
Size of DEMUX = 1×2^2

No of select inputs = 2

S_1	S_0	Y_0	Y_1	Y_2	Y_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

$$Y_0 = s'_1 \cdot s'_0 \cdot I \quad Y_2 = s_1 \cdot s_0' \cdot I$$

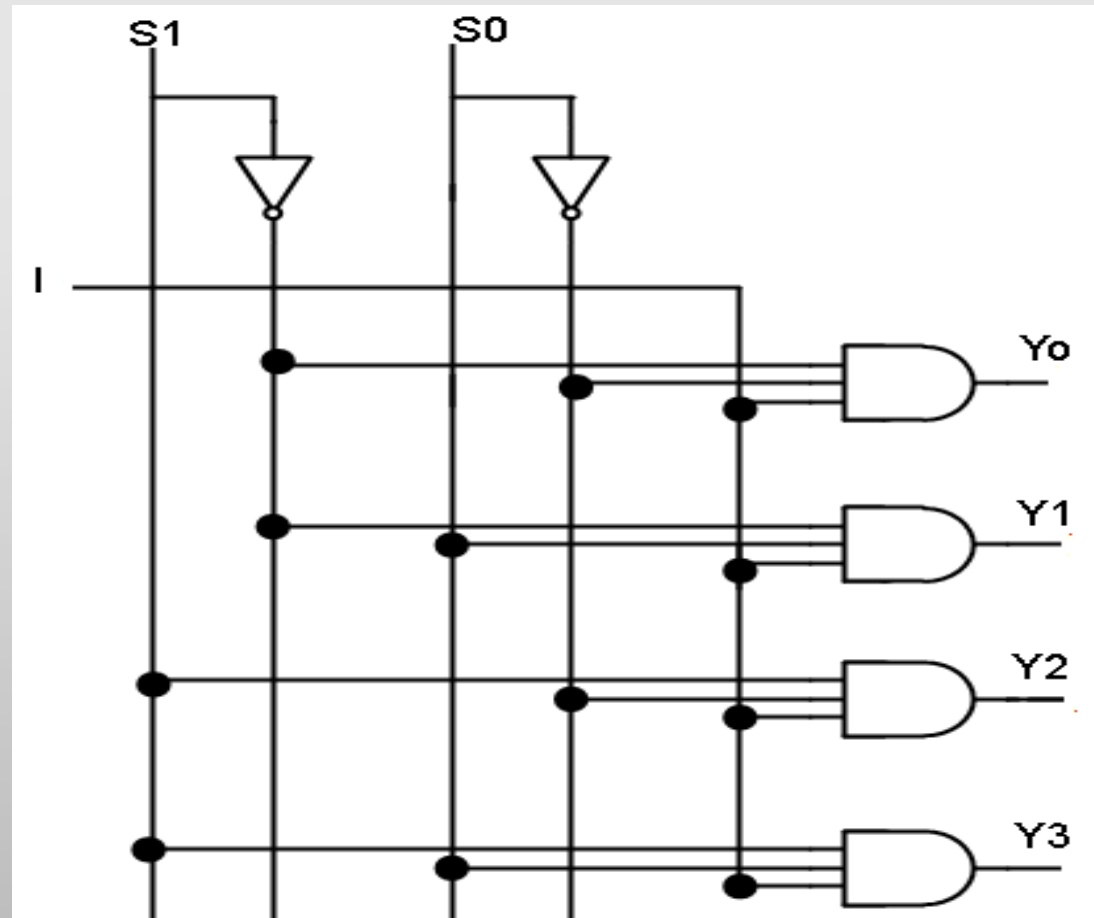
$$Y_1 = s_1' \cdot s_0 \cdot I \quad Y_3 = s_1 \cdot s_0 \cdot I$$



1:4 DEMUX

$$Y_0 = s'_1 \cdot s'_0 \cdot I \quad Y_2 = s_1 \cdot s_0' \cdot I$$

$$Y_1 = s_1' \cdot s_0 \cdot I \quad Y_3 = s_1 \cdot s_0 \cdot I$$



1:8 DEMUX

No of input = 1 (I)

No of outputs = 8

Size of DEMUX = 1×2^3

No of select inputs = 3

s2	s1	s0	y0	y1	y2	y3	y4	y5	y6	y7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Decoder

A decoder is a combinational circuit.

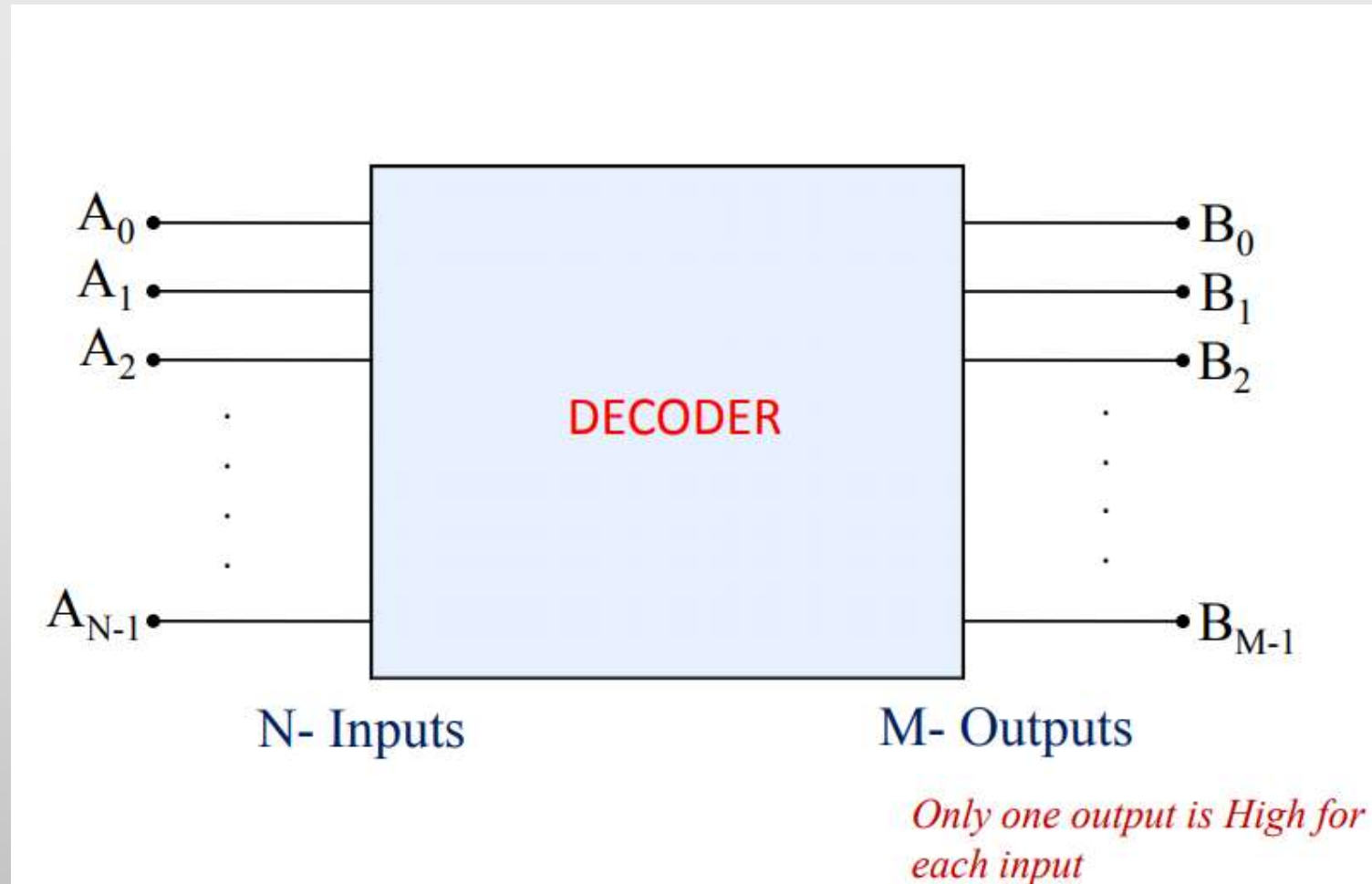
A decoder accepts a set of inputs that represents a binary number and activates only that output corresponding to the input number. All other outputs remain inactive. •
Fig. 1 shows the block diagram of decoder with 'N' inputs and 'M' outputs.

There are 'n' inputs, for each of these input combination only one output will be HIGH (active) all other outputs are LOW.

The numbers of output is ' 2^n '.

So, size of Decoder = $n \times 2^n$

Decoder

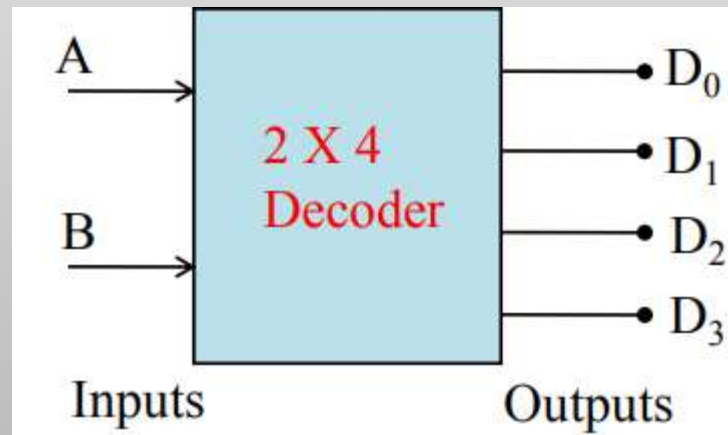


2x4 Decoder

No. of inputs = 2 (A,B)

No. of Outputs : $2^2 = 4$, (D0 , D1 , D2 and D3)

A	B	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



$$D0 = A'B'$$

$$D1 = A'B$$

$$D2 = AB'$$

$$D3 = AB$$

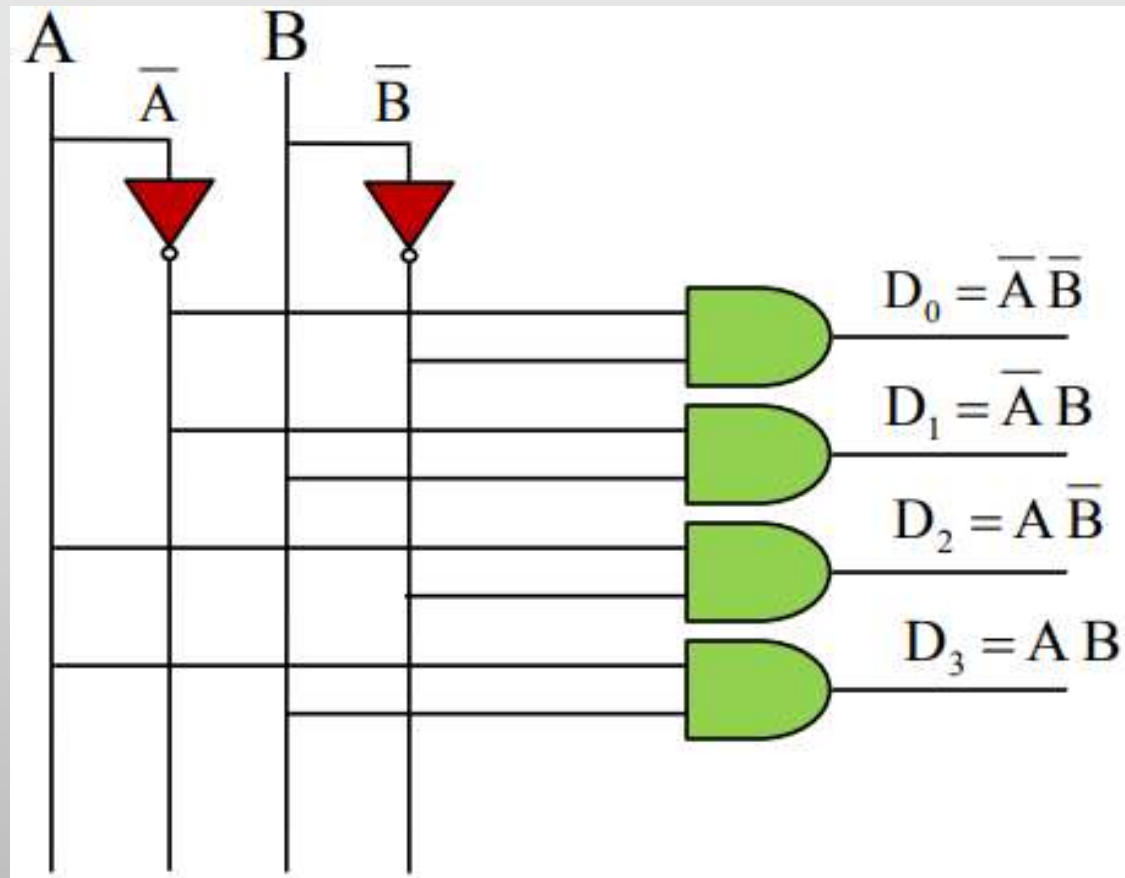
2x4 Decoder

$$D_0 = A'B'$$

$$D_1 = A'B$$

$$D_2 = AB'$$

$$D_3 = AB$$



3x8 Decoder

No. of inputs = 3 (A,B,C)

No. of Outputs : $2^3 = 8$, (D0, D1, D2, D3, D4, D5, D6, D7)

A	B	C	D0	D1	D2	D3	D4	D5	D6	D7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

$$D0 = A'B'C'$$

$$D1 = A'B'C$$

$$D2 = A'BC'$$

$$D3 = AB'C$$

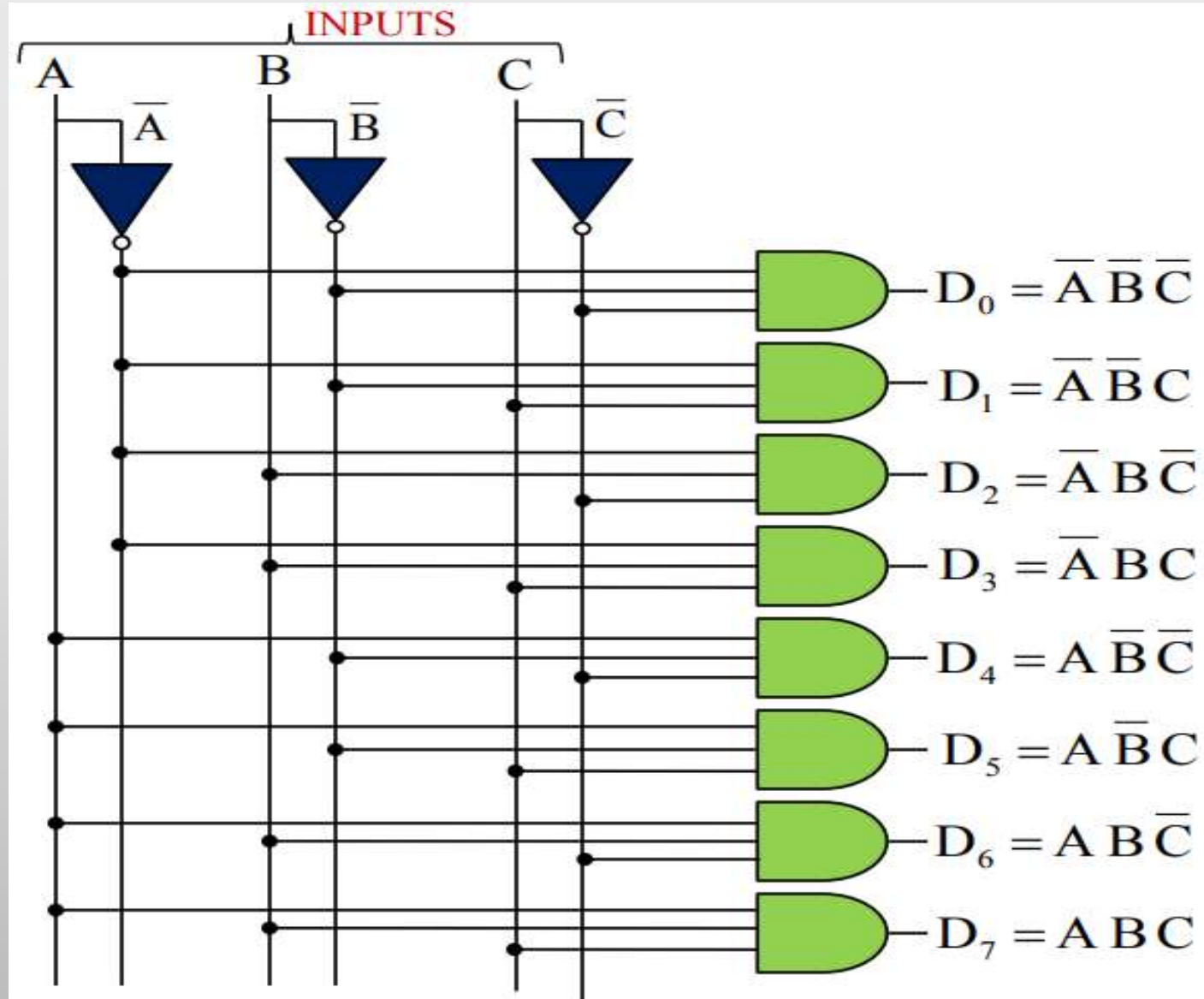
$$D4 = AB'C'$$

$$D5 = AB'CD$$

$$D6 = ABC'$$

$$D7 = ABC$$

3x8 Decoder



1x2 Decoder

A	D0	D1
0	1	0
1	0	1

$$D0 = A'$$

$$D1 = A$$

Implementation of Boolean Function using Decoder

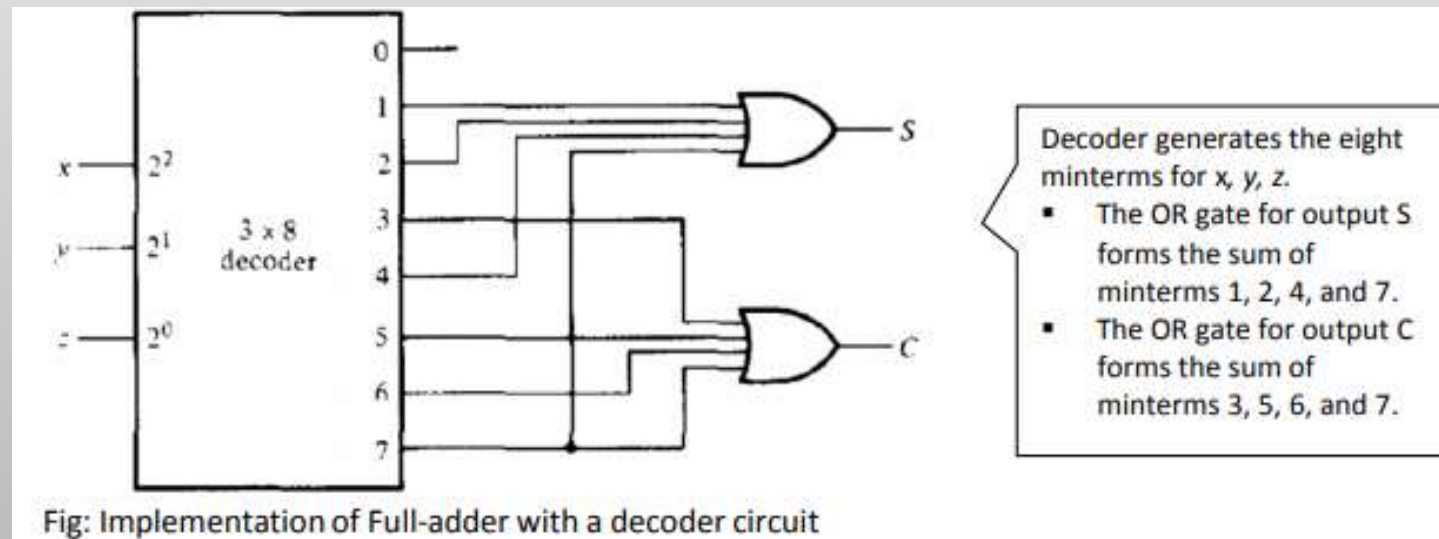
We can implement any Boolean function using decoder and OR gates.

Implement the Full adder circuit using a Decoder.

$$S(a,b,c) = \sum(1,2,4,7)$$

$$C(a,b,c) = \sum(3,5,6,7)$$

In decoder; no. of inputs = 3 and no. of outputs = $2^3 = 8$
size = 3×8



Implementation of Boolean Function using Decoder

1. Realize $F(A,B,C,D) = \sum(0,1,3,5,7,8,11,13,15)$ using decoder.

In decoder;

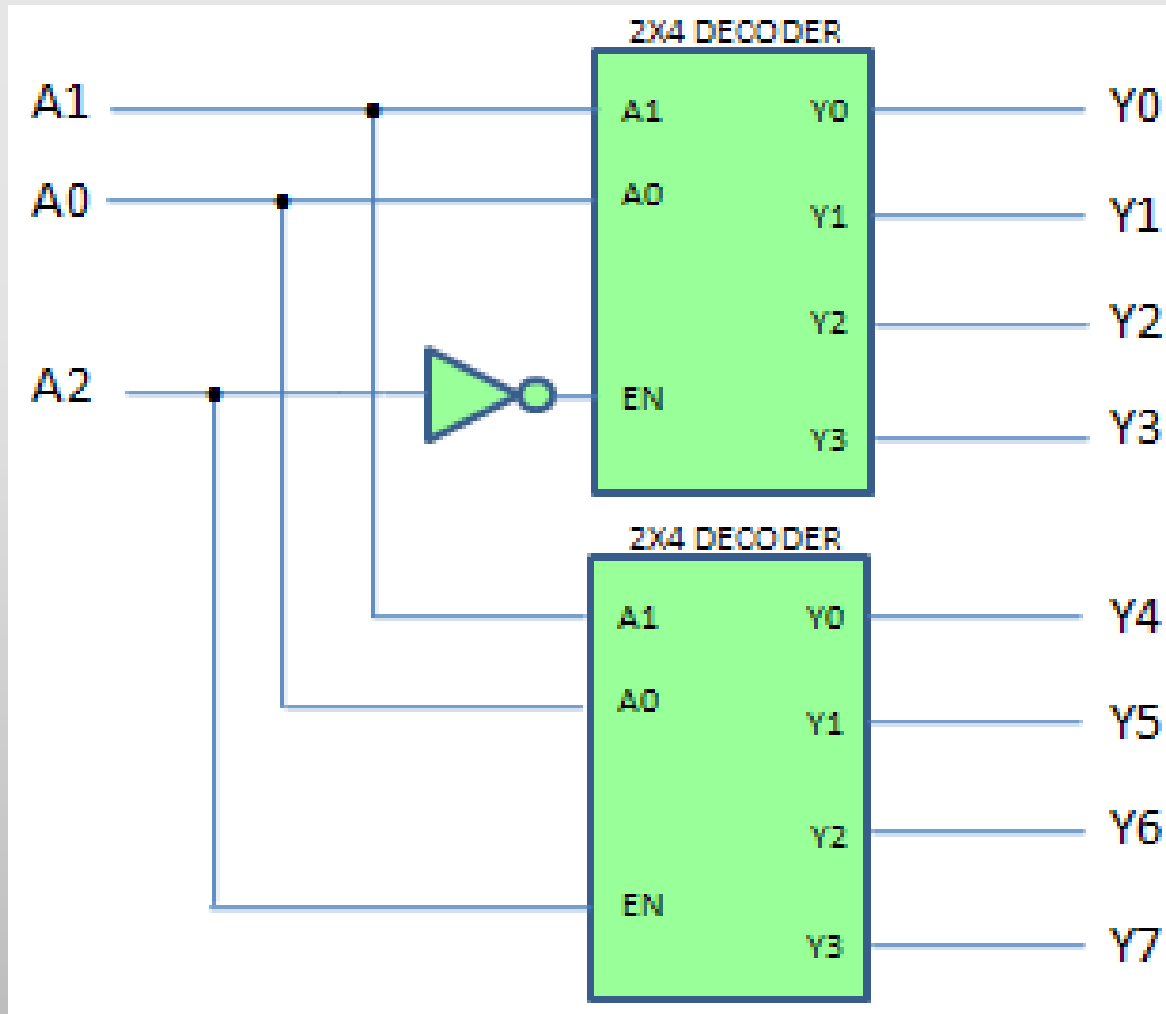
No of inputs = 4 and outputs = 16

Size = 4 x 16

2. Realize $F(A,B,C,D) = \sum(0,1,3,5,7,8,11,13,15)$ using decoder and NOR gate.

Decoder

Design 3x8 Decoder using two 2x4 Decoder.

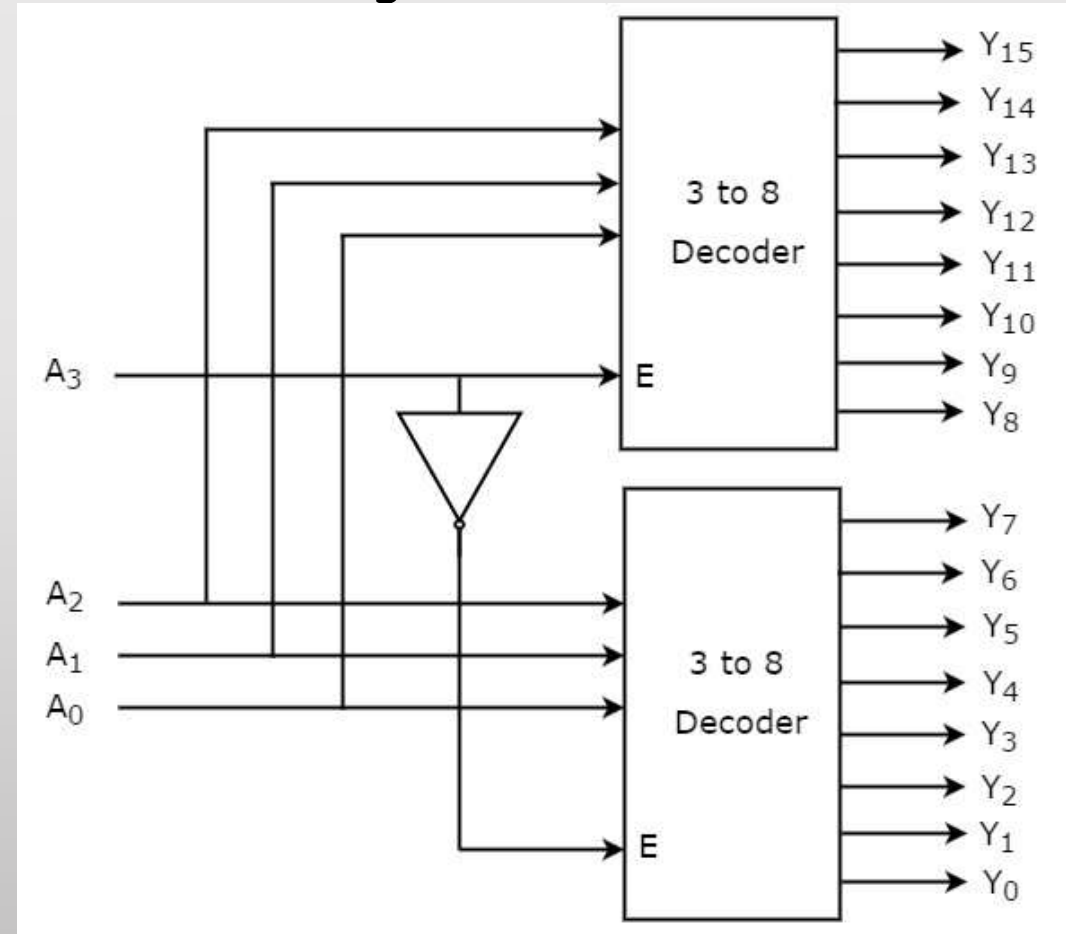


When $A2 = 0$,
 $En = 1$ in top decoder,
 it is in operation,
 $En = 0$ in Bottom decoder
 and it is not
 in operation.

Similarly, When $A2 = 1$,
 $En = 1$ in bottom decoder,
 it is in operation,
 $En = 0$ in top decoder
 and it is not
 in operation.

Decoder

Design 4x16 Decoder using two 3x8 Decoder.

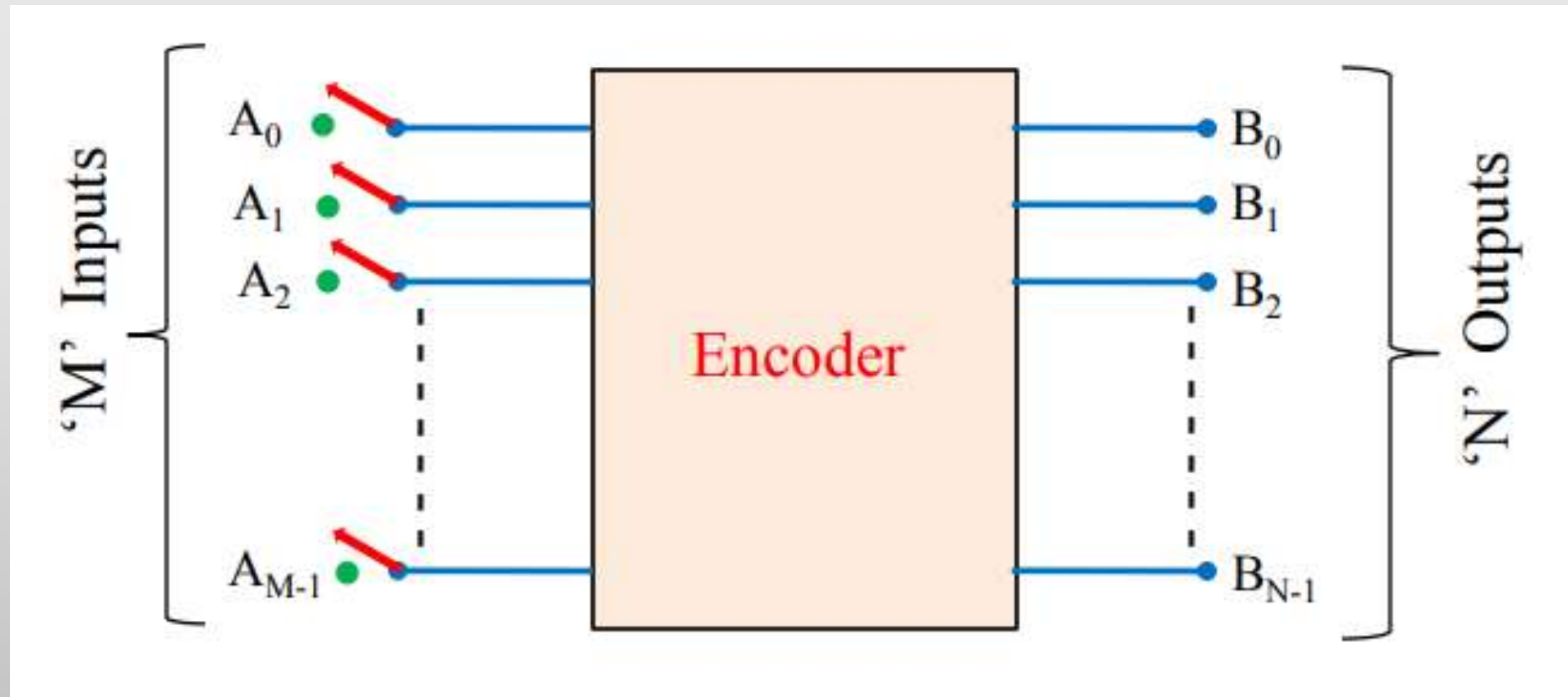


Encoder

An Encoder is a combinational logic circuit.

- It performs the inverse operation of Decoder.
- The opposite process of decoding is known as Encoding.
- An Encoder has 'M' input lines, only one of which is activated at a given time, and produces an N-bit output code, depending on which input is activated.
- Size of Encoder = $2^n \times n$

Encoder



8x3 Encoder ($2^n \times n$)

No. of inputs = $2^3 = 8$; (D0, D1, D2, D3, D4, D5, D6, D7)

No. of Outputs = 3 ; (x,y,z)

D0	D1	D2	D3	D4	D5	D6	D7	x	y	z
1	0	0	0	0	0	0	0	0	0	0
0	1	0	0	0	0	0	0	0	0	1
0	0	1	0	0	0	0	0	0	1	0
0	0	0	1	0	0	0	0	0	1	1
0	0	0	0	1	0	0	0	1	0	0
0	0	0	0	0	1	0	0	1	0	1
0	0	0	0	0	0	1	0	1	1	0
0	0	0	0	0	0	0	1	1	1	1

$$x = D4 + D5 + D6 + D7$$

$$y = D2 + D3 + D6 + D7$$

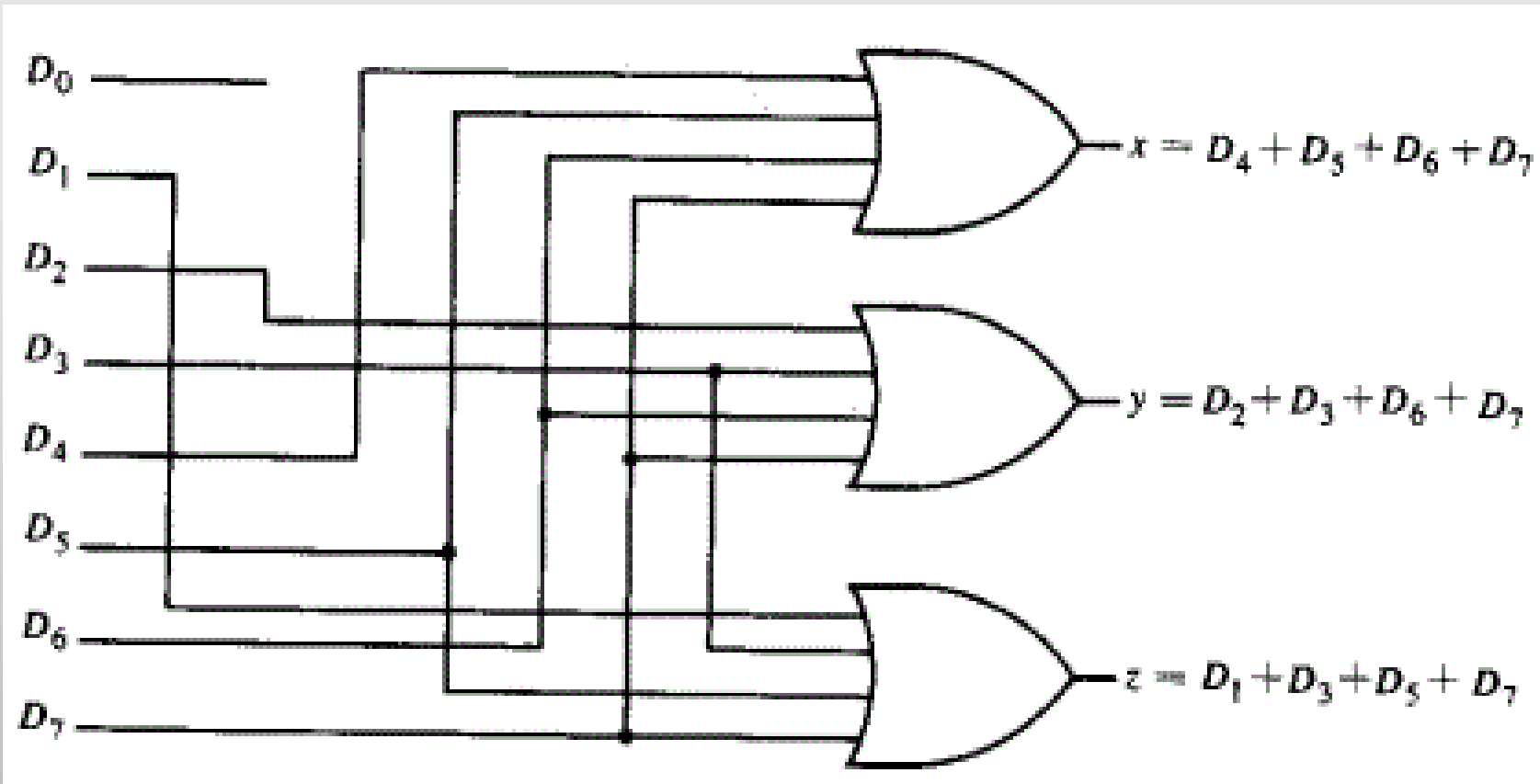
$$z = D1 + D3 + D5 + D7$$

8x3 Encoder

$$x = D_4 + D_5 + D_6 + D_7$$

$$y = D_2 + D_3 + D_6 + D_7$$

$$z = D_1 + D_3 + D_5 + D_7$$



4x2 Encoder

No. of inputs = 4; (D0, D1, D2, D3)

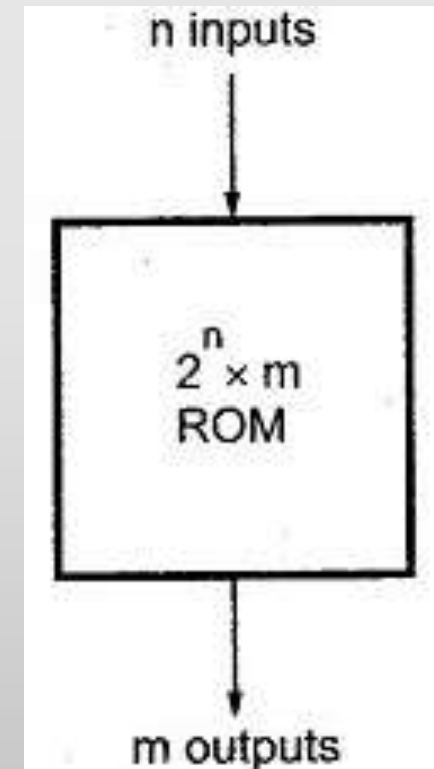
No. of Outputs = 2; (x,y)

D0	D1	D2	D3	x	y
1	0	0	0	0	0
0	1	0	0	0	1
0	0	1	0	1	0
0	0	0	1	1	1

$$X = D2 + D3$$

$$Y = D1 + D3$$

- ROM is a combinational circuit.
- It is used as storage medium in electronic devices.
- It includes both the decoder and OR gates within a single IC package.
- Size of the ROM
 - $2^n \times m \rightarrow 2^n$ words of m bits can be stored.
 - Where; ' 2^n ' is number of words.
 - ' m ' is number of bit per word.
 - If $2^n \times m$ is the size of ROM, n is the input of Decoder used and m is the number of OR gates.
 - Each combination of i/p and o/p is called address and word respectively.



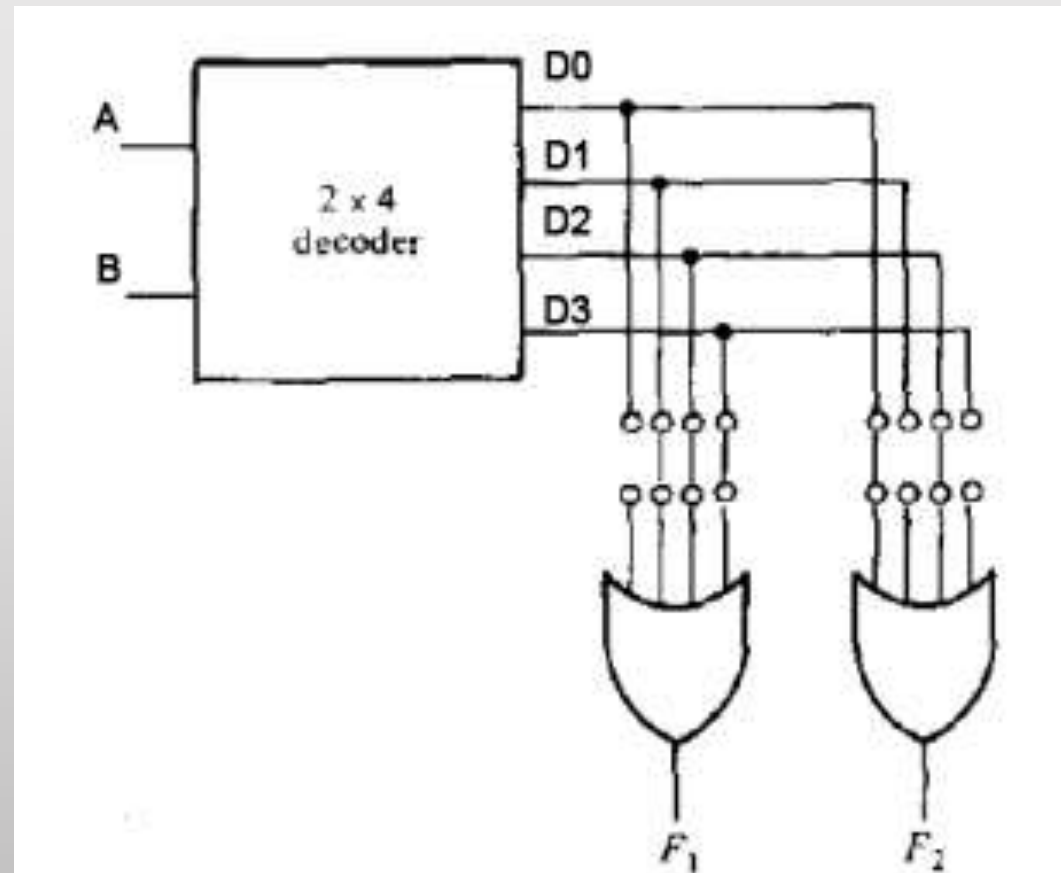
Implementation of Boolean Function using ROM

Implement $F1(A,B) = \sum(1,2,3)$ and $F2(A,B) = \sum(0,2)$ using ROM.

Size of the ROM = $2^2 \times 2$

Size of the decoder = 2×4

Number of OR gates = 2



Implementation of Boolean Function using ROM

Implement Full adder circuit using ROM.

$$S(a,b,c) = \sum(1,2,4,7)$$

$$C(a,b,c) = \sum(3,5,6,7)$$

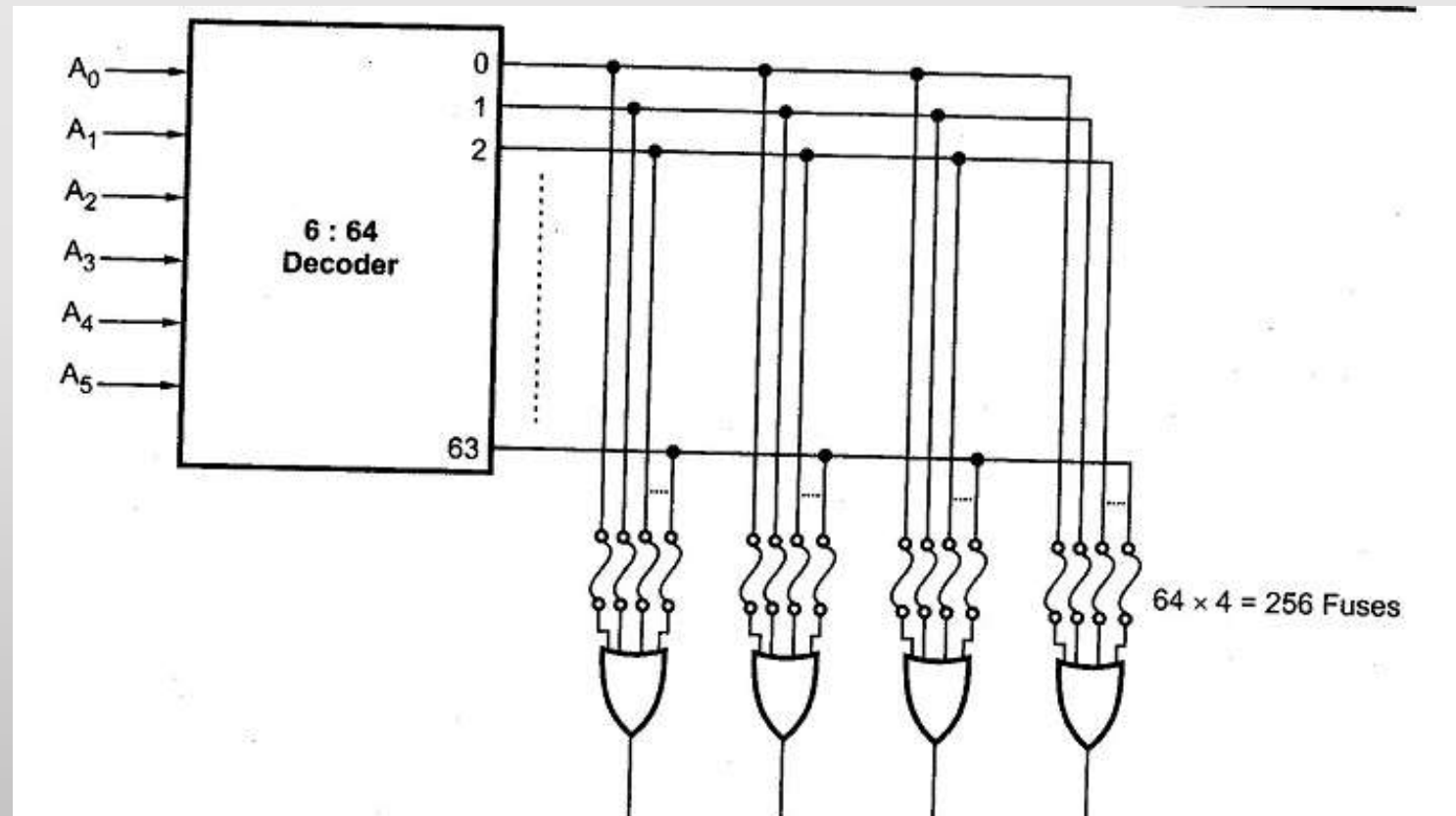
Size of the ROM = $2^3 \times 2$

Size of the decoder = 3×8

Number of OR gates = 2

Implementation of Boolean Function using ROM

If we have to design a ROM having a decoder of size 6 x 64 :



Types of ROM

PROM

EPROM

EEPROM

Flash Memory

Programmable Logic Array(PLA)

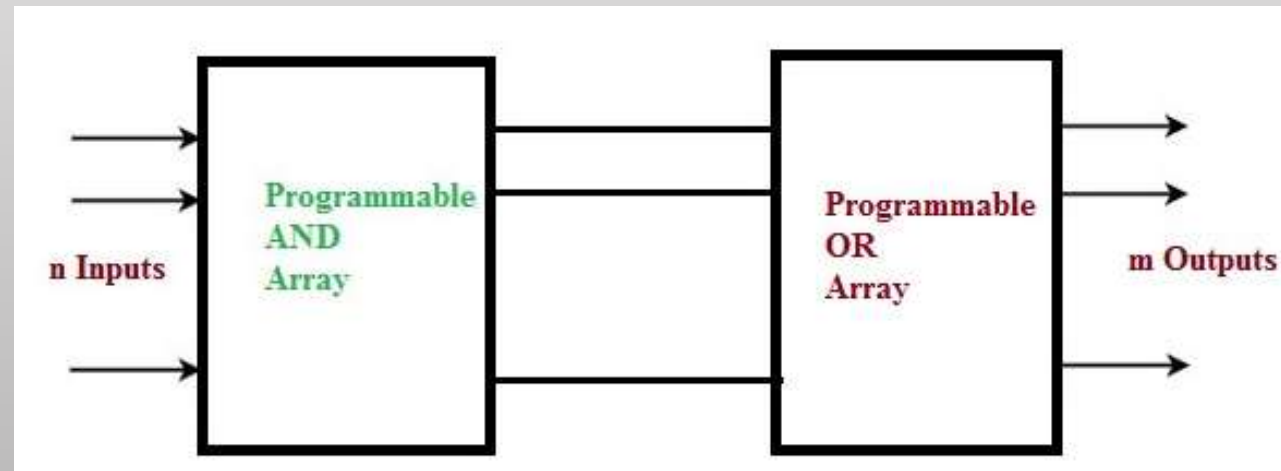
PLA is similar to ROM in concept. It is a Programmable logic device with programmable AND gates followed by programmable OR gates.

Here, Decoder is replaced by a group of AND gates which can be programmed.

A $m \times n$ PLA has 'm' buffer Inverter.

Number of AND gates = number of minterms (excluding repeated minterms)

Number of OR gates = n



Implementation of Boolean Function using PLA

Implement $F1(A,B,C) = \sum(3,5,6,7)$ and $F2(A,B,C) = \sum(0,2,4,7)$ using PLA.

No of inputs = 3

No of outputs = 2

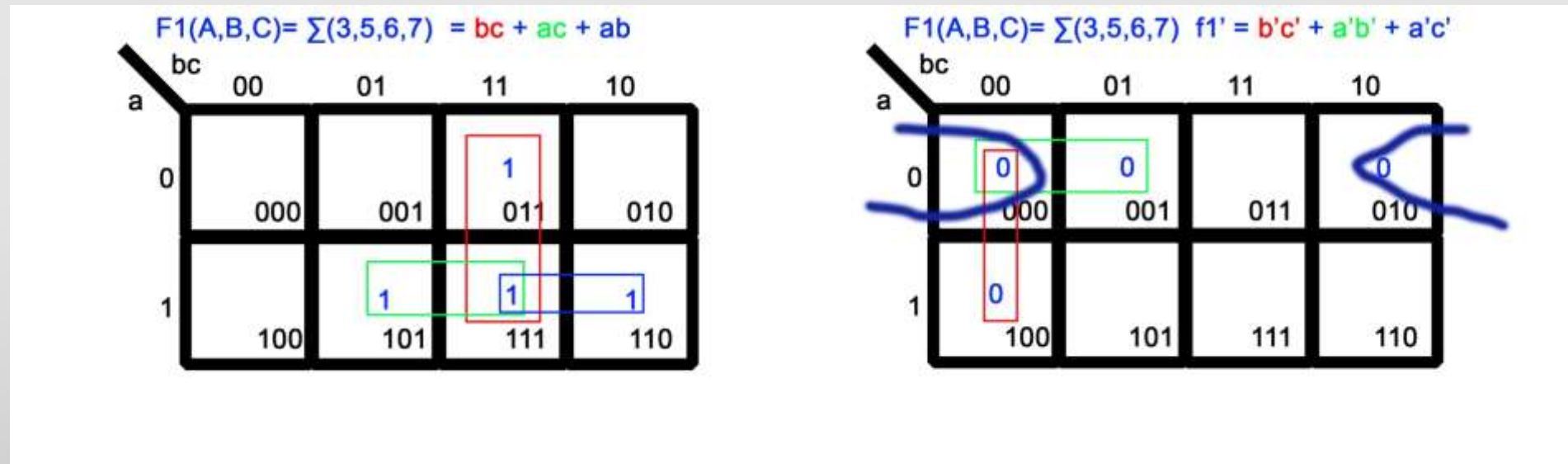
A 3 x 2 PLA has '3' buffer Inverter.

Number of AND gates = number of minterms (excluding repeated minterms)

Number of OR gates = 2

Implementation of Boolean Function using PLA

Implement $F1(A,B,C) = \sum(3,5,6,7)$ and $F2(A,B,C) = \sum(0,2,4,7)$ using PLA.



Similarly,

$$F2 = A'C' + B'C' + ABC$$

$$F2' = A'C + B'C + ABC'$$

Implementation of Boolean Function using PLA

Implement $F1(A,B,C) = \sum(3,5,6,7)$ and $F2(A,B,C) = \sum(0,2,4,7)$ using PLA.

$$F1 = AB + AC + BC$$

$$F1' = A'B' + B'C' + A'C'$$

$$F2 = A'C' + B'C' + ABC$$

$$F2' = A'C + B'C + ABC'$$

$$F1 F2 = 6 \text{ terms}$$

$$F1' F2' = 6 \text{ terms}$$

$$F1' F2 = 4 \text{ terms} \quad \text{So, } F1' F2 \text{ is selected.}$$

$$F1 F2' = 6 \text{ terms}$$

Implementation of Boolean Function using PLA

Implement $F1(A,B,C) = \sum(3,5,6,7)$ and $F2(A,B,C) = \sum(0,2,4,7)$ using PLA.

$F_1' F_2$ is selected. $F1' = A'B' + B'C' + A'C'$

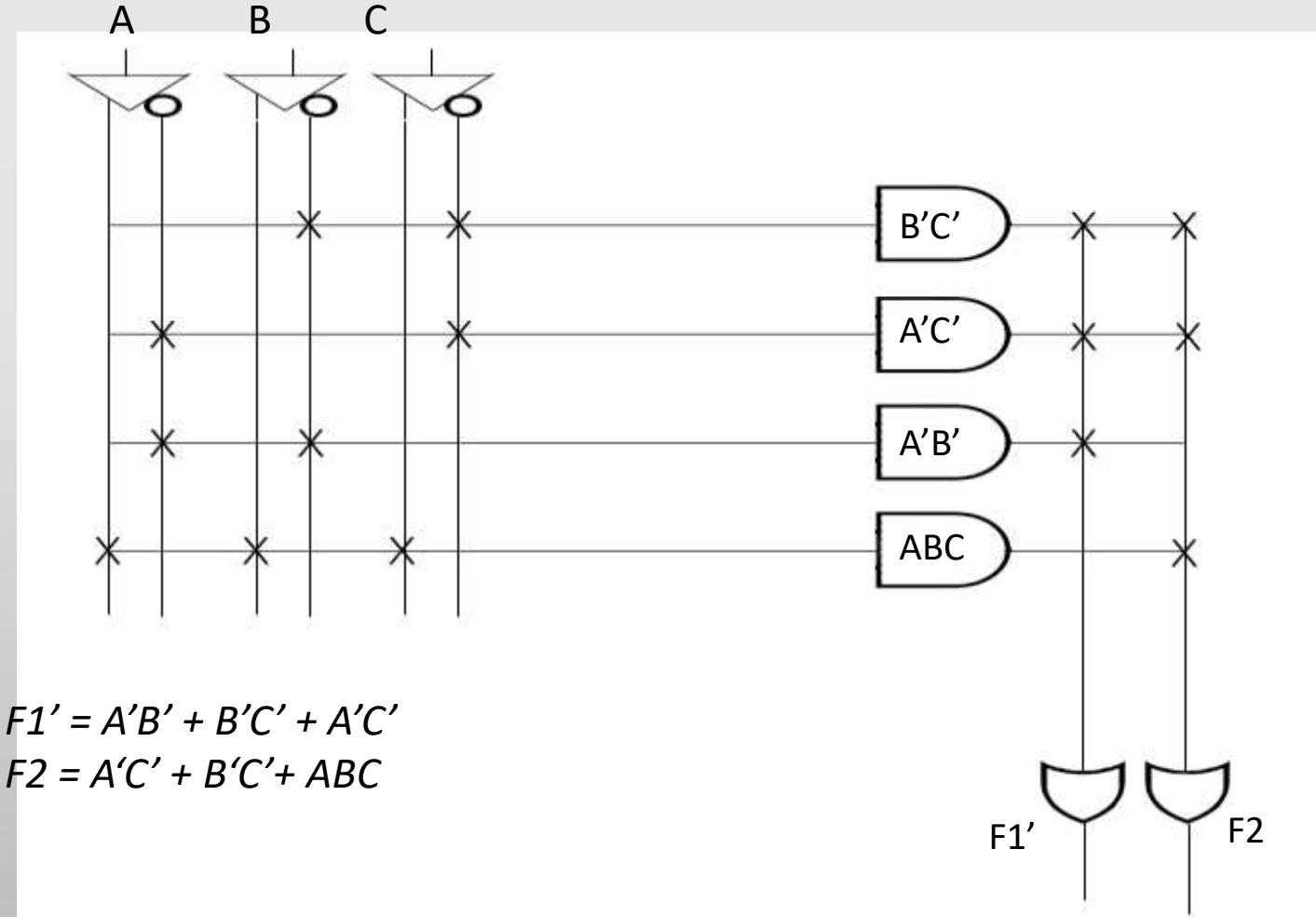
$$F2 = A'C' + B'C' + ABC$$

PLA Program table for the combination $F_1' F_2$ is:

Product Terms	Inputs			Outputs	
	A	B	C	F1(C)	F2(T)
$A'B'$	0	0	-	1	-
$B'C'$	-	0	0	1	1
$A'C'$	0	-	0	1	1
ABC	1	1	1	-	1

Implementation of Boolean Function using PLA

Implement $F1(A,B,C) = \sum(3,5,6,7)$ and $F2(A,B,C) = \sum(0,2,4,7)$ using PLA.



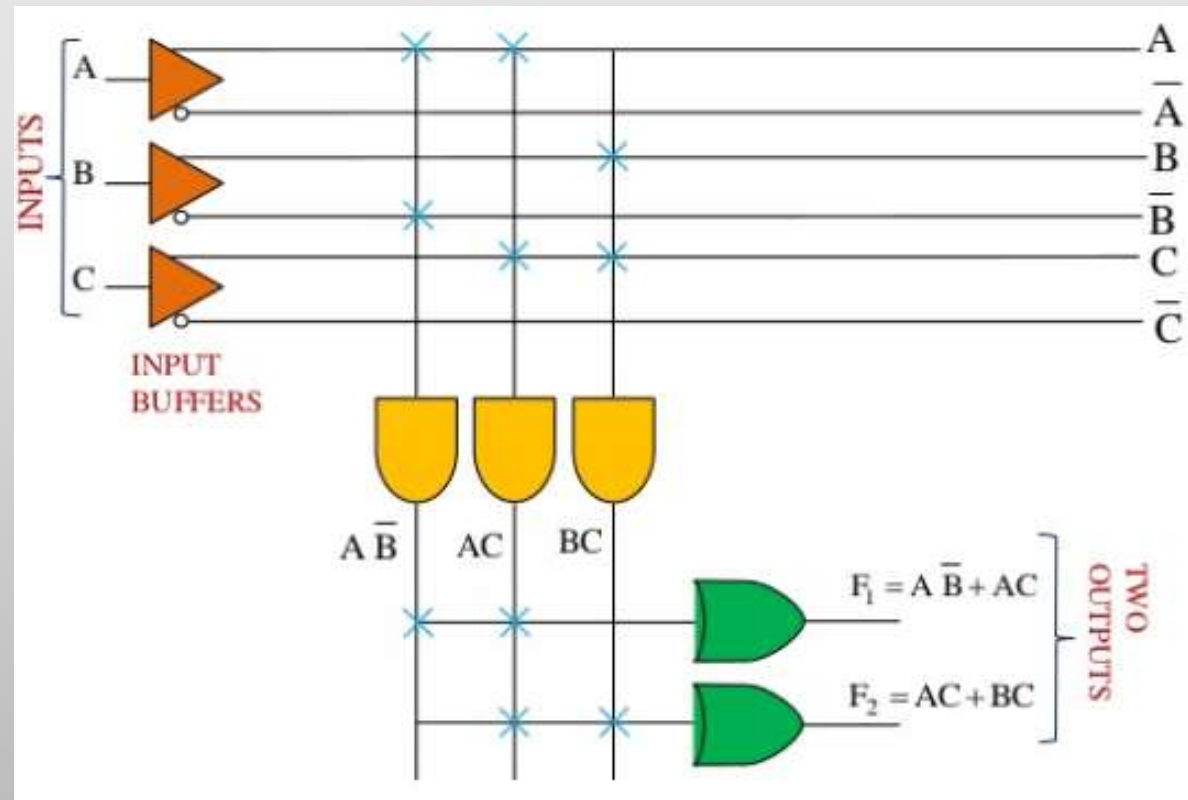
Implementation of Boolean Function using PLA

Implement $F1(A,B,C) = \sum(4,5,7)$ and $F2(A,B,C) = \sum(3,5,7)$ using PLA.

Since, $F1$ $F2$ is selected;

$$F1 = AB' + AC$$

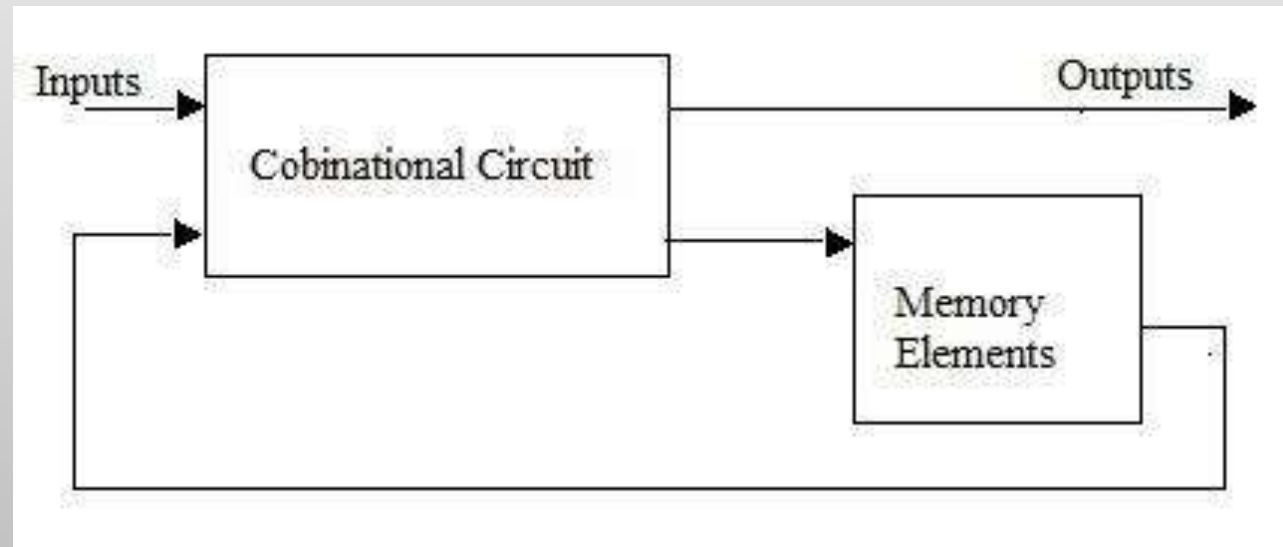
$$F2 = BC + AC$$



Sequential Logic

Sequential Circuits

- The outputs of the circuit will depend upon the present inputs as well as on the sequence of the previous outputs of the circuits.
- It is basically a combination of a combinational circuit and a memory unit.



Sequential Circuits

Combinational Circuit	Sequential Circuit
Logic gates are the basic elements used.	Flip flops are the basic elements used.
It consists of input, output variables and logic gates inter connection.	It consists of input, output variables ,logic gates inter connection and memory unit.
No memory unit present.	Present
Outputs can be calculated from the function with input.	Output depends upon present inputs as well as past outputs.
Input triggers the combinational circuit.	Clock pulse and previous outputs triggers the S C

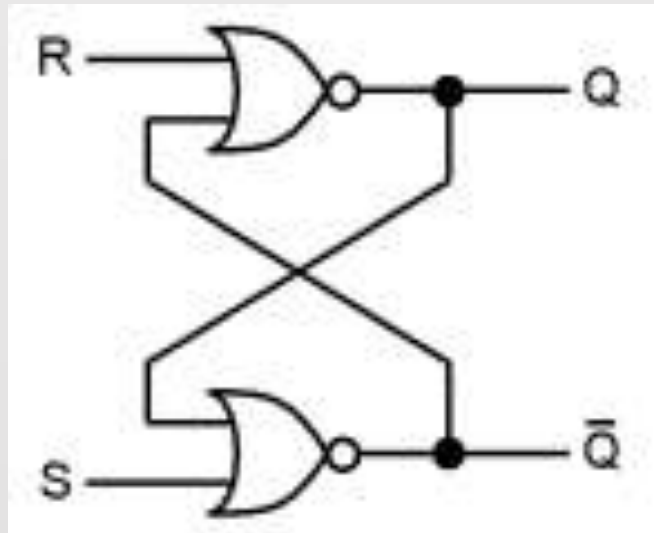
Latch

Latch is storage device. A latch is constructed from the cross coupled NAND or NOR gates.

The outputs of Latch are compliment of each other.

i.e. $Q \neq Q'$

SR Latch



SR latch using NOR gates

S	R	Q	Q'
0	0	1	1
0	1	0	1
1	0	1	0
1	1	Not used	Not used

Case I

When $S = 1$ and $R = 0$; $Q = 1$ and $Q' = 0$

When $S = 0$ and $R = 0$; $Q = 1$ and $Q' = 0$ (Memory)

Case II

When $S = 0$ and $R = 1$; $Q = 0$ and $Q' = 1$

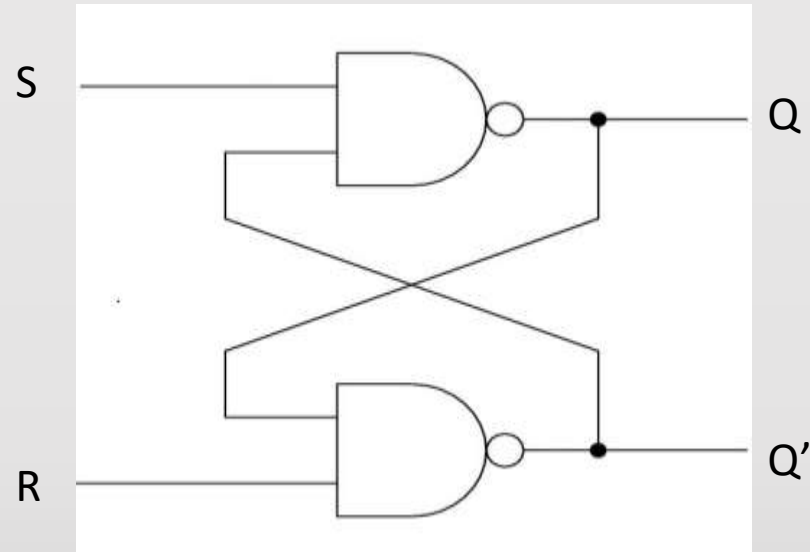
When $S = 0$ and $R = 0$; $Q = 0$ and $Q' = 1$ (Memory)

Case III

When $S = 1$ and $R = 1$; $Q = 0$ and $Q' = 0$ [Not Used]

A	B	F
0	0	1
0	1	0
1	0	0
1	1	0

SR Latch



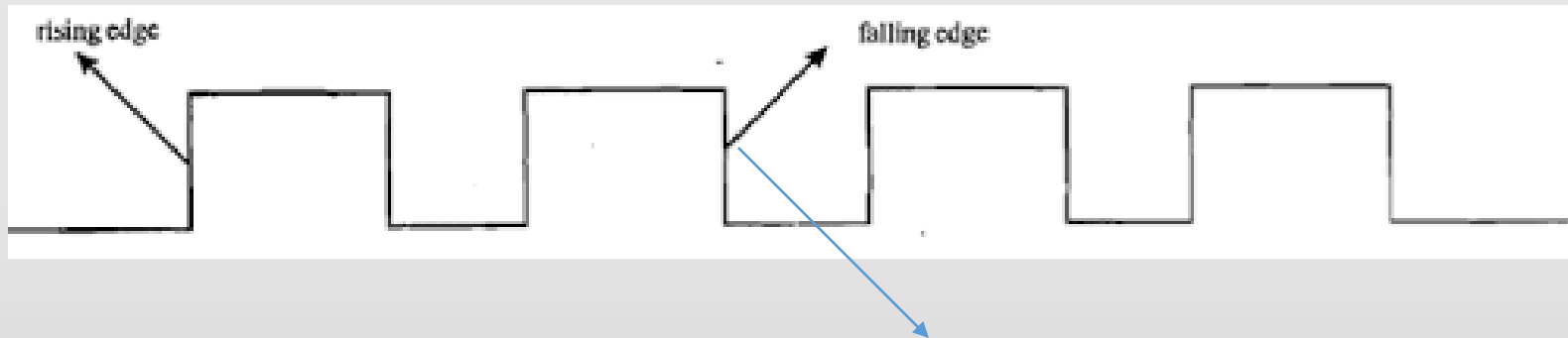
SR latch using NAND gates

S	R	Q	Q'
0	0	Not used	Not used
0	1	1	0
1	0	0	1
1	1	Memory	Memory

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

Clock Pulse and Triggering

Varying Signal that triggers or control the operation of Flip flops.

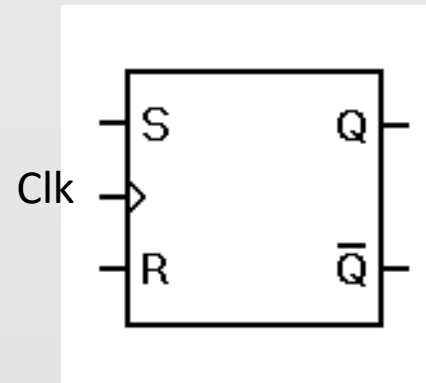
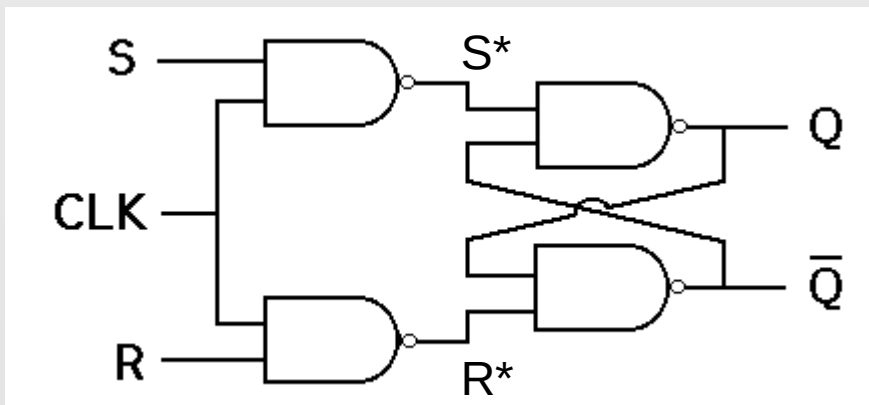


Triggering Methods

Level Triggering

Edge Triggering

SR Flip Flop



S*	R*	Q	Q'
0	0	Not used	Not used
0	1	1	0
1	0	0	1
1	1	Memory	Memory

TT of SR latch using NAND

$$S^* = (S \cdot \text{Clk})' = S' + \text{Clk}'$$

$$R^* = (R \cdot \text{Clk})' = R' + \text{Clk}'$$

When $\text{clk} = 0$;

$$S^* = S' + 1$$

$$= 1$$

$$R^* = R' + 1$$

$$= 1$$

When $\text{clk} = 1$;

$$S^* = S' + 0 = S'$$

$$R^* = R' + 0 = R'$$

When $S = 0$ and $R = 0$

$$S^* = 0' = 1$$

$$R^* = 0' = 1$$

When $S = 0$ and $R = 1$

$$S^* = 0' = 1$$

$$R^* = 1' = 0$$

When $S = 1$ and $R = 0$

$$S^* = 1' = 0$$

$$R^* = 0' = 1$$

When $S = 1$ and $R = 1$

$$S^* = 1' = 0$$

$$R^* = 1' = 0$$

TT of SR flip flop

Clk	S	R	Q	Q'
0	x	x	Mem	Mem
1	0	0	Mem	Mem
1	0	1	0	1
1	1	0	1	0
1	1	1	Invalid	Invalid

SR Flip Flop

Clk	S	R	Q_{n+1}
0	x	x	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Not used

Truth table of SR flip flop

Q_n	Q_{n+1}	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0

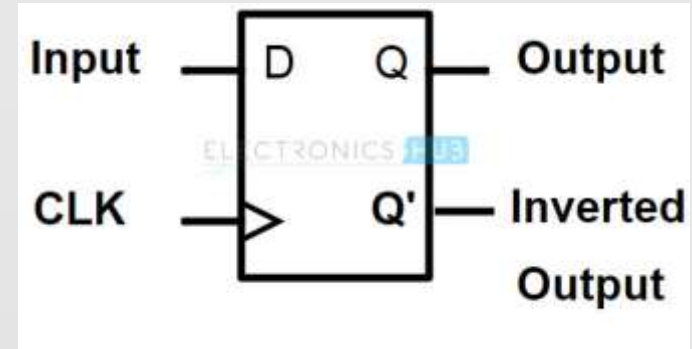
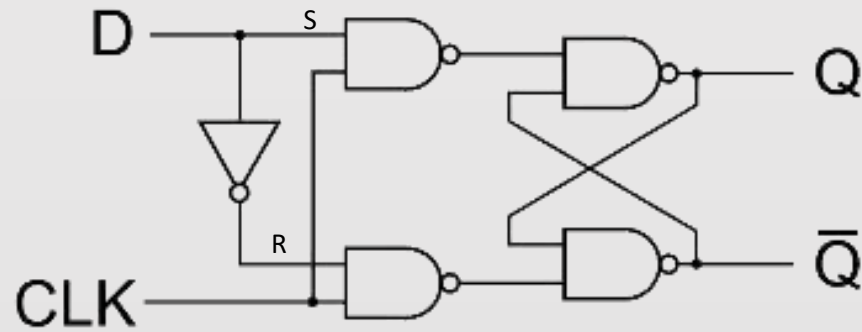
Excitation table of SR flip flop ©Lincoln University

Q_n	S	R	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	x
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	x

Characteristics table of SR flip flop

$$Q_{n+1} = Q_n R' + S$$

D Flip Flop



Clk	S	R	Q_{n+1}
0	x	x	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Not used

TT of SR flip flop

Clk	D	Q_{n+1}
0	x	Q_n
1	0	0
1	1	1

TT of D flip flop

D Flip Flop

Clk	D	Q_{n+1}
0	x	Q_n
1	0	0
1	1	1

TT of D flip flop

Q_n	D	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

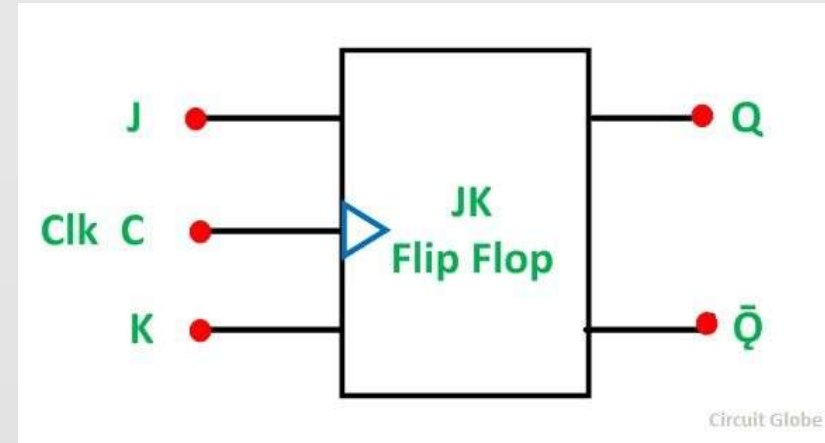
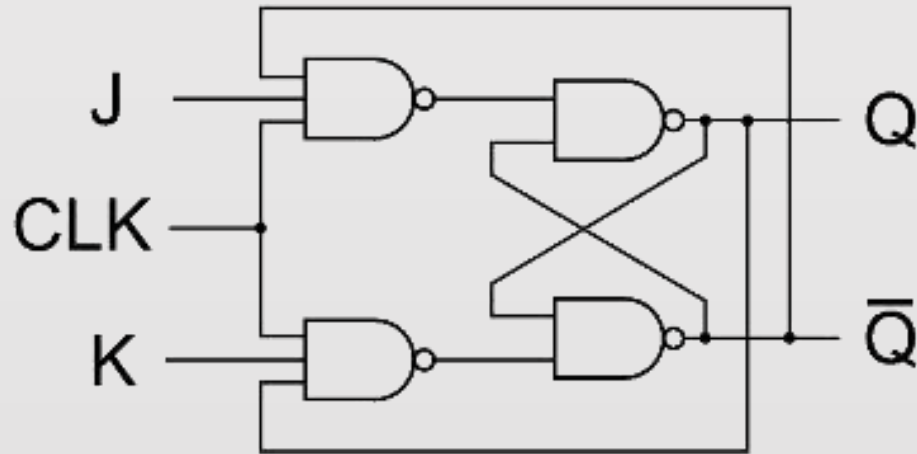
Characteristics of D flip flop

$$Q_{n+1} = D$$

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

Excitation of D flip flop

JK Flip Flop



CLK	S	R	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	<i>Invalid</i>

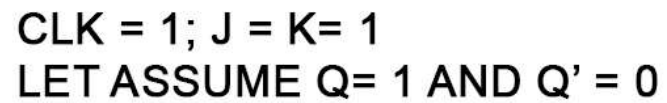
T.T for S-R Flip-Flop

CLK	J	K	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Q'_n

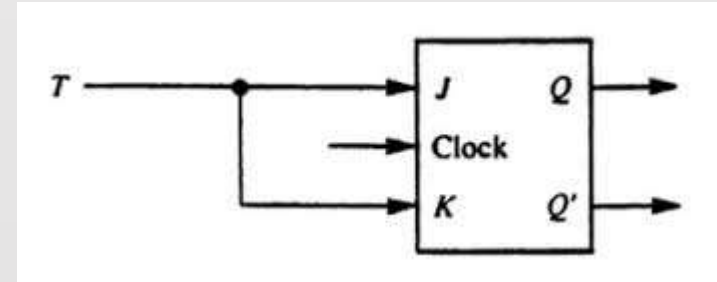
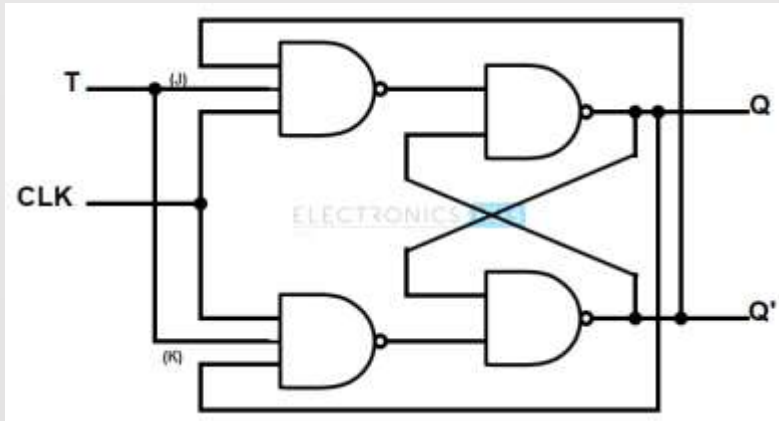
T.T for J-K Flip-Flop

Toggling

Your Partner for Excellence
Texas
 College of Management & IT



T Flip Flop



CLK	J	K	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Q_n'

T.T for J-K Flip-Flop

CLK	T	Q_{n+1}
0	X	Q_n
1	0	Q_n
1	1	Q_n'

T.T for T Flip-Flop

T Flip Flop

CLK	T	Q_{n+1}
0	X	Q_n
1	0	Q_n
1	1	Q_n'

T.T for T Flip-Flop

Q_n	T	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

Characteristics table for T Flip-Flop

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Excitation Table for T Flip-Flop

JK Flip Flop

CLK	J	K	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Q'_n

T.T for J-K Flip-Flop

Q_n	Q_{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

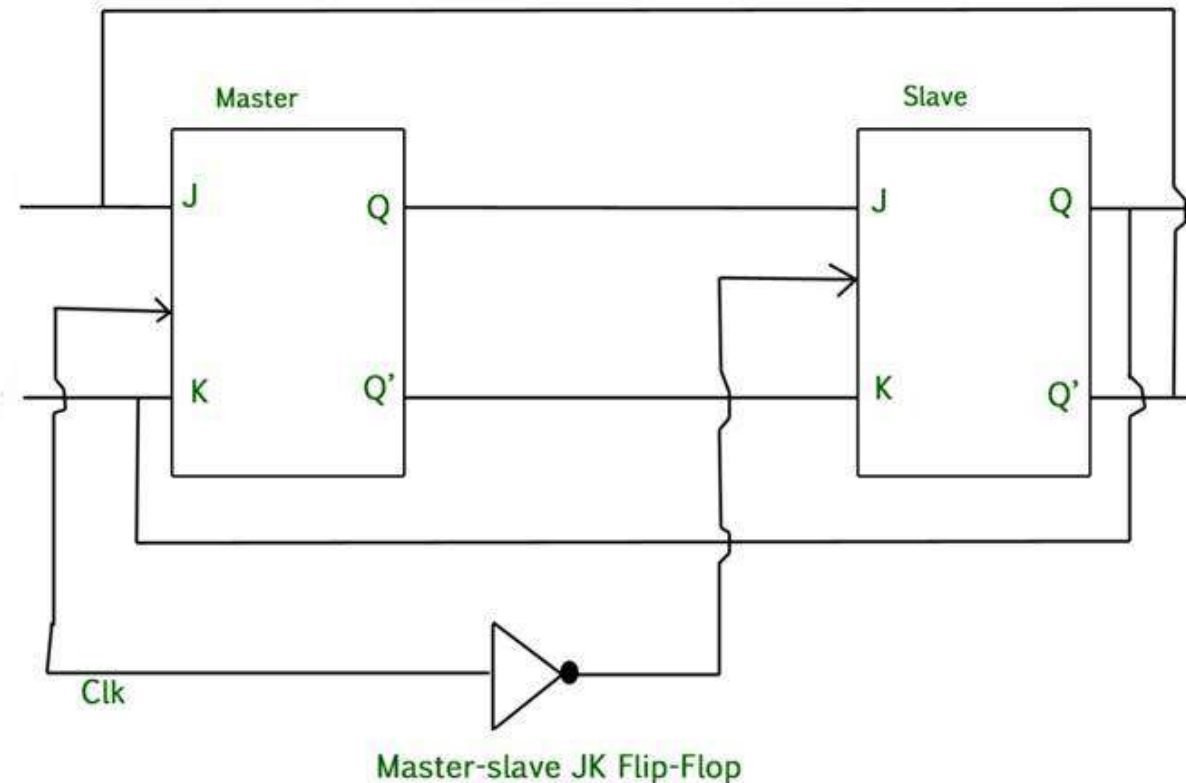
Excitation table for J-K Flip-Flop

Q_n	J	K	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

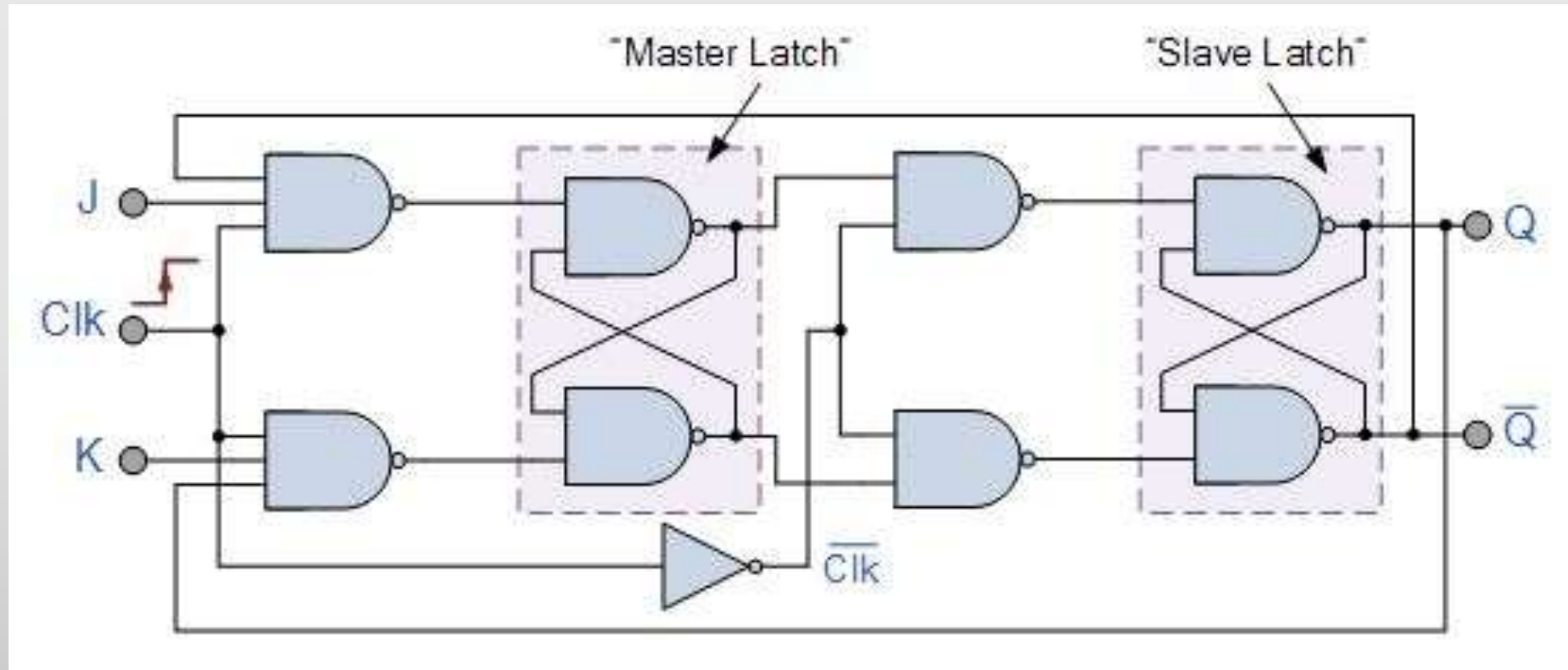
Characteristics Table for J-K Flip-Flop

Master - Slave JK Flip Flop

- Combination of 2 JK flip-flops in series.
- One acts as 'master' and the other as 'slave'
- It also includes an Inverter(Not Gate).



Master Slave JK Flip Flop



Master Slave JK Flip Flop

- When the clock is high, master is operational but slave is not operational(it will keep memory)
- When the clock is low, master is not functional but slave is. The output at slave is fed back to master but the clock is low. So, In this case effect of feedback is eliminated(which eliminates racing)

CLK	J	K	Q_{n+1}
0	X	X	Q_n
1	0	0	Q_n
1	0	1	0
1	1	0	1
1	1	1	Q'_n

T.T for J-K Flip-Flop

Conversion of Flip-Flop

(Realization of one Flip-Flop using other Flip-Flop)

Steps for Flip-Flop Conversion

- Make Characteristics table for required flip-flop.
- Make Excitation table for available flip-flop.
- Write Boolean expression for available flip-flop.
- Draw the Block Diagram.

Example 1 – JK to D

- Required flip-flop : D,

Available flip-flop : JK

Q_n	D	Q_{n+1}	J	K
0	0	0	0	x
0	1	1	1	x
1	0	0	x	1
1	1	1	x	0

Char table of D (required f/f)

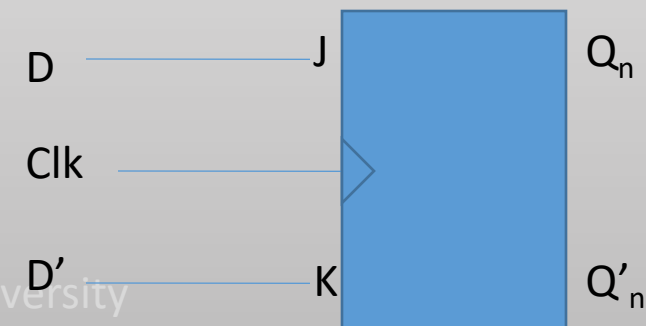
Q_n	Q_{n+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

Excitation table of JK (available f/f)

Using K map

$$J = D$$

$$K = D'$$



Example 2 – SR to JK

- Required flip-flop : JK,

Q_n	J	K	Q_{n+1}	S	R
0	0	0	0	0	x
0	0	1	0	0	x
0	1	0	1	1	0
0	1	1	1	1	0
1	0	0	1	x	0
1	0	1	0	0	1
1	1	0	1	x	0
1	1	1	0	0	1

From K map

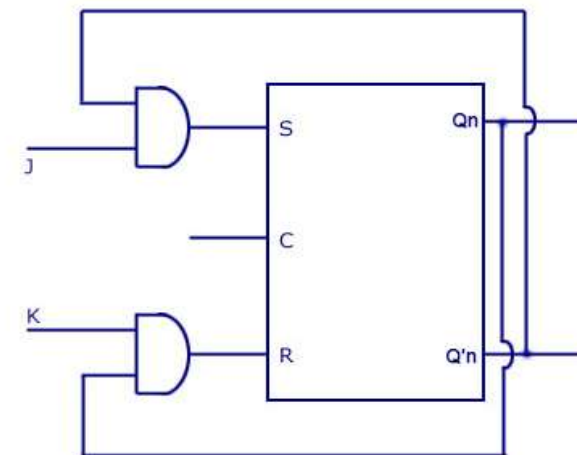
$$S = Q_n'J$$

$$R = Q_nK$$

©Lincoln University

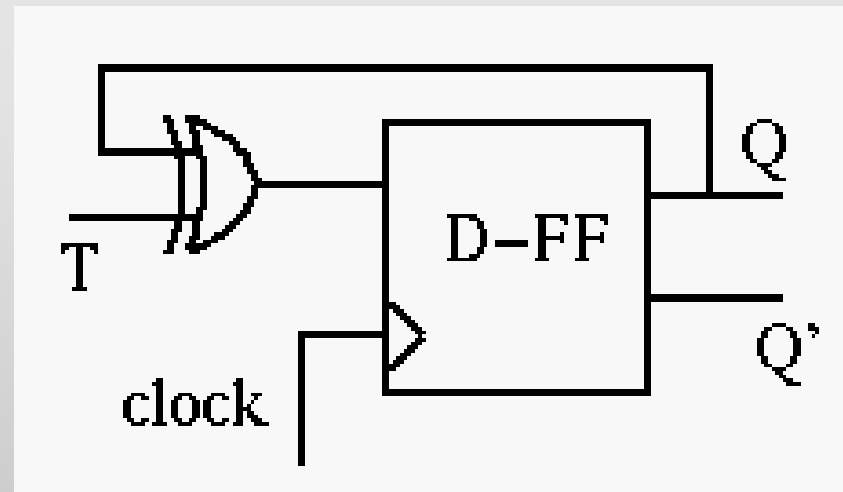
Available flip-flop : SR

Q_n	Q_{n+1}	S	R
0	0	0	x
0	1	1	0
1	0	0	1
1	1	x	0



Example 3 – T to D

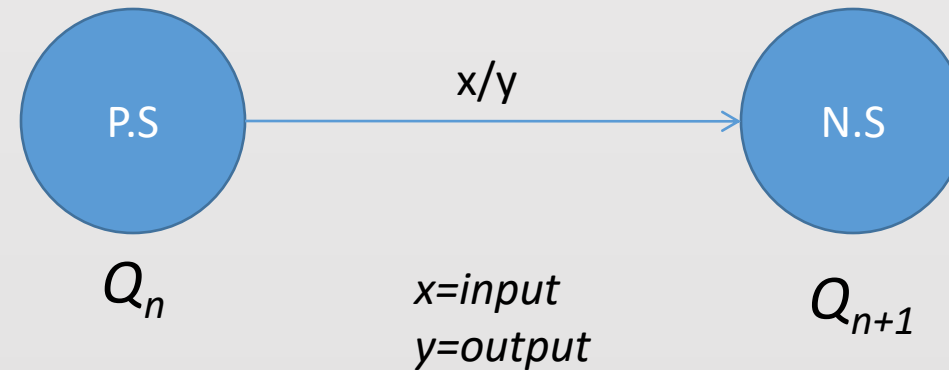
- Required flip-flop : T Available flip-flop : D



Direct Inputs

- Also called asynchronous inputs.
- Flip-flop have control over the outputs (Q and Q') regardless of clock signal.
- These inputs are called the preset (PRE) and clear (CLR).
- When preset = 0, Q equals to 1
- When clear = 0, Q equals to 0

State Diagram and State Table

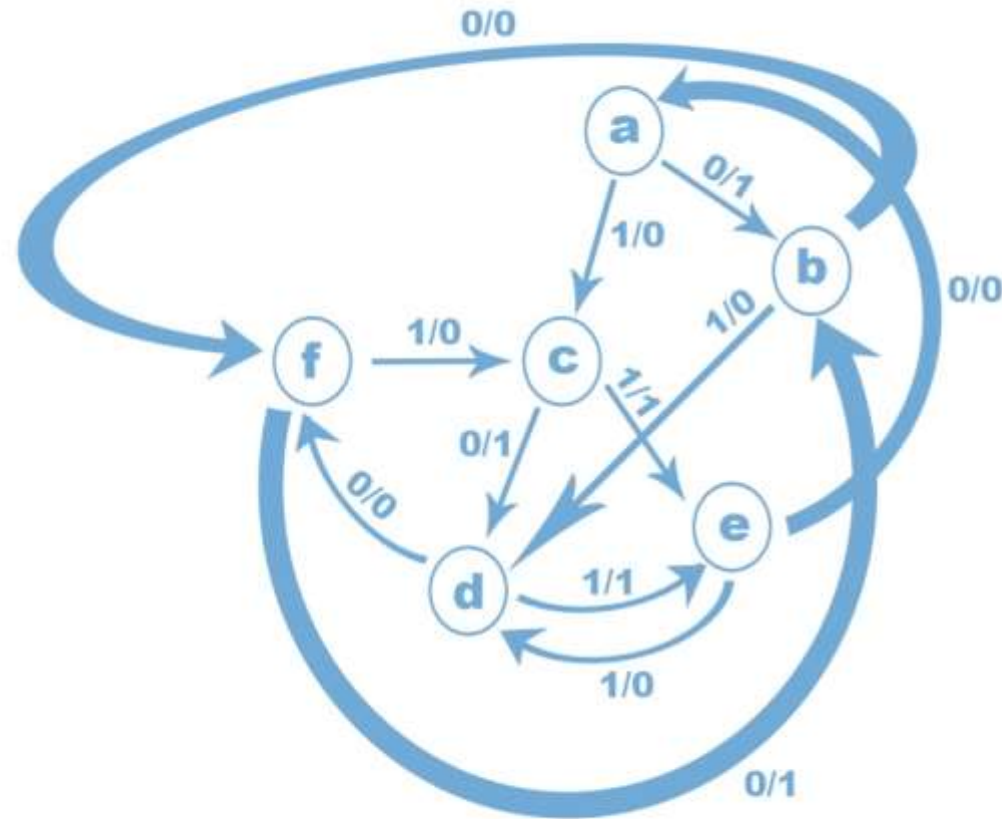


The relationship that exists among the inputs, outputs, present states and next states can be specified by either the State Diagram or State Table.

Design Procedure for Sequential Circuit

- Obtain the state table.
- Reduce the no. of state using state reduction method(If possible)
- Do state assignment(If required)
- Determine the number of flip flop required.
- Derive circuit excitation table from state table/diagram.
- Obtain the expression for circuit output and flip flop output.
- Implement the circuit.

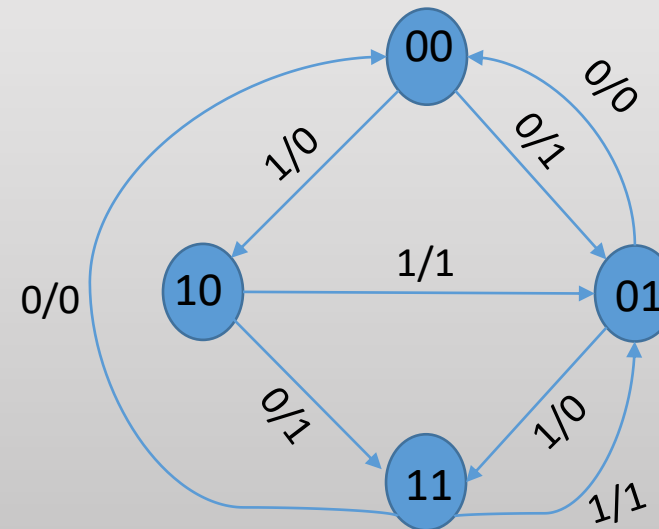
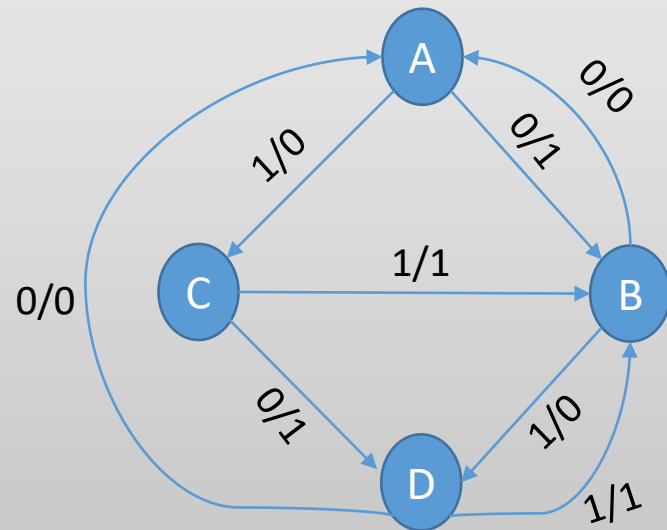
State Reduction and Assignment



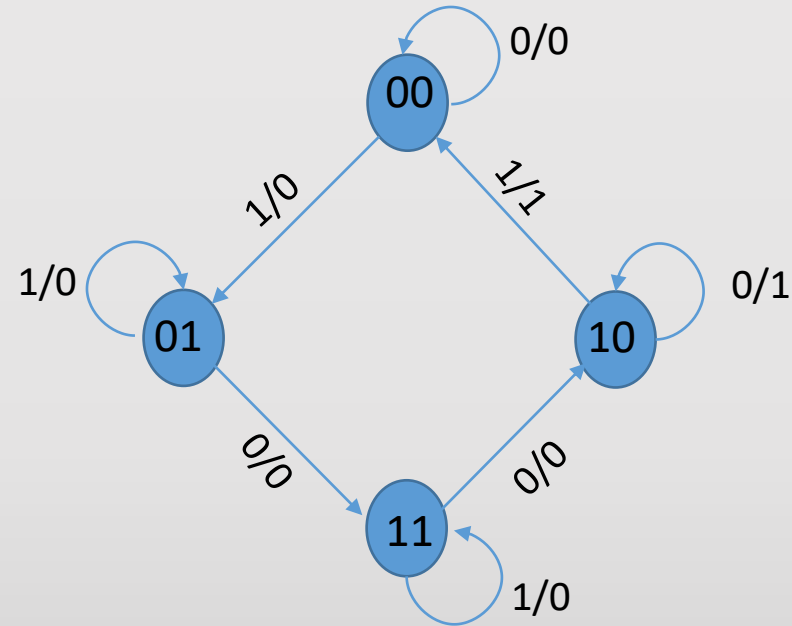
Present State	Next State		Output	
	x=0	x=1	x=0	X=1
A	B	C	1	0
B	F	D	0	0
C	D	E	1	1
D	F	E	0	1
E	A	D	0	0
F	B	C	1	0

Present State	Next State		Output	
	x=0	x=1	x=0	X=1
A	B	C	1	0
B	A	D	0	0
C	D	E	1	1
D	A	E	0	1
E	A	D	0	0

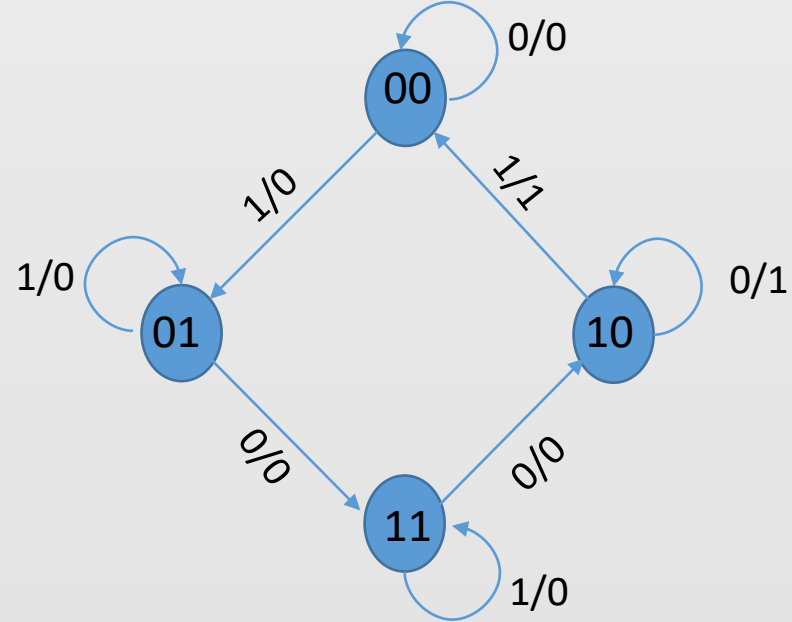
Present State	Next State		Output	
	x=0	x=1	x=0	X=1
A	B	C	1	0
B	A	D	0	0
C	D	B	1	1
D	A	B	0	1



Design a sequential circuit from the following state diagram using T flip-flop.



Present State		Next State				Output	
		x=0		x=1		x=0	X=1
0	0	0	0	0	1	0	0
0	1	1	1	0	1	0	0
1	0	1	0	0	0	1	1
1	1	1	0	1	1	0	0



Present State			Next State		Output (y)	Using T f/f	
Q_a	Q_b	x	Q_a^+	Q_b^+		T_a	T_b
0	0	0	0	0	0	0	0
0	0	1	0	1	0	0	1
0	1	0	1	1	0	1	0
0	1	1	0	1	0	0	0
1	0	0	1	0	1	0	0
1	0	1	0	0	1	1	0
1	1	0	1	0	0	0	1
1	1	1	1	1	0	0	0

$$T_a = Q_a'.Q_b.x' + Q_a.Q_b'x$$

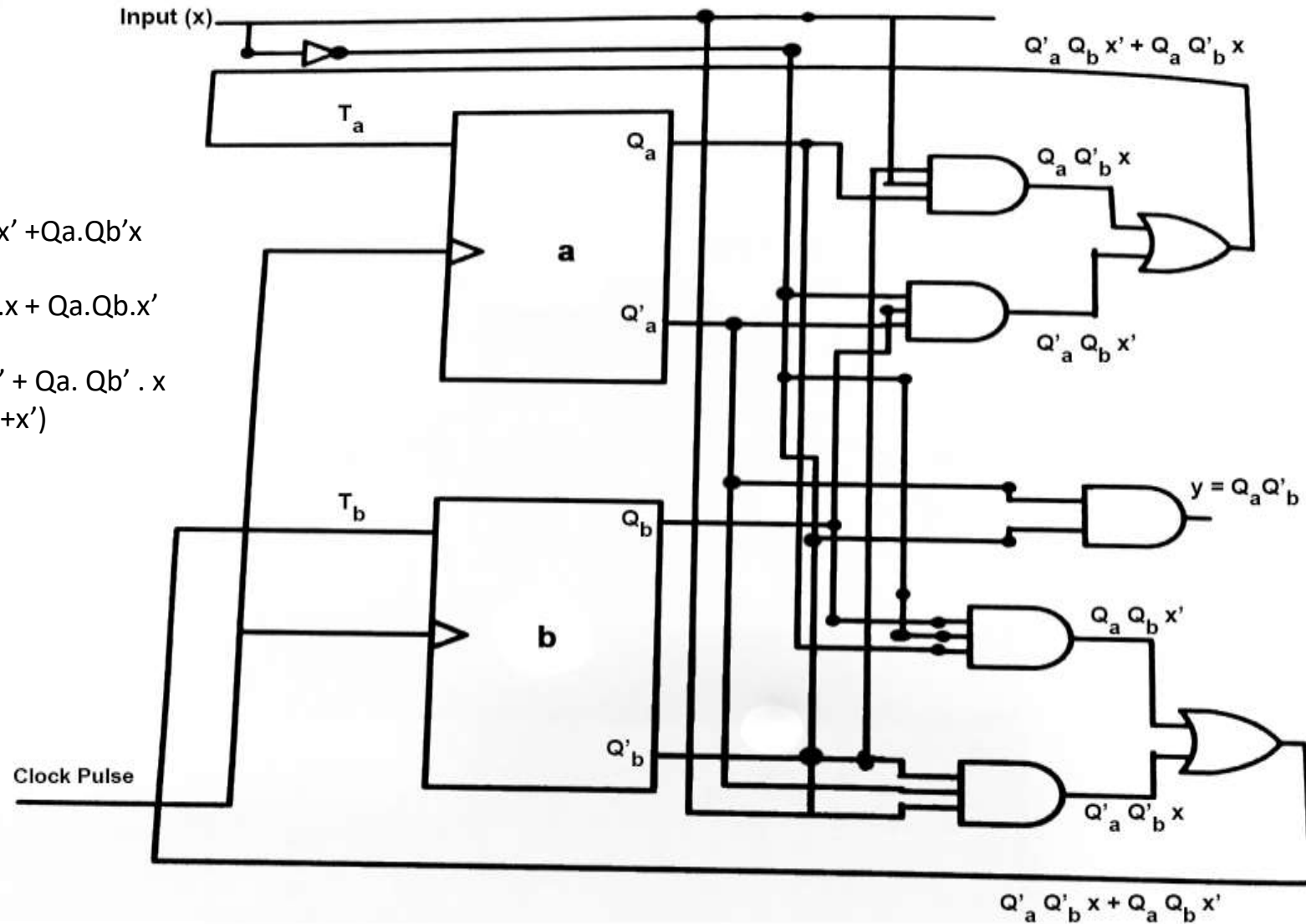
$$T_b = Q_a'.Q_b'.x + Q_a.Q_b.x'$$

$$\begin{aligned}
 Y &= Q_a.Q_b'.x' + Q_a.Q_b'.x \\
 &= Q_a.Q_b'(x+x') \\
 &= Q_a.Q_b'
 \end{aligned}$$

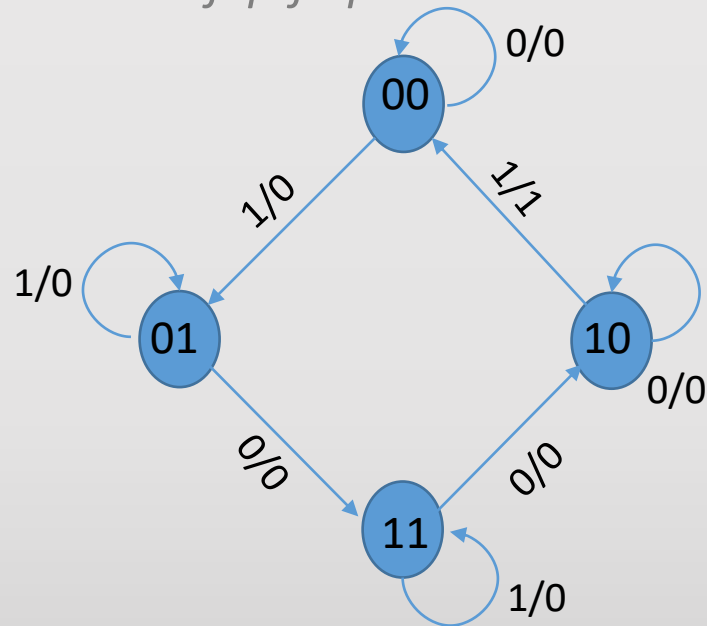
$$T_a = Q_a'.Q_b.x' + Q_a.Q_b'.x$$

$$T_b = Q_a'.Q_b'.x + Q_a.Q_b.x'$$

$$\begin{aligned} Y &= Q_a.Q_b'.x' + Q_a.Q_b'.x \\ &= Q_a.Q_b'(x+x') \\ &= Q_a.Q_b' \end{aligned}$$



From the state diagram given below, Design a Sequential circuit using SR flip-flop.



Present State			Next State		Output (y)	Using SR f/f			
Q_a	Q_b	x	Q_a^+	Q_b^+		S_a	R_a	S_b	R_b
0	0	0	0	0	0	0	x	0	x
0	0	1	0	1	0	0	x	1	0
0	1	0	1	1	0	1	0	x	0
0	1	1	0	1	0	0	x	x	0
1	0	0	1	0	0	x	0	0	x
1	0	1	0	0	1	0	1	x	x
1	1	0	1	0	0	x	0	0	1
1	1	1	1	1	0	x	0	x	0

Present State			Next State		Output (y)	Using SR f/f			
Q_a	Q_b	x	Q_a^+	Q_b^+		S_a	R_a	S_b	R_b
0	0	0	0	0	0	0	x	0	x
0	0	1	0	1	0	0	x	1	0
0	1	0	1	1	0	1	0	x	0
0	1	1	0	1	0	0	x	x	0
1	0	0	1	0	0	x	0	0	x
1	0	1	0	0	1	0	1	0	x
1	1	0	1	0	0	x	0	0	1
1	1	1	1	1	0	x	0	x	0

$$S_a(Q_a, Q_b, x) = \Sigma (2) \text{ \& } d(Q_a, Q_b, x) = \Sigma (4, 6, 7)$$

$$R_a(Q_a, Q_b, x) = \Sigma (5) \text{ \& } d(Q_a, Q_b, x) = \Sigma (0, 1, 3)$$

$$S_b(Q_a, Q_b, x) = \Sigma (1) \text{ \& } d(Q_a, Q_b, x) = \Sigma (2, 3, 7)$$

$$R_b(Q_a, Q_b, x) = \Sigma (6) \text{ \& } d(Q_a, Q_b, x) = \Sigma (0, 4, 5)$$

$$y(Q_a, Q_b, x) = \Sigma (5)$$

$$S_a(Q_a, Q_b, x) = \Sigma (2) \text{ \& } d(Q_a, Q_b, x) = \Sigma (4, 6, 7)$$

Qbx \ Qa	00	01	11	10
0	000	001	011	010
1	100	101	111	110

Map for S_a : A blue circle highlights the cell (1, 10) containing '1'. Blue 'x' marks are present in cells (1, 00), (1, 11), and (0, 110).

$$S_a = Q_b x'$$

$$S_b(Q_a, Q_b, x) = \Sigma (1) \text{ \& } d(Q_a, Q_b, x) = \Sigma (2, 3, 7)$$

Qbx \ Qa	00	01	11	10
0	000	001	011	010
1	100	101	111	110

Map for S_b : A blue circle highlights the cell (0, 01) containing '1'. Blue 'x' marks are present in cells (0, 11), (0, 10), and (1, 11).

$$S_b = Q_a' x$$

$$R_a(Q_a, Q_b, x) = \Sigma (5) \text{ \& } d(Q_a, Q_b, x) = \Sigma (0, 1, 3)$$

Qbx \ Qa	00	01	11	10
0	000	001	011	010
1	100	101	111	110

Map for R_a : A blue circle highlights the cell (1, 01) containing '1'. Blue 'x' marks are present in cells (0, 00), (0, 01), and (0, 11).

$$R_a = Q_a' x$$

$$R_b(Q_a, Q_b, x) = \Sigma (6) \text{ \& } d(Q_a, Q_b, x) = \Sigma (0, 4, 5)$$

Qbx \ Qa	00	01	11	10
0	000	001	011	010
1	100	101	111	110

Map for R_b : A blue circle highlights the cell (1, 10) containing '1'. Blue 'x' marks are present in cells (0, 00), (1, 00), and (1, 01).

$$R_b = Q_a x'$$

$$y(Q_a, Q_b, x) = \Sigma (5)$$

		Qbx			
		00	01	11	10
Qa	0	000	001	011	010
	1	100	101	111	110

$$y = Q_a Q'_b x$$

$$y = Q_a Q'_b x$$

$$S_a = Q_b x'$$

$$S_b = Q'_a x$$

$$R_a = Q'_b x$$

$$R_b = Q_a x'$$

$$y = Q_a Q_b' x$$

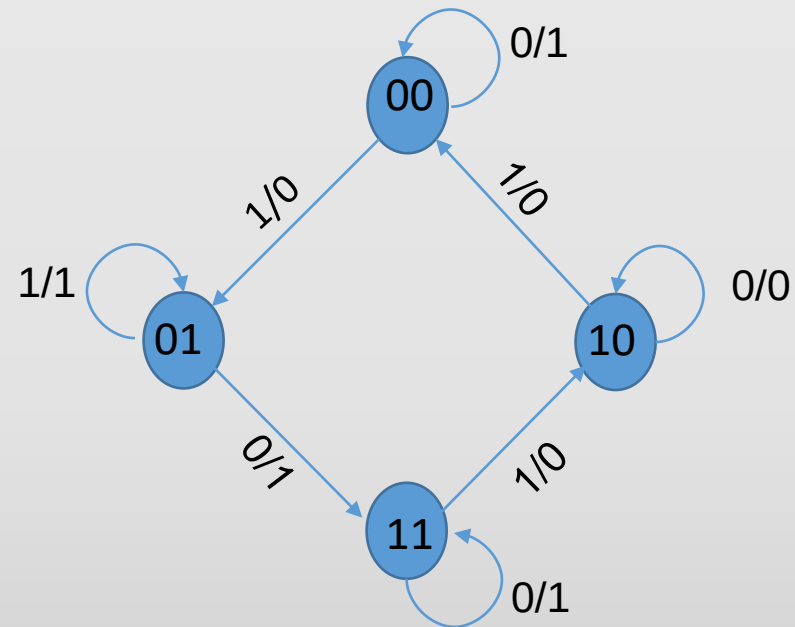
$$S_a = Q_b x'$$

$$S_b = Q_a' x$$

$$R_a = Q_b' x$$

$$R_b = Q_a x'$$

Design a sequential circuit from the following state diagram using S-R flip-flop.



Sequential Logic

Counters

Counters

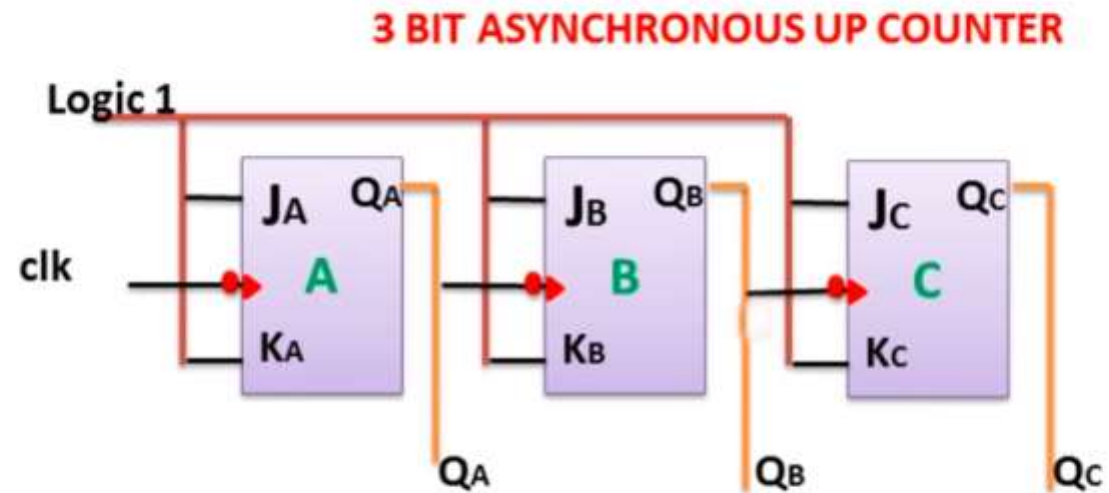
- A special type of sequential circuit used to count the pulse is known as a counter. It's a collection of flip flops where the clock signal is applied is known as counters.
- The counter that counts in ascending order is Up Counter
- The counter that counts in descending order Down Counter.

Counters are divided into two categories:

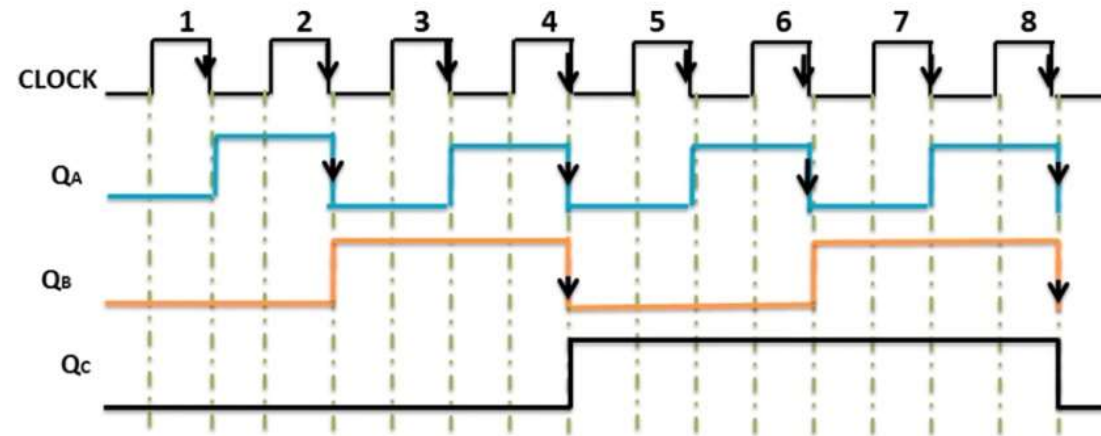
Asynchronous(Ripple) counter - No Universal clock for flip-flops

Synchronous counter – Universal clock for flip-flops

3-bit Asynchronous Up Counter



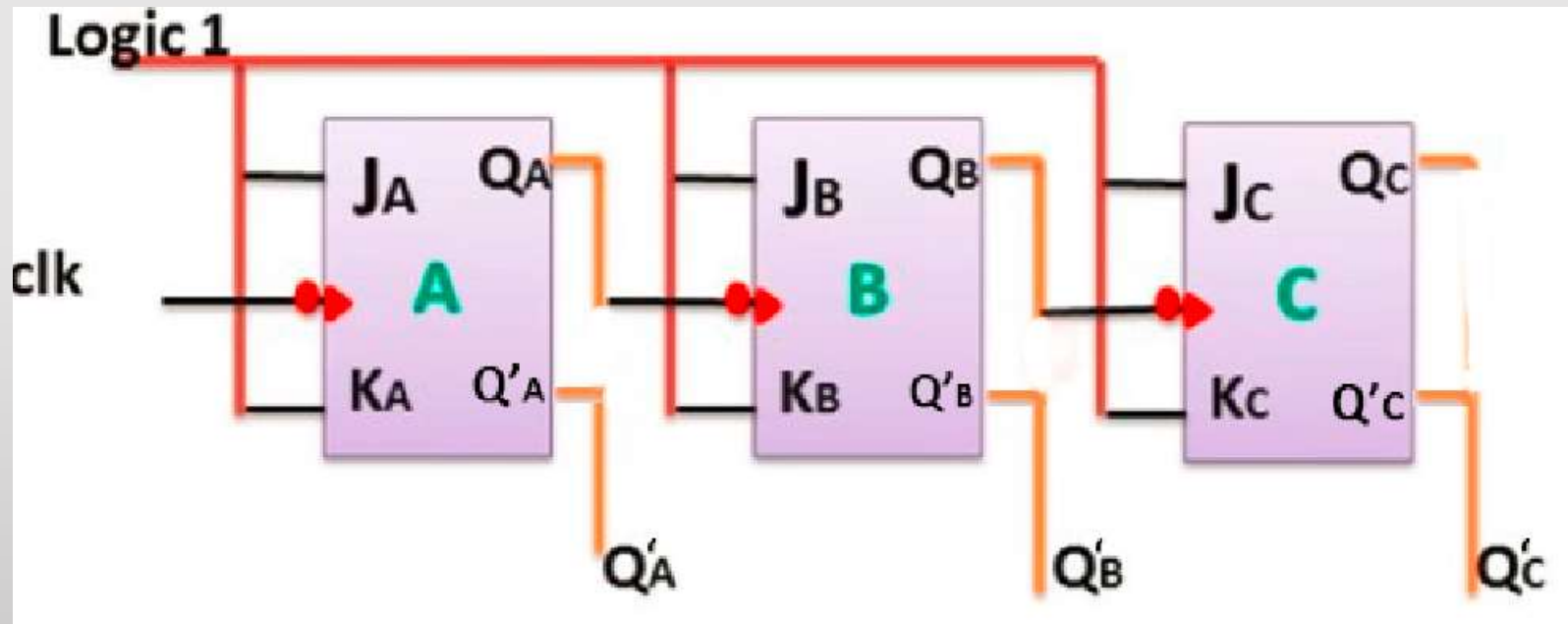
3-bit Asynchronous Up Counter



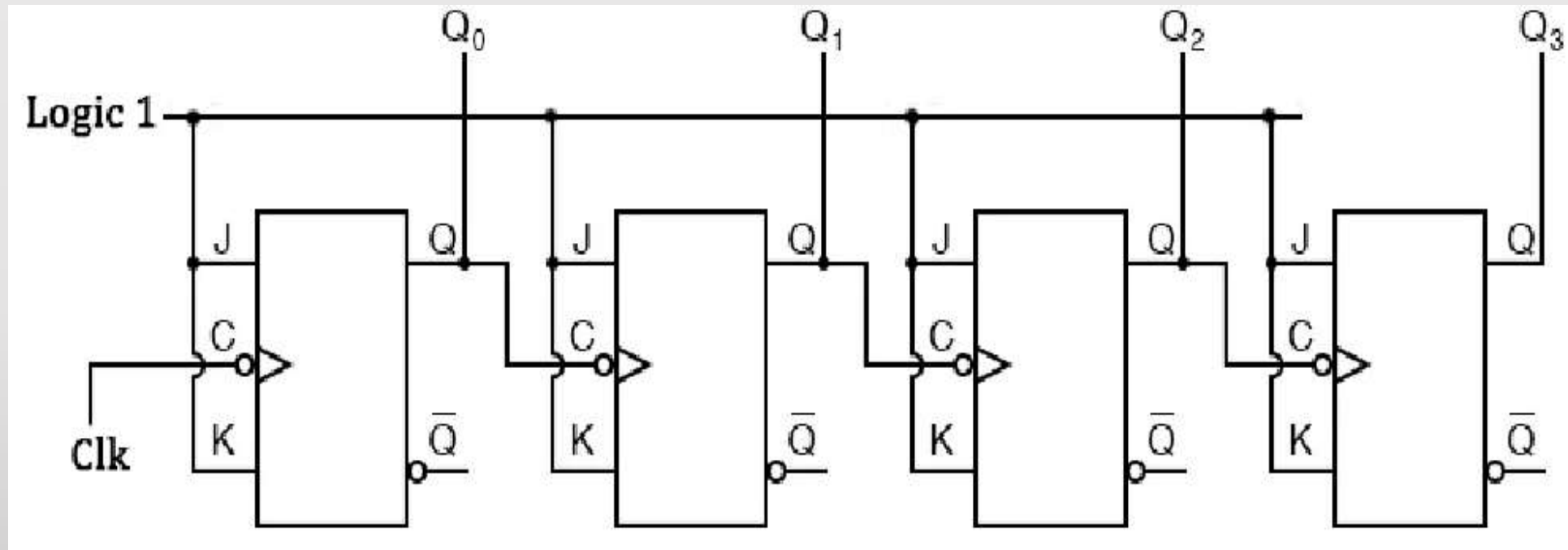
3-bit Asynchronous Up Counter

Clk	Qc	Qb	Qa	Dec
Initially(0)	0	0	0	0
1st	0	0	1	1
2nd	0	1	0	2
3rd	0	1	1	3
4th	1	0	0	4
5th	1	0	1	5
6th	1	1	0	6
7th	1	1	1	7

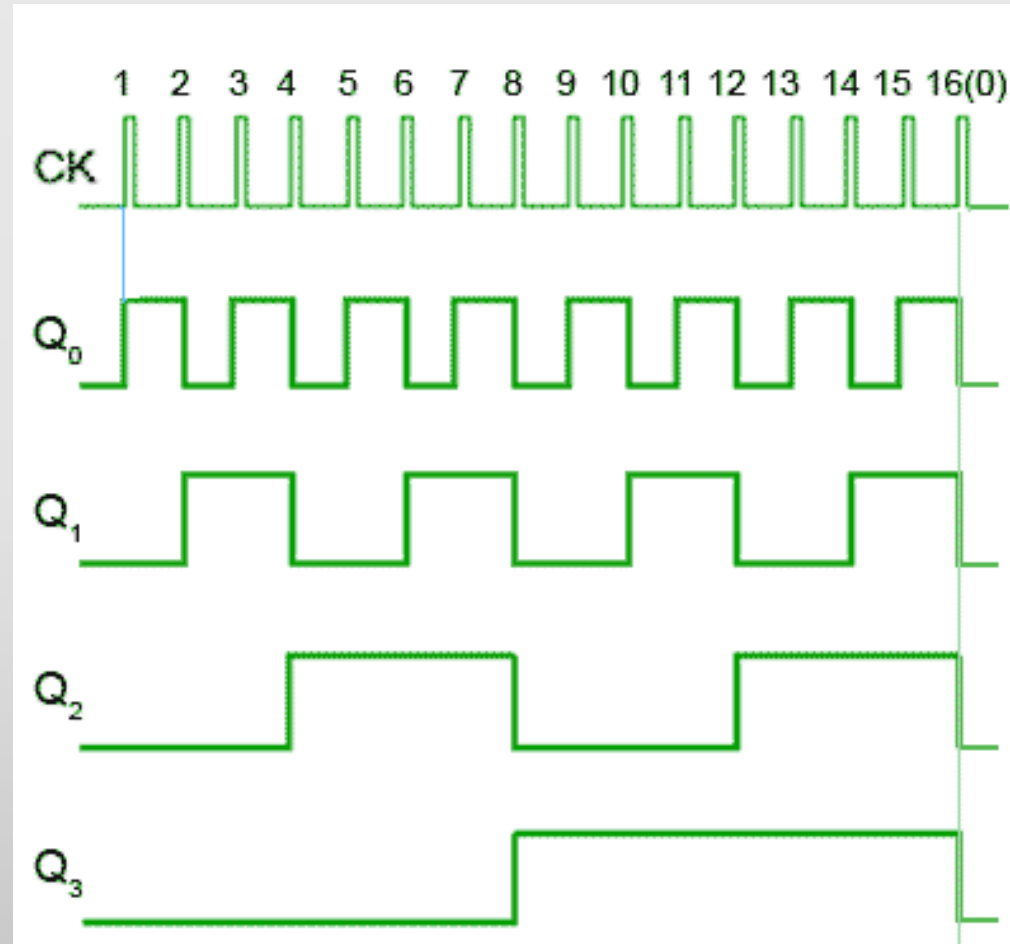
3-bit Asynchronous Down Counter



4-bit Asynchronous Up Counter



4-bit Asynchronous Up Counter



4-bit Asynchronous Up Counter

Clk	Q3	Q2	Q1	Q0	Dec
Initially(0)	0	0	0	0	0
1st	0	0	0	1	1
2nd	0	0	1	0	2
3rd	0	0	1	1	3
4th	0	1	0	0	4
5th	0	1	0	1	5
6th	0	1	1	0	6
7th	0	1	1	1	7
8th	0	0	0	0	8
9th	1	0	0	1	9
10th	1	0	1	0	10
11th	1	0	1	1	11
12th	1	1	0	0	12
13th	1	1	0	1	13
14th	1	1	1	0	14

- 1. Design a 4-bit Asynchronous Down Counter.*
- 2. Design a 2-bit Asynchronous Up Counter.*
- 3. Design a 2-bit Asynchronous Down Counter.*

3 - bit Up/Down Ripple Counter

M	Q	Q'	Clk
0 (up)	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1 (down)	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

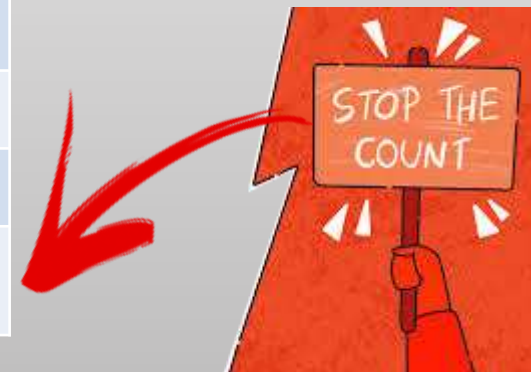
M	QQ'		
		1	1
	1	1	

$$\text{Clk} = \text{MQ}' + \text{M}'\text{Q}$$

BCD Ripple Counter (Decade Counter)

Mod -10 Counter

BCD				Dec
<i>Q3</i>	<i>Q2</i>	<i>Q1</i>	<i>Q0</i>	
0	0	0	0	0
0	0	0	1	1
0	0	1	0	2
0	0	1	1	3
0	1	0	0	4
0	1	0	1	5
0	1	1	0	6
0	1	1	1	7
1	0	0	0	8
1	0	0	1	9
1	0	1	0	10



Mod-n(*Modulus-n*) Counters

- **Mod-5 Counter** – has 5 states.

000(0) → 001(1) → 010(2) → 011(3) → 100(4)

Mod - 15 Counter - 15 states.... Max count - 14

Mod – 9 Counter – 9 states.... Max count - 8

Synchronous Counters

- How to design Synchronous Counters?
 1. Decide the number of flip flops.
 2. Excitation table of flip flop.
 3. State diagram and circuit excitation table.
 4. Obtain simplified equations using K map.
 5. Draw the logic diagram.

2-bit Synchronous Up Counter

00 → 01 → 10 →

1 1

Q _n	Q _{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	x	1
1	1	x	0

Present state		Next state		f/f inputs			
Q _a	Q _b	Q _{a+}	Q _{b+}	J _a	K _a	J _b	K _b
0	0	0	1	0	x	1	x
0	1	1	0	1	x	x	1
1	0	1	1	x	0	1	x
1	1	0	0	x	1	x	1

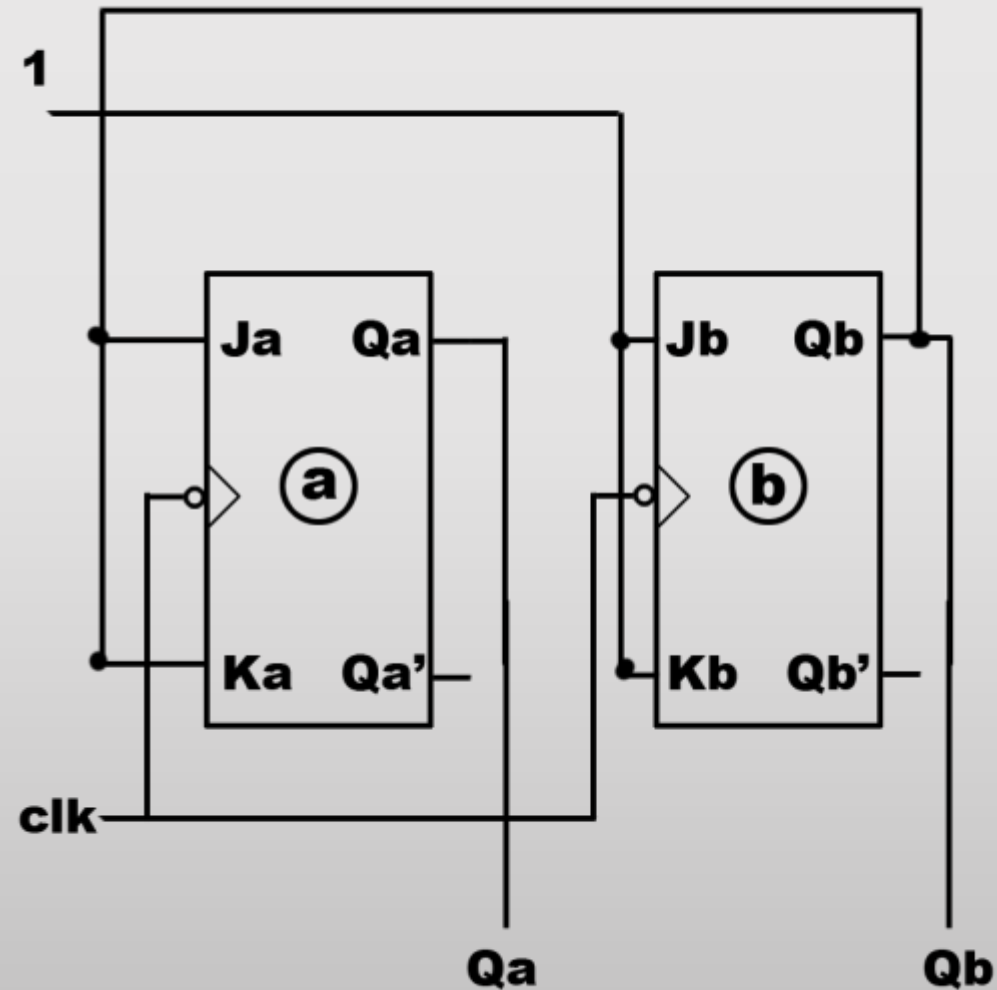
For J_a

	Q _b 0	1
Q _a 0		1
1	x	x

J_a = Q_b
 K_a = Q_b
 J_b = 1
 K_b = 1

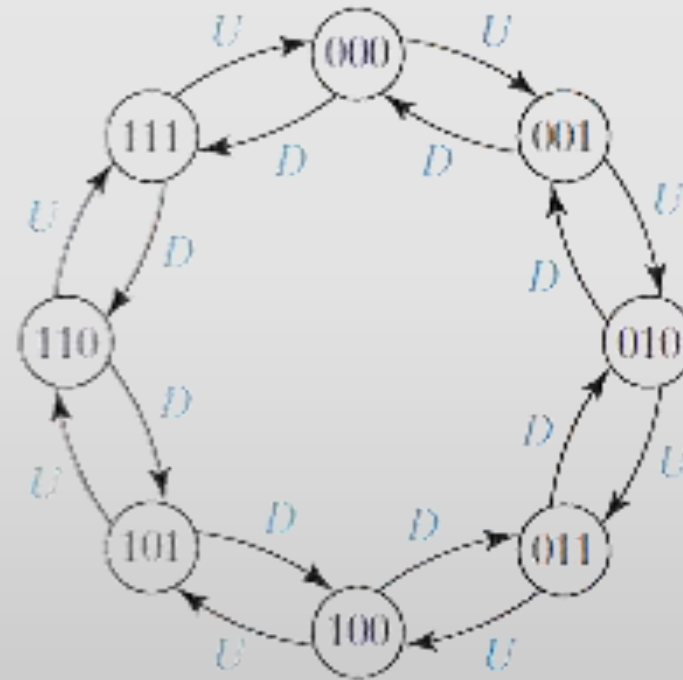
2-bit Synchronous Up Counter

$J_a = Q_b$
 $K_a = Q_b$
 $J_b = 1$
 $K_b = 1$



3-bit Synchronous Up/Down Counter

Q_n	Q_{n+1}	T
0	0	0
0	1	1
1	0	1
1	1	0



M	Q_C	Q_B	Q_A	Q_C^+	Q_B^+	Q_A^+	T_C	T_B	T_A
0	0	0	0	0	0	1	0	0	1
0	0	0	1	0	1	0	0	1	1
0	0	1	0	0	1	1	0	0	1
0	0	1	1	1	0	0	1	1	1
0	1	0	0	1	0	1	0	0	1
0	1	0	1	1	1	0	0	1	1
0	1	1	0	1	1	1	0	0	1
0	1	1	1	0	0	0	1	1	1
1	0	0	0	1	1	1	1	1	1
1	0	0	1	0	0	0	0	0	1
1	0	1	0	0	0	1	0	1	1
1	0	1	1	0	1	0	0	0	1
1	1	0	0	0	1	1	1	1	1
1	1	0	1	1	0	0	0	0	1
1	1	1	0	1	0	1	0	1	1
1	1	1	1	1	1	0	0	0	1

3-bit Synchronous Up/Down Counter

$$T_A = 1$$

$$T_B = M'Q_A + MQ_A'$$

$$T_C = M'Q_BQ_A + MQ_B'Q_A'$$

For T_C

		$Q_B Q_A$			
		00	01	11	10
$M Q_C$	00			1	
	01			1	
	11	1			
	10	1			

For T_B

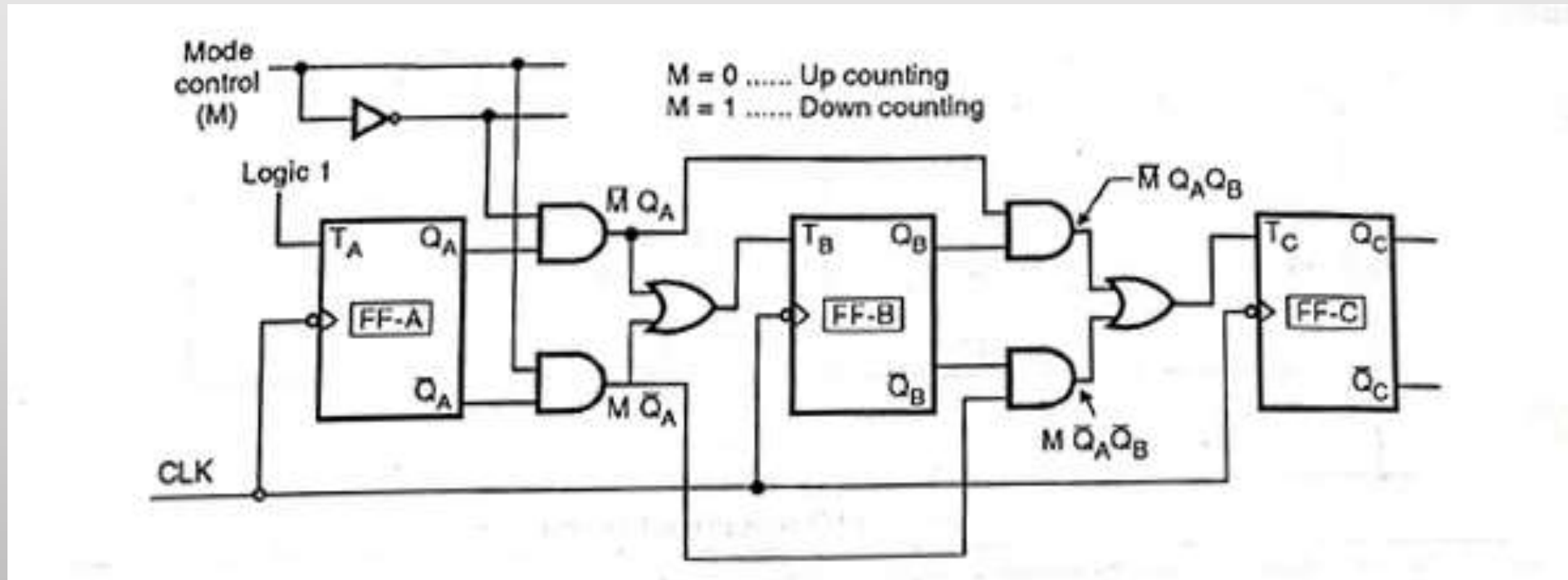
		$Q_B Q_A$			
		00	01	11	10
$M Q_C$	00		1	1	
	01		1	1	
	11	1			1
	10	1			1

3-bit Synchronous Up/Down Counter

$$T_A = 1$$

$$T_B = M'Q_A + MQ_A'$$

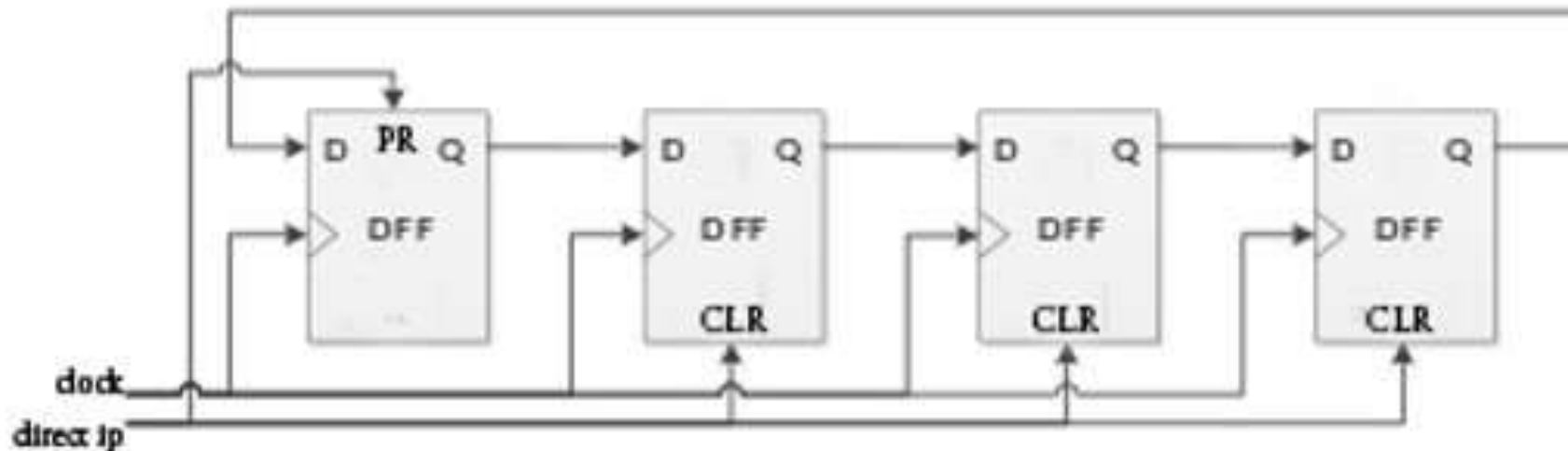
$$T_C = M'Q_BQ_A + MQ_B'Q_A'$$



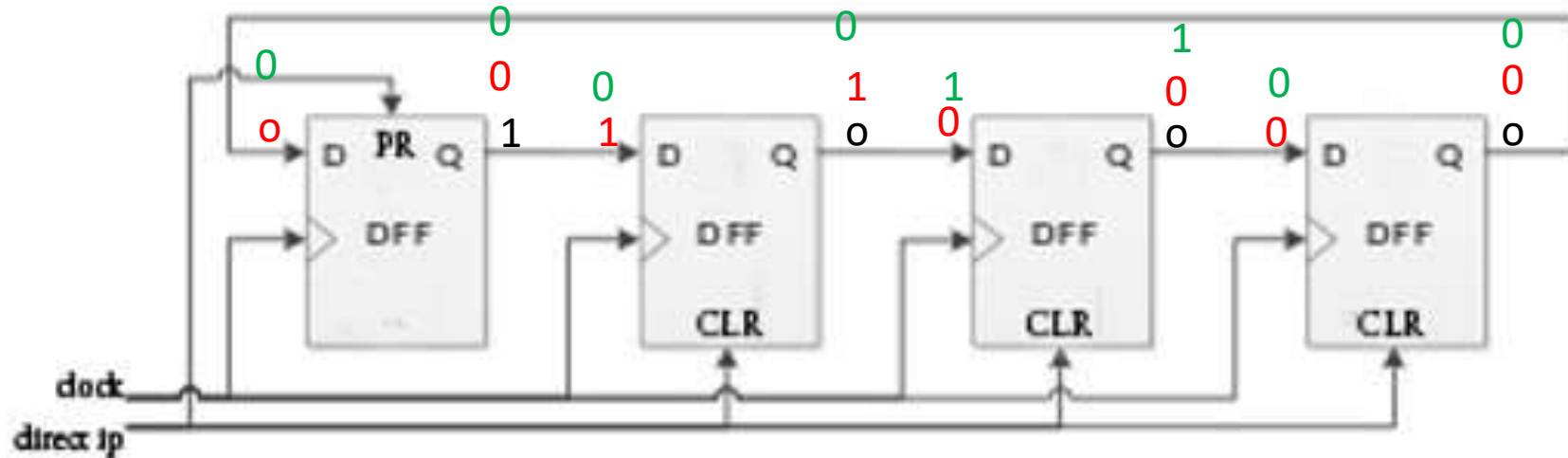
- 1. Design a 3-bit Synchronous Down Counter.*
- 2. Design a 3-bit Synchronous Up Counter.*

Ring Counter

- Special type of Counter.
- Output of last flip-flop is connected to first flip-flop.
- No. of states = No. of flip-flops used.
- $PR = 0$; $Q = 1$ and $CLR = 0$; $Q = 0$



Ring Counter

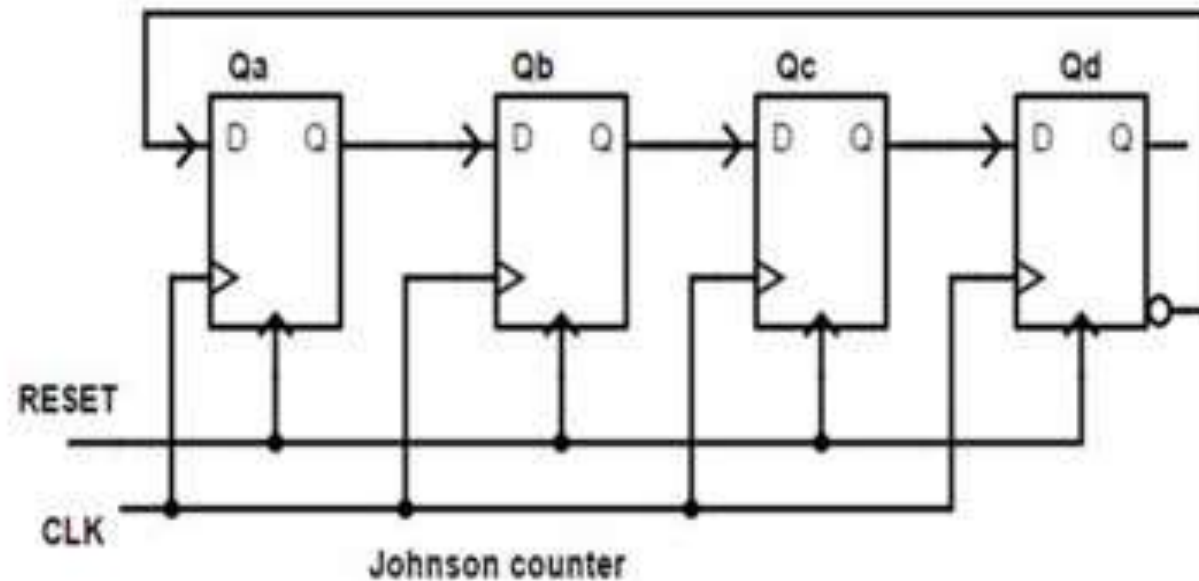


Direct i/p	Q1	Q2	Q3	Q4
low	1	0	0	0
high	0	1	0	0
high	0	0	1	0
high	0	0	0	1
high	1	0	0	0

Johnson Counter

- Modified from Ring Counter.
- Twisted Ring Counter.
- No. of states = $2 \times [\text{No. of flip-flops used}]$

Q_A	Q_B	Q_C	Q_D
0	0	0	0
1	0	0	0
1	1	0	0
1	1	1	0
1	1	1	1
0	1	1	1
0	0	1	1
0	0	0	1
repeat			



Sequential Logic

Registers

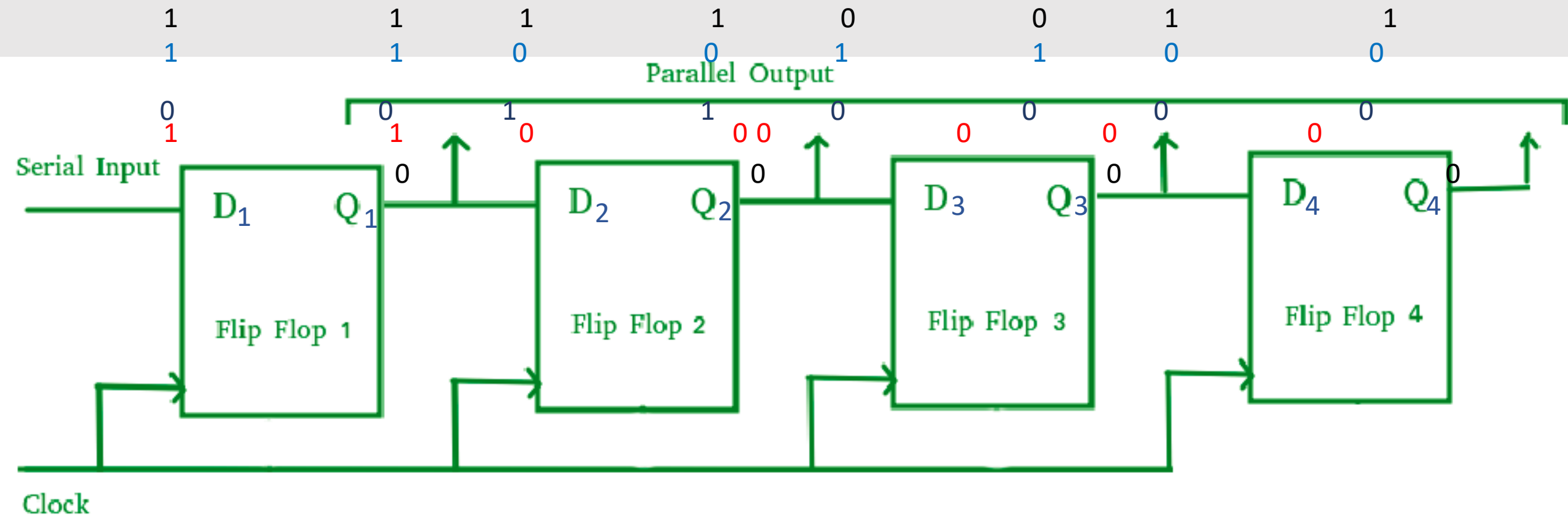
What is Register?

- Group of flip-flops.
- n bit register consists n numbers of flip flops.
- Stores multi bit data.

Shift Register

- It can shift the binary information stored in flip-flop either to left or right.
- Types:
 1. Serial In Serial Out Register(SISO)
 2. Serial In Parallel Out Register(SIPO)
 3. Parallel In Serial Out Register(PISO)
 4. Parallel In Parallel Out Register(PIPO)

1101 Serial In Parallel Out Register(SIPOO)



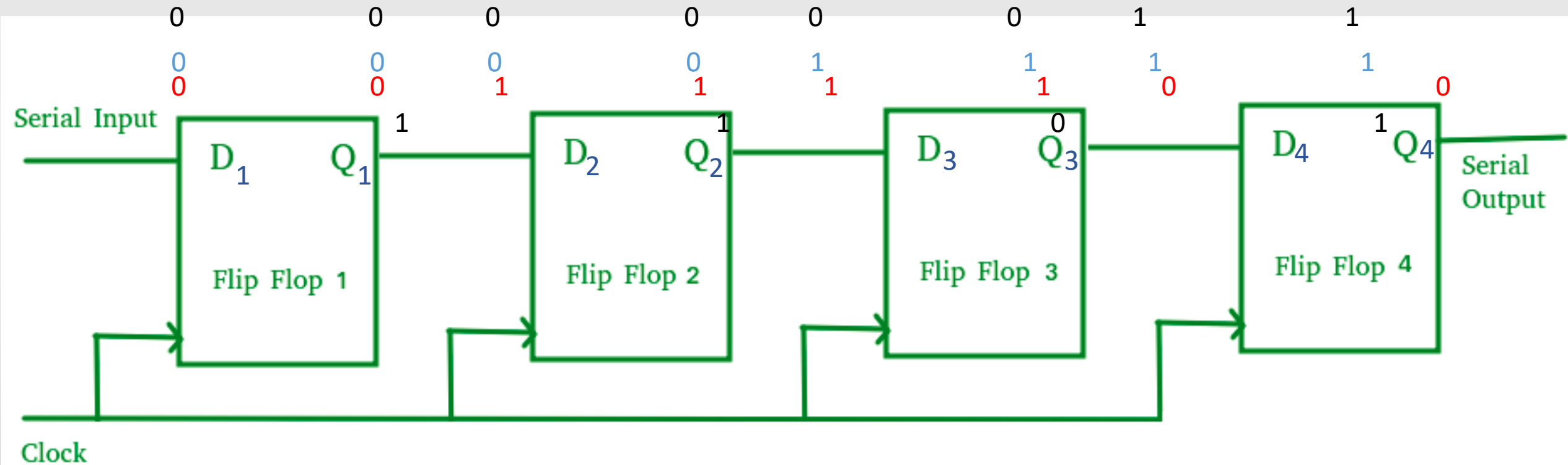
Serial In Parallel Out Register(SIPOO)

1101

Clk	Q1	Q2	Q3	Q4
Initially	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	1	0	1	0
4	1	1	0	1

1101

Serial In Serial Out Register(SISO)



Serial In Serial Out Register(SISO)

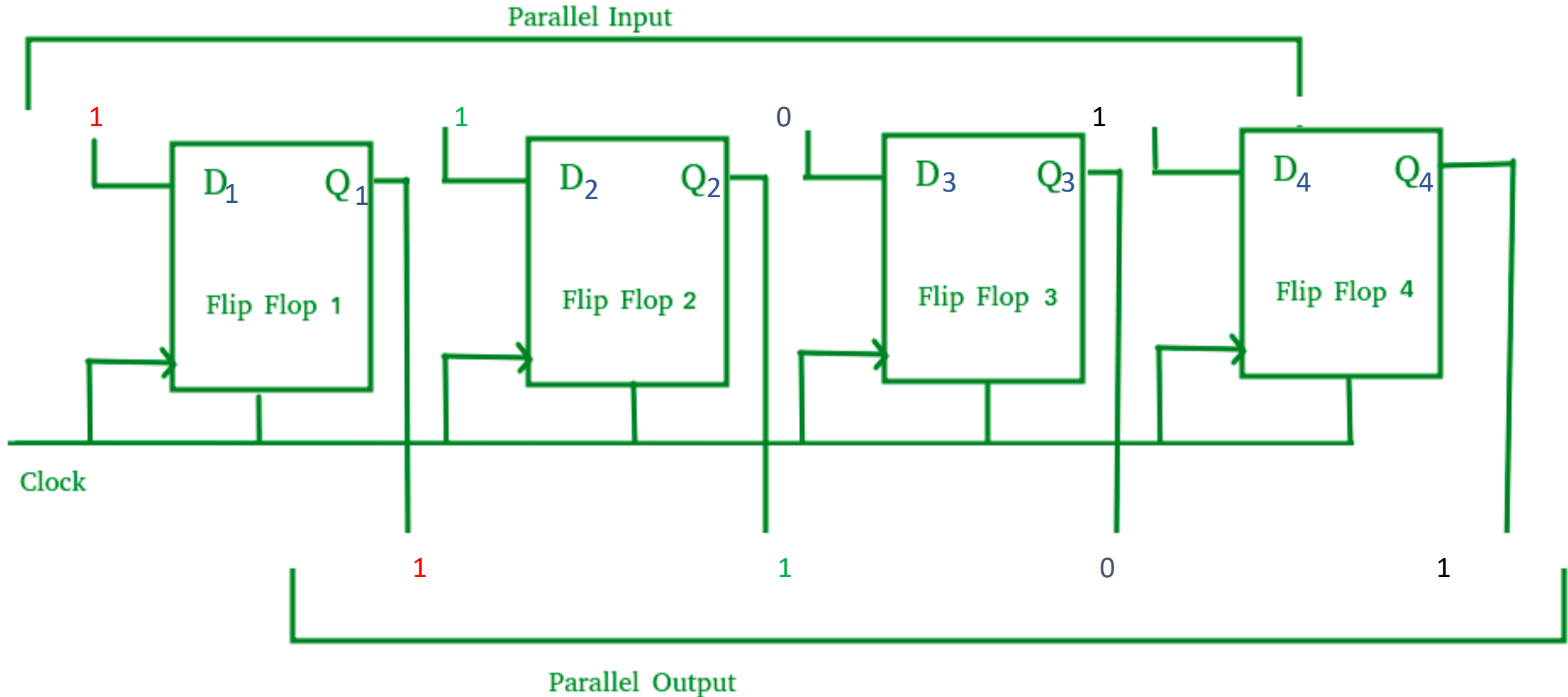
1101

Clk	Q1	Q2	Q3	Q4
Initially	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	1	0	1	0
4	1	1	0	1
5	0	1	1	0
6	0	0	1	1
7	0	0	0	1

1101

1101

Parallel In Parallel Out Register(PIPO)



Parallel In Parallel Out Register(PIPO)

1101

Clk	Q1	Q2	Q3	Q4
Intially	0	0	0	0
1	1	1	0	1

1101 Parallel In Serial Out Register(PISO)

Shift/Load

